CSE 751: Advanced NLP

Contents

1 Neural Network Foundations

1.1 The Perceptron Formula

A perceptron computes $f(wx+b)$, where w is the weight, x is the input, b is the bias, and f is a non-linear activation function such as sigmoid, tanh, or ReLU. The linear combination $wx + b$ is first computed, and then passed through f to introduce non-linearity.

1.2 A Neural Network as a Collection of Perceptrons

A neural network is essentially many perceptrons working together. Each node in a layer is one perceptron. For example, a layer with 3 nodes contains 3 perceptrons, each computing its own $f(wx + b)$.

1.3 How a Neural Network Learns Complex Non-Linear Patterns

A single perceptron applies one small non-linear transformation — a single "bend" in the data. This alone cannot capture complex patterns. However, when hundreds of these bends are stacked across multiple layers, the combination becomes powerful.

Think of it like origami: one fold of paper is simple, but many folds in different directions produce a complex 3D shape. Similarly:

- Each neuron applies a tiny, simple transformation.
- A layer of neurons applies many transformations simultaneously.
- The next layer transforms those results again.
- Layer after layer, the network builds increasingly abstract representations.

For example, in image classification:

- Layer 1 detects edges.
- Layer 2 combines edges into shapes.
- Layer 3 combines shapes into features.
- The final layer produces the class prediction.

No single step is intelligent. The depth and combination of simple transformations is what enables the network to learn complex patterns.

1.4 How Backpropagation Works

Training a neural network follows four steps:

1. **Forward pass:** Data passes through all layers and the network produces a prediction.
2. **Loss calculation:** The prediction is compared to the true label. The loss is a single number representing how wrong the prediction was.

3. **Backpropagation:** Starting from the loss, blame is traced backwards through the network using the **chain rule**. Each weight's contribution to the error is quantified as a gradient.
4. **Gradient descent:** Each weight is nudged slightly in the direction that reduces the loss. The size of the nudge is controlled by the **learning rate**.

This process repeats thousands of times. Over time, the weights converge to values that produce accurate predictions.

1.5 Which Layers Get Their Weights Updated

In a network with 1 input layer, 2 hidden layers, and 1 output layer, weight updates happen in all layers *except* the input layer:

- Output layer weights ✓
- Hidden layer 2 weights ✓
- Hidden layer 1 weights ✓
- Input layer weights × (the input layer has no weights — it only passes data forward)

1.6 Backpropagation vs. Gradient Descent — An Important Distinction

These two terms are often confused but refer to separate steps:

- **Backpropagation:** Computes the gradients — how much each weight contributed to the loss. It traces blame backwards using the chain rule.
- **Gradient descent:** Uses those gradients to actually update the weights — nudging each weight to reduce the loss.

Backpropagation figures out *how much* each weight is to blame. Gradient descent does the actual correction.

1.7 When Backpropagation Begins

Backpropagation cannot start until a loss value exists. The sequence is:

1. Forward pass completes and an output is produced.
2. Loss is calculated by comparing the output to the true label.
3. Backpropagation begins, working backwards from the loss.

1.8 Input Layer Size for Tabular Data

For tabular data, the number of neurons in the input layer equals the number of features. For example, a dataset with 3 columns (features) requires 3 neurons in the input layer — one per feature, all fed in simultaneously.

1.9 Stochastic Gradient Descent: Training Row by Row

For a tabular dataset with 10 rows, training with SGD proceeds as follows:

1. Network processes row 1 (forward pass) → output produced → loss calculated → backpropagation → gradient descent (weights updated).
2. Move to row 2 — repeat the same process with the already-updated weights.
3. Continue through row 10.

This approach — updating weights after every single sample — is called **Stochastic Gradient Descent (SGD)**.

Other approaches include:

- **Batch Gradient Descent:** Process all rows first, then update weights once.
- **Mini-batch Gradient Descent:** Process a small group of rows, then update weights.

1.10 What Is an Epoch

One **epoch** is one complete pass through the entire training dataset. For a 10-row dataset, one epoch means the network has processed all 10 rows once.

2 Recurrent Neural Networks (RNNs)

For sequential data such as sentences, the order of elements matters — something a standard feedforward neural network cannot capture. Unlike tabular data where all features are fed in at once, an RNN processes one element at a time across multiple **time steps**, maintaining a **hidden state** (memory) that carries information from previous steps forward.

2.1 The Problem That Made RNNs Necessary

Before RNNs existed, the dominant tool for pattern recognition was the feedforward neural network. A feedforward network takes a fixed-size input, passes it through several layers, and produces a fixed-size output. It has no memory. Every prediction is made completely from scratch.

This works perfectly well for tasks where the entire input is available all at once and order does not matter — for example, classifying whether a photo contains a cat. But it fails badly the moment you need to process a **sequence**: data that arrives one piece at a time, where the meaning of each piece depends on what came before it.

Consider the task of predicting the next word in a sentence. If you have read "The clouds are in the…," the next word is almost certainly *sky*. But a feedforward network sees only the current word — it has already forgotten "clouds," "are," and "in." A related problem appeared in speech recognition, machine translation, handwriting generation, and any other task where time and order matter.

The specific failure was this: **there was no mechanism to carry information forward across time steps.** Each prediction ignored all prior history. Feedforward networks were, as Karpathy put it, "doomed from the get-go by a fixed number of computational steps."

2.2 The Key Insight That Made RNNs Work

The insight was simple but powerful: **add a loop**. Let the network feed its own output from step t back as an additional input at step $t+1$.

This creates a **hidden state** — a vector of numbers that the network updates at every step and carries forward. The hidden state is a running, compressed summary of everything the network has seen so far. At each new time step, the network looks at two things: the new input arriving right now, and the hidden state from the previous step (which already summarizes all prior inputs). It combines both to produce a new hidden state and a prediction, then passes the hidden state forward.

Critically, the **same set of weights** is reused at every time step. The network does not learn a separate function for step 1, another for step 2, and so on. It learns one general rule: "how to update my running summary given a new input." This makes the architecture work on sequences of *any* length.

2.3 Intuition in Plain English

2.3.1 The core idea: a brain that reads one word at a time and keeps notes

Imagine you are reading a mystery novel, but with a strange rule: you can see only one word at a time on a notepad, and after reading each word you must erase it. The challenge is to understand the story.

To follow along, you would have to keep notes. Each time you read a new word, you update your notes: combine the new word with what you already wrote down, then produce a new, updated set of notes. You carry those notes forward to the next word. The notes are not a full transcript — they are a compressed summary. Maybe your notes say: "butler was mentioned, diamond is missing, it is raining." That summary is your memory.

That is exactly what an RNN does. The notepad is called the **hidden state**. At every step, the RNN reads one new piece of input (say, one word), combines it with what is already in the notepad, writes a new updated notepad, and then uses that notepad to make a prediction — like guessing the next word — before moving on.

2.3.2 Why the same rule every time?

The RNN uses exactly the same rule for updating its notepad at every single step, no matter how long the sequence is. Think of a person who has developed a habit: every time they hear something new, they do the same thing — compare it to what they already know, update their mental model, and move on. The habit does not change; only the content of their memory does.

2.3.3 The loop

What makes an RNN different from a regular neural network is the loop. A regular neural network is like a camera that takes one photo and gives one label. An RNN is like a video camera that watches a movie frame by frame, updating its understanding after every frame, and can predict what comes next at any point.

2.4 Key Assumptions and What Happens When They Are Violated

Assumption 1: Order matters. RNNs are built for data where position in the sequence carries meaning. If your data has no meaningful order — for example, a bag of independent measurements — the recurrence adds no value. A feedforward network would work equally well with less complexity.

Assumption 2: Dependencies are mostly local or medium-range. The vanilla RNN theoretically passes information over unlimited time steps, but in practice the signal from early steps gets diluted as it travels through many updates. When critical dependencies span dozens or hundreds of steps, vanilla RNNs fail. This was documented rigorously by Bengio et al. (1994) and Hochreiter (1991), and it is the primary motivation behind LSTMs and GRUs.

Assumption 3: The same rule applies at every position. Because the same weight matrices are used at every step, the network assumes that the same computation that is useful at step 1 is also useful at step 100. If the statistical properties of the sequence shift dramatically over time, the shared weights may not generalize well across all positions.

Assumption 4: The hidden state is large enough to hold what matters. The hidden state is a fixed-size vector. If the sequence contains more information than fits in that vector, older information will be overwritten. The network has a **memory bottleneck**: it must decide what to keep and what to discard, and with a vanilla RNN, this decision is not learned explicitly — recent inputs simply tend to dominate.

2.5 When the Method Breaks Down

Vanishing and exploding gradients. Training an RNN requires sending an error signal backward through every time step. For a sequence of length T, this error must pass through T matrix multiplications. If the dominant values in the weight matrix are less than 1, repeated multiplication shrinks the error signal exponentially to zero — **gradients vanish** and early time steps receive almost no learning signal. The network becomes unable to learn that something at step 1 matters for the prediction at step 50. Conversely, if the values are greater than 1, the error signal grows exponentially — **gradients explode**, causing wildly unstable updates.

Long-range dependencies. As the Colah blog explains: predicting the word *French* at the end of "I grew up in France... I speak fluent *French*" requires remembering *France* from many steps earlier. The vanilla RNN fails this reliably once the gap between the clue and the prediction grows large.

Long sequences. Processing is sequential — each step depends on the previous one, so no step can be computed until the previous one finishes. This prevents parallelization,

making very long sequences slow to process at training time.

Autoregressive error accumulation. When generating sequences (e.g., generating text one character at a time), the model feeds its own previous output as the next input. Any early mistake compounds forward. Karpathy noted that his character-level LSTM could produce locally coherent output but would lose global structure: it might open a `\begin{proof}` and close it with `\end{lemma}`, because the dependency was too long-range to survive in the hidden state.

2.6 Edge Cases Where RNNs Fail

Very short sequences. On sequences of length one or two, the recurrent connection adds no benefit — there is no history to carry. A simpler feedforward network is just as capable.

Very long sequences. For sequences of thousands of steps, vanilla RNNs fail completely due to vanishing gradients. Even LSTMs degrade significantly. Transformer models, which can attend to any position directly, generally outperform RNNs on very long sequences.

Tasks requiring future context. A standard (unidirectional) RNN at step t only knows about inputs up to step t. For tasks like part-of-speech tagging, where knowing what comes *after* a word helps identify it, a unidirectional RNN is structurally limited. Bidirectional RNNs address this at the cost of requiring the full sequence upfront.

Very small datasets. Like all deep learning models, RNNs are data-hungry. With few training examples, they overfit easily and fail to learn generalizable sequence patterns.

Variable-length sequences in a batch. Because time steps must be processed sequentially, batching sequences of different lengths requires padding shorter sequences to match the longest one. This wastes computation and can introduce noise into the hidden states.

2.7 Known Weaknesses and Failure Modes

Cannot be parallelized across time. Unlike feedforward networks or transformers, RNNs process sequences one step at a time. Step $t{+}1$ cannot begin until step t is complete. This makes RNNs much slower to train on modern GPUs, which are designed for massively parallel computation.

Memory bottleneck. Everything the network needs to remember is compressed into one fixed-size vector. Information from early steps competes with recent information, and older information tends to be overwritten. The network has no explicit mechanism to choose what to keep.

Practical forgetting. Even when gradients do not completely vanish, the learned hidden state in a vanilla RNN tends to reflect recent inputs far more strongly than distant ones. Information is progressively diluted at each step.

Difficult to train. Gradient clipping (to prevent exploding gradients), careful weight initialization, and learning rate tuning are all necessary in practice. Training is less stable than training feedforward networks.

Lack of interpretability. The hidden state is a dense vector of floating-point numbers with no direct human-readable meaning. It is difficult to inspect what the network is "remembering" at any given step.

2.8 RNN Input Representation

The number of neurons in the RNN input layer is determined by how each word is *represented*, not by the number of words in the sentence. The number of words determines the number of time steps, not the input layer size.

Common representations:

- **One-hot encoding:** A vocabulary of 1000 words ⇒ 1000 input neurons. At each time step, the vector for the current word is fed in — all zeros except for a single 1 at the position corresponding to that word. If the vocabulary contains 30 unique words, the input layer will have **30 neurons**.
- **Word embeddings:** Each word compressed into a dense vector of, say, 300 numbers ⇒ 300 input neurons. Modern systems prefer this over one-hot encoding.

At each time step, one word's representation is fed into the same input layer. The input layer size stays fixed regardless of sequence length.

2.9 How an RNN Processes and Trains on a Sequence

2.9.1 Step-by-step sequence processing

For a sentence with 5 words, the RNN processes it one word at a time:

- Step 1: Feed word 1 → hidden layers update memory
- Step 2: Feed word 2 → hidden layers use memory from step 1
- ⋮
- Step 5: Feed word 5 → final output produced

2.9.2 What a "step" means in an RNN

One step corresponds to one word passing through the entire network — input layer → hidden layer 1 → hidden layer 2 → output layer.

The key difference from a standard neural network is that the hidden layers take **two inputs** at each step:

- The current word (from the input layer).
- The hidden state (memory) carried forward from the previous step.

2.9.3 Output layer behavior at intermediate steps

In a many-to-one RNN (e.g., sentiment analysis), the output layer only activates at the **final time step**:

- **Steps 1–4:** The output layer is not activated. Hidden layers process words and update memory.
- **Step 5:** The output layer activates and produces the final prediction (e.g., positive or negative sentiment).

Other RNN output configurations include:

- **Many-to-many:** Output produced at every step (e.g., machine translation, part-of-speech tagging).
- **One-to-many:** One input, many outputs (e.g., image captioning).

2.9.4 Training on sequential data

For sentiment analysis on 10 sentences, each with 5 words, training proceeds as follows for each sentence:

1. **Step 1:** Feed word 1 → hidden layers update memory. No output yet.
2. **Step 2:** Feed word 2 → hidden layers combine current word with memory from step 1. No output yet.
3. **Steps 3–4:** Same process continues.
4. **Step 5:** Feed word 5 → output produced → loss calculated → backpropagation (BPTT) → gradient descent.

Then move to sentence 2 and repeat. Processing all 10 sentences constitutes one epoch.

2.10 End-to-End Trace: Inputs, Mechanism, Outputs

We trace the complete forward pass of a simple RNN on the sentence "the red dog," where the task is to predict the part of speech (DET, ADJ, or NOUN) for each word.

Step 1: Represent the input. Each word is converted into a numerical vector $\mathbf{x}^{(t)}$. For example, a one-hot vector of size $|V|$ (the vocabulary size) where the entry for that word is 1 and all others are 0. Modern systems use learned word embeddings — dense lower-dimensional vectors.

Step 2: Initialize the hidden state. Before the first word, the hidden state $\mathbf{h}^{(0)}$ is set to the zero vector. This represents empty memory at the start of the sequence.

Step 3: Process each word one at a time. At each time step t, the hidden state is updated using:

$$\mathbf{h}^{(t)} = \tanh\big(\mathbf{U}\mathbf{h}^{(t-1)} + \mathbf{W}\mathbf{x}^{(t)}\big)$$

- $\mathbf{h}^{(t)}$ — the new hidden state (updated memory) after seeing the t-th input.
- $\mathbf{h}^{(t-1)}$ — the previous hidden state (accumulated memory from all prior steps).
- $\mathbf{x}^{(t)}$ — the current input vector (the encoding of the current word).

- $\mathbf{U}$ — the recurrent weight matrix. It determines how the previous memory is carried forward and blended into the new state. This is the "memory of the past" connection.
- $\mathbf{W}$ — the input weight matrix. It determines how the new input is incorporated into the hidden state. This is the "perception of the present" connection.
- tanh — the hyperbolic tangent activation function. It squashes all values into the range $[-1, 1]$, keeping the hidden state bounded and introducing non-linearity so the network can learn complex patterns.

This equation says: multiply the previous memory by $\mathbf{U}$, multiply the new input by $\mathbf{W}$, add the two results together, and squeeze them through tanh to produce new memory. The same $\mathbf{U}$ and $\mathbf{W}$ are reused at every single time step.

Step 4: Compute the output at each step. After computing the hidden state, an output prediction is produced:

$$\mathbf{o}^{(t)} = \text{softmax}\big(\mathbf{V}\mathbf{h}^{(t)}\big)$$

- $\mathbf{o}^{(t)}$ — the output at step t: a probability distribution over all possible labels (here: DET, ADJ, NOUN).
- $\mathbf{V}$ — the output weight matrix. It maps from the hidden state space to the label space.
- softmax — converts raw scores into probabilities that sum to 1. The label with the highest probability is the predicted class.

This equation says: take the current memory $\mathbf{h}^{(t)}$, multiply by $\mathbf{V}$ to get a score for each label, then pass through softmax to get a proper probability distribution. The predicted label is $\arg\max \mathbf{o}^{(t)}$.

Step 5: Compute the loss. The cross-entropy loss at each step compares the predicted distribution to the true label:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

- L_{CE} — the cross-entropy loss for one time step. It measures how wrong the prediction is.
- $\hat{\mathbf{y}}_t$ — the predicted probability distribution over labels at step t.
- $\mathbf{y}_t$ — the true label (as a one-hot vector; 1 at the correct class, 0 everywhere else).
- w_{t+1} — the index of the correct next label.
- $-\log(\cdot)$ — the negative logarithm. It equals 0 when the probability assigned to the correct label is 1 (perfect prediction) and grows toward infinity as the assigned probability approaches 0.

The loss is small when the model correctly assigns high probability to the right label and large when it does not. The total training loss is the average across all steps: $L = \frac{1}{T}\sum_{t=1}^{T} L_{CE}^{(t)}$.

Step 6: Backpropagation Through Time (BPTT). To adjust the weights, we compute how much each weight — $\mathbf{U}$, $\mathbf{W}$, $\mathbf{V}$ — contributed to the total loss. Because the same weights appear at every time step, their gradient is the sum of contributions from all steps. The error signal flows backward through the unrolled computation graph from step T back to step 1. This is BPTT: standard backpropagation applied to the unrolled RNN.

Step 7: Weight update. The weights $\mathbf{U}$, $\mathbf{W}$, $\mathbf{V}$ are nudged in the direction that reduces the loss (gradient descent). Because these weights are shared across all time steps, one update to $\mathbf{U}$ affects the computation at every step simultaneously.

"Same across all time steps" describes what happens during one pass through a single sequence. When the RNN reads the words "the red dog," it uses the exact same $\mathbf{U}$, $\mathbf{W}$, and $\mathbf{V}$ at step 1, step 2, and step 3. The weights do not change while it is reading that sentence. Think of it like a recipe: the same recipe is followed for every word in the sequence.

"Nudged" describes what happens after the full sequence has been processed and the network checks how wrong its predictions were. At that point — not during the reading, but after — it adjusts $\mathbf{U}$, $\mathbf{W}$, and $\mathbf{V}$ slightly to do better next time. The next time the network reads a sentence, it again uses those updated weights at every step. The recipe gets tweaked between sequences, not mid-sequence.

Concrete trace through "the red dog":

1. $t = 1$, input = "the." $\mathbf{h}^{(1)} = \tanh(\mathbf{U}\mathbf{h}^{(0)} + \mathbf{W}\mathbf{x}_{\text{the}})$. Since $\mathbf{h}^{(0)} = \mathbf{0}$, this is just $\tanh(\mathbf{W}\mathbf{x}_{\text{the}})$. Output $\mathbf{o}^{(1)}$ gives probabilities over {DET, ADJ, NOUN}. Predicted label: DET.
2. $t = 2$, input = "red." $\mathbf{h}^{(2)} = \tanh(\mathbf{U}\mathbf{h}^{(1)} + \mathbf{W}\mathbf{x}_{\text{red}})$. The hidden state $\mathbf{h}^{(1)}$ already encodes information about "the." The network knows a determiner came before this word. Predicted label: ADJ.
3. $t = 3$, input = "dog." $\mathbf{h}^{(3)} = \tanh(\mathbf{U}\mathbf{h}^{(2)} + \mathbf{W}\mathbf{x}_{\text{dog}})$. The hidden state $\mathbf{h}^{(2)}$ encodes information about both "the" and "red." The network knows a DET and an ADJ have appeared, so a NOUN is expected. Predicted label: NOUN.

At every step, the prediction is informed by the entire prior history of the sequence, not just the current word. This is the core power of the recurrence.

2.11 Backpropagation Through Time (BPTT) in Detail

After the loss is computed at the final step, backpropagation traces blame backwards through all time steps. Since the **same weights are shared across all time steps**, gradient descent collects gradients from all steps, sums them, and updates the shared weights once:

$$\Delta W = G_1 + G_2 + G_3 + G_4 + G_5$$

The blame propagates as follows (using a 5-step sequence as an example):

- Step 5: output layer weights + hidden state at step 5 receive blame directly from the loss.
- Step 4: hidden state at step 4 influenced step 5's memory, so it receives blame too.
- Step 3: blame traces back further.
- Steps 2–1: blame continues propagating back through the full sequence.

2.11.1 Gradients multiply during backpropagation; sum during weight update

There is an important distinction between what happens during backpropagation versus gradient descent:

- **During BPTT (chain rule):** Gradients are *multiplied* as blame propagates backwards through time steps.
- **During gradient descent:** The gradients from all steps are *summed* to update the shared weights.

The cumulative gradient reaching each time step is:

$$\begin{aligned}
&\text{Step 5:} && G_5 && (1)\\
&\text{Step 4:} && G_5 \times G_4 && (2)\\
&\text{Step 3:} && G_5 \times G_4 \times G_3 && (3)\\
&\text{Step 2:} && G_5 \times G_4 \times G_3 \times G_2 && (4)\\
&\text{Step 1:} && G_5 \times G_4 \times G_3 \times G_2 \times G_1 && (5)
\end{aligned}$$

This is exactly why the **vanishing gradient problem** occurs. If each local gradient $G_i < 1$, multiplying many of them together makes the gradient at step 1 nearly zero — the network effectively forgets information from early steps. This limitation motivated the development of **Long Short-Term Memory (LSTM)** networks.

2.12 Cost and Tradeoffs

What you give up:

Speed and parallelism. Each step depends on the previous one, so the computation cannot be parallelized across the time dimension. Transformer models process entire sequences in parallel, making them orders of magnitude faster to train on modern GPU hardware. This is the most significant practical cost of RNNs.

Long-term memory. Vanilla RNNs are fundamentally limited in how far back they can effectively remember, due to vanishing gradients. Even LSTMs — designed to address this — degrade on sequences of thousands of steps. Transformers handle arbitrary-range dependencies with no structural limit.

Training stability. Exploding gradients require gradient clipping. Vanishing gradients require careful initialization and sometimes truncated BPTT. Training is more finicky than training feedforward networks.

Interpretability. The hidden state is a dense, opaque vector. There is no easy way to inspect what the network is remembering at any given step or why it made a particular prediction.

What you gain:

Variable-length sequences. The same three weight matrices ($\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$) handle sequences of any length. No truncation, no fixed window. This is the core practical advantage.

Sequential memory. Each prediction is conditioned on the entire prior history in a continuous and learned way — not a hard-coded window. The network learns what to remember and how to compress it, entirely from data.

Parameter efficiency. The same weight matrices are reused at every step. Compared to an unrolled feedforward network of equivalent depth and sequence length, the RNN uses far fewer parameters.

Natural fit for sequential data. The architecture is structurally aligned with the problem. As the Jurafsky & Martin textbook notes, language is inherently temporal — a sequence that unfolds in time. RNNs model this directly rather than forcing sequential data into a fixed-size input.

Remarkable practical results. Karpathy demonstrated that a character-level LSTM trained on raw text can produce plausible Shakespeare, syntactically valid LaTeX, and structurally reasonable C code — all by predicting the next character. The hidden state implicitly learns complex syntactic and stylistic structure without any explicit supervision.

2.13 The Role of Matrices U, W, and V in an RNN

Think of an RNN as a student reading a story one word at a time, keeping a notepad.

At every step, two things arrive:

- A **new word** (the current input, x)
- The **notepad** from the last step (the previous hidden state, h)

The student must combine them into a **new, updated notepad** (the new hidden state). Then look at the notepad and **guess the label**. The three matrices are the student's three abilities.

2.13.1 W — "How do I read new words?"

When a new word arrives, **W decides how much that word contributes to the notepad.**

Every word is represented as a vector of numbers (e.g., a one-hot or embedding vector). W multiplies that vector to transform it into a format the hidden state can absorb.

Think of W as the student's *reading ability* — the skill of looking at a new word and figuring out what is important about it for memory.

W is the connection between: input → hidden state.

2.13.2 U — "How much do I trust my old notes?"

When the old notepad arrives, **U decides how much of the previous memory survives and gets mixed in.**

U multiplies the old hidden state to transform it into a contribution for the new hidden state.

Think of U as the student's *memory-integration skill* — the ability to look at yesterday's notes and decide what is still relevant today.

U is the connection between: old hidden state → new hidden state.

2.13.3 Putting W and U Together: The Core Formula

$$h^{(t)} = \tanh\big(Uh^{(t-1)} + Wx^{(t)}\big)$$

In plain English:

- $Wx^{(t)}$ = "Here is what the new word tells me"
- $Uh^{(t-1)}$ = "Here is what I already remembered"
- Add them → "My combined understanding right now"
- Pass through tanh → "Compress it so numbers don't explode"
- Result = **new notepad** $h^{(t)}$

Both W and U are learned during training. The network learns: *how much should I trust old memory vs. new input?* That balance is baked into U and W.

2.13.4 V — "How do I make a prediction from my notes?"

After the notepad (hidden state) is updated, **V translates the notepad into a prediction.**

The notepad is a dense vector of numbers — not human-readable. V maps that memory space into the label space (DET, ADJ, NOUN).

Think of V as the student's *answering skill* — the ability to look at the notepad and say "given everything I know, my best guess for this word's tag is NOUN."

$$o^{(t)} = \text{softmax}\big(Vh^{(t)}\big)$$

V is the connection between: hidden state → output.

2.13.5 Why Three Separate Matrices?

Because they do three fundamentally different jobs:

Matrix	Job	Connects
W	Process new input	input $\rightarrow$ hidden
U	Carry old memory forward	hidden $\rightarrow$ hidden
V	Make predictions	hidden $\rightarrow$ output

All three matrices are **shared across every time step**. They do not change while reading a sentence — only after the loss is computed via BPTT. During a single forward pass, they are fixed, like a fixed habit applied to every word.

2.14 Shape Compatibility in Matrix Multiplication and How W's Values Are Determined

2.14.1 How W's Values Are Determined

W starts random. Before training begins, every element of W is initialized to a small random number (drawn from a Gaussian or uniform distribution). The network knows nothing yet.

Then, after each forward pass and loss calculation, backpropagation computes $\partial L/\partial W$ — how much each element of W contributed to the error. Gradient descent then nudges every element slightly:

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial L}{\partial W}$$

This repeats thousands of times. Over training, W's values settle into numbers that make $Wx^{(t)}$ produce useful contributions to the hidden state. The network learns W — you do not set it by hand.

2.14.2 Matrix Multiplication Shape Rules

For two matrices A and B, the product AB is only legal if:

The number of columns in A equals the number of rows in B.

If A is shape $[m \times n]$ and B is shape $[n \times p]$, then AB gives shape $[m \times p]$. The inner dimensions must match. The outer dimensions become the result shape.

2.14.3 Applying This to W, U, and V

Fix concrete numbers:

- Vocabulary size = 30 $\Rightarrow$ input vector x has shape $[30 \times 1]$
- Hidden state size = 5 $\Rightarrow$ h has shape $[5 \times 1]$
- Output classes = 3 (DET, ADJ, NOUN) $\Rightarrow$ output o has shape $[3 \times 1]$

$Wx^{(t)}$ **must produce** $[5 \times 1]$**:** x is $[30 \times 1]$, so W needs 30 columns and 5 rows. W is $[5 \times 30]$.

$$[5 \times 30] \cdot [30 \times 1] = [5 \times 1]\checkmark$$

$Uh^{(t-1)}$ **must also produce** $[5 \times 1]$**:** $h^{(t-1)}$ is $[5 \times 1]$, so U needs 5 columns and 5 rows. U is $[5 \times 5]$.

$$[5 \times 5] \cdot [5 \times 1] = [5 \times 1]\checkmark$$

The addition $Uh^{(t-1)} + Wx^{(t)}$ is $[5 \times 1] + [5 \times 1] = [5 \times 1]$ (shapes must be identical for element-wise addition). ✓

$Vh^{(t)}$ **must produce** $[3 \times 1]$**:** $h^{(t)}$ is $[5 \times 1]$, so V needs 5 columns and 3 rows. V is $[3 \times 5]$.

$$[3 \times 5] \cdot [5 \times 1] = [3 \times 1]\checkmark$$

2.14.4 Summary of Matrix Shapes

Matrix	Shape	Values Determined By
W	[hidden_size × input_size]	You set the sizes; values learned via backprop
U	[hidden_size × hidden_size]	You set hidden_size; values learned via backprop
V	[output_size × hidden_size]	You set the sizes; values learned via backprop

The **sizes** are fixed by your architecture choices. The **values inside** those matrices are what training learns.

2.15 Why U Must Always Be a Square Matrix

Yes, U must always be square — and this is not a convention but a shape requirement forced by the recurrence itself.

U multiplies $h^{(t-1)}$, which has shape [hidden_size × 1], and must produce something of shape [hidden_size × 1] so it can be added with $Wx^{(t)}$. The input and output of U must be the same size — hidden_size — so U is always [hidden_size × hidden_size].

2.16 Initialization of U

Yes. U is initialized randomly, just like W and V, then learned through backpropagation the same way.

2.17 How the Hidden State h Is Shaped and Valued

2.17.1 Shape

You decide it. The hidden size is a hyperparameter you pick before training (e.g., 5, 128, 512). This fixes h as [hidden_size × 1] for the entire model. Larger hidden size = more memory capacity, but more parameters and more computation.

2.17.2 Values

h is never learned directly. It is always *computed* at each step:

$$h^{(t)} = \tanh\big(Uh^{(t-1)} + Wx^{(t)}\big)$$

- $h^{(0)}$ is set to the zero vector — no memory before the sequence starts.
- $h^{(1)}$ is computed from $h^{(0)}$ and the first input.
- $h^{(2)}$ is computed from $h^{(1)}$ and the second input.
- And so on.

h is not a parameter of the model. It is a result — freshly recomputed at every time step during the forward pass, thrown away after the sequence is done, and rebuilt from scratch for the next sequence. What gets learned are U and W, whose values shape what h becomes.

3 Bidirectional RNN

3.1 Motivation and Core Idea

A regular RNN reads a sentence from left to right, one word at a time, keeping a running notepad (the hidden state) as it goes. But it can only see what came before. When it is processing word #3, it has no idea what words #4, #5, #6 are going to say.

That causes real problems. Take this sentence:

> *"I went to the **bank** to fish."*

versus

> *"I went to the **bank** to deposit money."*

The word "bank" appears in the same position in both sentences. When a regular RNN gets to "bank," it has only seen *"I went to the"* — which tells it nothing. The clue that separates the two meanings comes **after** the word "bank." A regular RNN is blind to that.

A **Bidirectional RNN** fixes this by running two regular RNNs at the same time on the same sentence:

- **RNN #1** reads left → right (past to future).
- **RNN #2** reads right ← left (future to past), starting from the last word and working backward.

At every word position, you have two hidden states — one carrying memory of everything *before* this word, and one carrying memory of everything *after* this word. These are concatenated into one combined vector, which is used to make the prediction for that position.

Think of it like two detectives investigating the same crime — one starts from the beginning, one starts from the end. Neither one announces a verdict on their own. They meet in the middle, share their notes, and then together decide: guilty or not guilty.

One important limitation: bidirectional RNNs need the **whole sequence available upfront**. The backward RNN has to start from the end, so it cannot be used when input arrives word by word in real time. It is only practical when the full input is available ahead of time — like reading a sentence for translation or POS tagging.

3.2 Training on a Fixed-Length Dataset

Consider 10 sentences, each with 5 words. Each sentence is one training example processed one at a time. Here is what happens for one sentence, say *"The cat sat on mat"*:

Step 1 — Forward RNN runs left to right. It reads: The $\rightarrow$ cat $\rightarrow$ sat $\rightarrow$ on $\rightarrow$ mat, updating its hidden state at each step:

$$h_f^1 \rightarrow h_f^2 \rightarrow h_f^3 \rightarrow h_f^4 \rightarrow h_f^5$$

By h_f^5, this RNN has seen the whole sentence from the left.

Step 2 — Backward RNN runs right to left. It reads: mat $\rightarrow$ on $\rightarrow$ sat $\rightarrow$ cat $\rightarrow$ The, updating its own hidden state:

$$h_b^5 \rightarrow h_b^4 \rightarrow h_b^3 \rightarrow h_b^2 \rightarrow h_b^1$$

Note: h_b^5 is the backward RNN's *first* computation (starts at "mat"). h_b^1 is its *last*.

Step 3 — Combine at each position. At every word position t, concatenate the two hidden states:

$$\text{position } t: \quad [h_f^t \; ; \; h_b^t]$$

Step 4 — Compute output and loss. This depends on the task. For word-level labeling (e.g., POS tagging), compute an output at all 5 positions and average the losses. For sentence-level classification (e.g., sentiment), use only h_f^5 and h_b^1 — the final states of each RNN — concatenate them, and compute one loss.

Step 5 — Backpropagation through both RNNs. Gradients flow backward through time in each RNN separately:

- Through the **forward RNN**: from position 5 back to position 1 (standard BPTT).
- Through the **backward RNN**: from position 1 back to position 5 (same BPTT logic, reversed direction).

Each RNN updates its own separate weights: W_f, U_f for the forward one, W_b, U_b for the backward one, plus the shared output weight V.

The same process repeats for all 10 sentences — that is one epoch. This repeats for many epochs until the loss converges.

Key point: Both RNNs never talk to each other during the forward pass. They run independently and only meet at the concatenation step. Backpropagation also treats them independently — gradients for the forward RNN never flow into the backward RNN's weights, and vice versa.

3.3 Output Responsibility of Each RNN in a Bidirectional Architecture

Neither RNN produces "positive or negative" on its own. That is not how it works.

Both RNNs only produce **hidden states**. That is their only job.

- Forward RNN finishes → produces h_f^5 (summarizes the sentence left-to-right).
- Backward RNN finishes → produces h_b^1 (summarizes the sentence right-to-left).

These two are concatenated: $[h_f^5 \; ; \; h_b^1]$

Then this combined vector is fed into one single output layer, which produces one prediction — positive or negative.

There is only **one output**, not two. The backward RNN is not a classifier. It is a context builder. Its hidden state carries right-to-left information, hands it to the output layer, and the output layer makes the final call using both directions together.

3.4 Concatenation as a Column Vector, Not a Matrix

The hidden states h_f^5 and h_b^1 are each a single column vector — the hidden state at one specific time step. Concatenation means stacking them vertically:

$$\begin{bmatrix} h_f^5 \\ h_b^1 \end{bmatrix}$$

So if both have shape $[2 \times 1]$, the result is $[4 \times 1]$ — not $[2 \times 2]$.

Making it 2×2 would turn it into a matrix, which would break the output layer's matrix multiplication that comes after. The result must stay a single column vector, just longer.

3.5 Hidden State Dimensionality: Vector vs. Matrix

h is a vector in a vanilla RNN as well. That is the standard case for a single sequence.

h can technically become a matrix in one situation: **batch processing**. If you feed multiple sentences through the RNN at the same time (say a batch of 8 sentences), you stack their individual h vectors side by side and get a matrix of shape [hidden_size $\times$ 8]. But that is just an implementation convenience — conceptually, each sentence still has its own h vector. You are simply processing several of them in parallel.

- Single sequence: h is always a vector.
- Batched sequences: h becomes a matrix, but only as a computational shortcut.

3.6 Hidden State Notation in RNNs: h_t vs $h_1, h_2, \ldots$

h_t is the general symbolic notation for the hidden state at time step t. When referring to specific steps, the subscript becomes concrete:

- After processing word 1 → h_1 $\quad (t = 1)$
- After processing word 2 → h_2 $\quad (t = 2)$

- After processing word 5 $\rightarrow h_5 \quad (t = 5)$

In a stacked RNN with multiple hidden layers, each layer maintains its own hidden state at every time step. For two hidden layers:

- h_t^1 = hidden state of layer 1 at time step t
- h_t^2 = hidden state of layer 2 at time step t

When people write h_t without a layer superscript, they typically refer to the *top* hidden layer's output, since that is what feeds into the output layer.

3.7 Why the Top Hidden Layer — Not the Output Layer — Is Called h_t

h_t is specifically the recurrent memory: the vector carried forward to the next time step. That is its defining role.

The output layer's vector (commonly written o_t or $\hat{y}_t$) is a prediction. It is produced, used to compute loss, and discarded. It does **not** feed back into the next time step and plays no role in the recurrence.

The hidden layers are the ones performing the recurrence:

$$h_t^1 = \tanh\big(W^1 x_t + U^1 h_{t-1}^1\big)$$
$$h_t^2 = \tanh\big(W^2 h_t^1 + U^2 h_{t-1}^2\big)$$
$$o_t = \text{softmax}\big(V h_t^2\big)$$

The top hidden layer is referred to as h_t because it is the most summarized representation of everything the network has seen, and it is directly connected to the output. The output vector is a different object with a different role: it is a classification result, not a memory state.

3.8 Each Hidden Layer Maintains Its Own Recurrent Memory

Each hidden layer carries its own previous state forward independently, not just the top layer:

- Layer 1 carries h_{t-1}^1 forward and uses it when processing the current word.
- Layer 2 carries h_{t-1}^2 forward and uses it when processing the current word.

The top hidden layer's state is colloquially called h_t because it is the final summarized memory before the output, but both layers have their own independent recurrence running in parallel.

3.9 How Deeper Hidden Layers Receive Current Word Information

In a stacked RNN, only the first hidden layer is directly connected to the input x_t. Deeper layers do not receive x_t directly — they do not need to.

Layer 1 processes x_t and produces h_t^1, which already encodes information about the current word. Layer 2 takes h_t^1 as its input, so x_t's information reaches it indirectly through layer 1's output.

$$h_t^1 = \tanh\big(W^1 x_t + U^1 h_{t-1}^1\big) \quad \text{(receives actual word vector)}$$
$$h_t^2 = \tanh\big(W^2 h_t^1 + U^2 h_{t-1}^2\big) \quad \text{(receives layer 1's output, not } x_t\text{)}$$

For layer 2, h_t^1 plays the same role that x_t played for layer 1 — it is the current input. The current word's information is already encoded within it. x_t does not skip layers; it flows upward through the network the same way it does in a standard feedforward network.

4 Long Short-Term Memory (LSTM)

4.1 The Problem That Made LSTMs Necessary

As shown in Section 2.11, the vanilla RNN suffers from vanishing gradients because the error signal must pass through a long chain of matrix multiplications and tanh activations. But there is a second, separate problem that is more architectural.

The vanilla RNN's hidden state h is asked to do two conflicting jobs at the same time: (1) carry long-term information across many steps, and (2) produce a useful working answer right now. These two goals fight each other. To make a good prediction at the current step, the hidden state gets heavily rewritten – and in doing so, it erases information that earlier steps had carefully stored. The network has no way to hold on to something important from step 2 while making a prediction at step 47.

LSTMs fix this by introducing a second, separate vector called the **cell state**. The cell state handles long-term memory. The hidden state handles the current working output. Two jobs, two separate tools.

4.2 The Key Insight: Two Memories Instead of One

The key structural change in an LSTM is splitting memory into two parts:

- c_t – the **cell state**. This is the long-term memory. It travels through the entire sequence and is updated gently, mostly through addition. Think of it as a conveyor belt: things only get added to it or erased from it when the network deliberately decides to do so.
- h_t – the **hidden state**. This is the short-term working output. It is produced at every step and passed forward, just like in the vanilla RNN. It is also the vector that feeds into the output layer to make a prediction.

The key to this design is that the cell state update is *additive*. Instead of being pushed through repeated matrix multiplications and tanh at every step, information flows through the cell state with very little distortion. This is what stops gradients from vanishing during backpropagation.

4.3 Intuition in Plain English

4.3.1 Two notebooks

In Section 2.3, the RNN was described as a student keeping one notepad. An LSTM gives that student a second tool: a **filing cabinet**.

- The **filing cabinet** (c_t) holds long-term facts: who the subject of the sentence is, whether a verb is still expected, which language is being discussed.
- The **working notepad** (h_t) holds what the student is thinking right now – information useful for the current prediction.

At every word, the student follows three rules, controlled by learned *gates*:

1. **What to throw away?** Look at the filing cabinet and erase anything no longer needed. (Forget gate.)
2. **What to file away?** Look at the current word and decide what is worth keeping in the filing cabinet for later. (Input gate + candidate.)
3. **What to write on the working notepad?** Read the filing cabinet and extract only what is needed for the current answer. (Output gate.)

4.3.2 What is a gate?

A gate is a filter. It is a vector of numbers between 0 and 1, produced by a sigmoid function applied to a linear combination of the current input and the previous hidden state. When you multiply a gate element-by-element with another vector (called *element-wise multiplication*, written $\odot$), values near 1 pass through almost unchanged, while values near 0 are nearly erased. The network learns the gate weights during training, which means it learns *what to keep and what to forget* directly from data – something the vanilla RNN could never do explicitly.

4.4 Key Assumptions and What Happens When They Are Violated

LSTMs keep the same first and third assumptions as vanilla RNNs (order matters; the same rule applies at every step). Two assumptions change:

Assumption 2: Long-range dependencies can be learned. The LSTM is specifically designed to retain information over long sequences. The cell state can, in principle, carry a signal from step 1 all the way to step 100 without it being overwritten. However, this relies on the forget gate learning to stay near 1 for important information. If training does not produce this behavior – for instance due to very long sequences or small datasets – the dependency can still be lost.

Assumption 4: Two fixed-size vectors are large enough. The LSTM has a cell state c and a hidden state h, both of fixed size. There is still a memory capacity limit. If a sequence contains more information than fits in these vectors, older information will still be overwritten. The difference from vanilla RNNs is that the overwriting is now *deliberate and learned*, rather than accidental.

4.5 Edge Cases Where LSTMs Fail Differently From Vanilla RNNs

Very long sequences. As noted in Section 2.6, even LSTMs degrade on sequences of thousands of steps. The forget gate can leak small amounts of information at every step; over hundreds of steps, this compounds into meaningful loss.

Forget gate initialized incorrectly. The forget gate is critical. If its weights start too close to zero (producing outputs near 0), the network forgets everything at every step before it has learned not to – essentially collapsing into a vanilla RNN for the first few training iterations. In practice, forget gate biases are often initialized to 1 rather than 0 to avoid this.

Tasks requiring future context. An LSTM is still unidirectional by default. It cannot look ahead. The same limitation described in Section 2.6 for vanilla RNNs applies here. A Bidirectional LSTM (Section 4) is required for those tasks.

4.6 Known Weaknesses and Failure Modes

Roughly four times more parameters. An LSTM has four weight matrices (one per gate/candidate operation) instead of one. With a vocabulary of 30 and hidden size of 5, the vanilla RNN had 190 parameters; an LSTM with the same sizes has approximately 700. This means the LSTM needs more training data to fit well and is more prone to overfitting on small datasets.

More computation per step. Each LSTM step computes six equations instead of one. Each of those equations involves matrix multiplication, meaning the LSTM is roughly four times slower per step than a vanilla RNN of the same hidden size.

Still cannot be parallelized across time. Step t still depends on step $t-1$. The sequential bottleneck described in Section 2.7 remains – the cell state does not change this.

Harder to diagnose. When a vanilla RNN makes a wrong prediction, there is one hidden state to inspect. In an LSTM, there are six intermediate computations per step. Figuring out which gate made the wrong call is much harder.

4.7 How an LSTM Processes a Sequence

The overall flow through a sequence is identical to what was described in Section 2.9 – one element at a time, left to right, producing a new state at each step. The difference is that each step now has *two* state vectors carried forward: h_t (the hidden state) and c_t (the cell state). Both start as the zero vector. Both are discarded when the sequence ends and rebuilt from scratch for the next sequence.

At every step t, the LSTM runs six equations in order. First it computes the three gate/candidate values using only the current input x_t and the previous hidden state h_{t-1}. Then it updates the cell state. Then it uses the updated cell state to produce the new hidden state. Finally, the hidden state produces the output prediction exactly as in the vanilla RNN:

$$\hat{o}^{(t)} = \text{softmax}(V h_t)$$

The output equation has not changed. Only the way h_t is produced has changed.

4.8 End-to-End Trace: The Six Equations

We trace one time step of an LSTM on the same architecture used in Section 2.10: vocabulary size = 30, hidden size = 5, output classes = 3 (DET, ADJ, NOUN). At step t, the inputs are x_t (shape $[30 \times 1]$), h_{t-1} (shape $[5 \times 1]$), and c_{t-1} (shape $[5 \times 1]$).

A note on notation. The four gate and candidate equations below follow Colah's blog: h_{t-1} and x_t are *concatenated* into one longer vector $[h_{t-1},\, x_t]$, and a single weight matrix W is applied to it. This is equivalent to the split form used in some textbooks, where two separate matrices are written: $U \cdot h_{t-1} + W_{\text{split}} \cdot x_t$. The single W in the concatenation form is just $[U \mid W_{\text{split}}]$ stitched together side by side into one bigger matrix – multiplying it against the concatenated vector produces the exact same result. Either form is correct; only the notation differs.

4.8.1 Step 1: Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1},\, x_t])$$

- f_t – the forget gate vector, shape $[5 \times 1]$. Each number is between 0 and 1.
- σ – the sigmoid function. It takes any number and squashes it into the range $(0, 1)$.
- W_f – the forget gate's single weight matrix, shape $[5 \times 35]$. It is applied to the full concatenated vector.
- $[h_{t-1},\, x_t]$ – the concatenation of the previous hidden state ($[5 \times 1]$) and the current input ($[30 \times 1]$), stacked into one vector of shape $[35 \times 1]$.

Look at where the network has been (h_{t-1}) and what just arrived (x_t), combine them with learned weights, and pass through sigmoid to produce a value between 0 and 1 for every slot in the cell state. A value of 1 means "keep this slot completely." A value of 0 means "erase this slot completely."

4.8.2 Step 2: Cell State Candidate

$$g_t = \tanh(W_g \cdot [h_{t-1},\, x_t])$$

- g_t – the candidate vector: proposed new information to write into the cell state, shape $[5 \times 1]$. Values range from -1 to 1.
- tanh – the hyperbolic tangent function. Squashes values into $(-1, 1)$, allowing both positive and negative contributions.
- W_g – the candidate's weight matrix, shape $[5 \times 35]$, applied to the concatenated vector $[h_{t-1},\, x_t]$.

This is the same kind of computation as the vanilla RNN's hidden state update from Section 2.10, but here it produces a *proposal* rather than a final answer. It asks: "Given everything seen so far and what just arrived, what new information might be worth remembering?" The next gate decides which parts of this proposal actually get written.

4.8.3 Step 3: Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1},\, x_t])$$

- i_t – the input gate vector, shape $[5 \times 1]$. Numbers between 0 and 1.
- W_i – the input gate's weight matrix, shape $[5 \times 35]$, applied to the concatenated vector $[h_{t-1},\, x_t]$.

The forget gate decided what to erase from old memory. The input gate now decides which parts of the candidate g_t are actually worth writing into the cell state. A value of 1 means "yes, file this away." A value of 0 means "ignore this candidate slot." Together, Steps 2 and 3 implement a learned write operation to the filing cabinet.

4.8.4 Step 4: Cell State Update

$$c_t = (i_t \odot g_t) + (f_t \odot c_{t-1})$$

- c_t – the updated cell state (filing cabinet) after this step, shape $[5 \times 1]$.
- $i_t \odot g_t$ – the new information to add: the candidate g_t filtered by the input gate i_t.
- $f_t \odot c_{t-1}$ – the surviving old information: the previous cell state c_{t-1} filtered by the forget gate f_t.
- $\odot$ – element-wise multiplication (Hadamard product). Multiply corresponding elements of two same-sized vectors. Position i of the result is just $(i_t)_i \times (g_t)_i$.
- c_{t-1} – the cell state from the previous step.

Take the old filing cabinet (c_{t-1}), erase what the forget gate said to erase, then add what the input gate approved from the new candidates. This operation is an *addition*, not a matrix multiplication through tanh. This matters enormously for training: when gradients flow backward through this equation from c_t to c_{t-1}, they only pass through the forget gate values f_t – not through repeated weight matrix multiplications. This is the gradient highway that prevents vanishing gradients.

4.8.5 Step 5: Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1},\, x_t])$$

- o_t – the output gate vector, shape $[5 \times 1]$. Numbers between 0 and 1.
- W_o – the output gate's weight matrix, shape $[5 \times 35]$, applied to the concatenated vector $[h_{t-1},\, x_t]$.

The cell state now holds the updated long-term memory. The output gate asks: "Of everything stored in the filing cabinet right now, which parts are actually useful for the current working answer?" Not all long-term memory is relevant at every step. The output gate learns to select only the relevant parts.

4.8.6 Step 6: New Hidden State

$$h_t = o_t \odot \tanh(c_t)$$

- h_t – the new hidden state (working notepad) for this step, shape $[5 \times 1]$.

- $\tanh(c_t)$ – the cell state squeezed into $(-1, 1)$ to keep values bounded.
- $o_t \odot \tanh(c_t)$ – the cell state, filtered by the output gate: only the relevant parts survive.

Squeeze the filing cabinet through tanh to normalize the values, then use the output gate to keep only what matters right now. The result h_t is the new working notepad. It gets passed forward to the next step as h_{t-1}, and it also feeds into the output equation $\hat{o}^{(t)} = \text{softmax}(Vh_t)$ to make the current prediction. The hidden state still plays exactly the same external role as in a vanilla RNN – only how it is produced has changed.

4.9 BPTT in LSTM: The Gradient Highway

Section 2.11 showed that in vanilla BPTT, the gradient reaching step 1 in a 5-step sequence is the product $G_5 \times G_4 \times G_3 \times G_2 \times G_1$. If each $G_i < 1$, this product vanishes to zero.

In an LSTM, the gradient with respect to the cell state follows a different path. Going backward from c_t to c_{t-1} using the update equation:

$$c_t = (i_t \odot g_t) + (f_t \odot c_{t-1})$$

the gradient of c_t with respect to c_{t-1} is simply f_t – the forget gate values. There is no matrix multiplication, no tanh squashing. If the forget gate learns to output values near 1 for slots it wants to preserve, the gradient for those slots flows backward nearly unchanged, step after step. Early time steps receive a strong, uncorrupted learning signal for the first time.

This does not mean gradients never vanish in an LSTM. If the forget gate consistently outputs values near 0 (which erases memory), the gradients also get erased in the same direction. But unlike the vanilla RNN, the LSTM can *learn* to keep the forget gate near 1 for important information – and when it does, early time steps receive a gradient.

4.10 The Weight Matrices in an LSTM

The vanilla RNN had three weight matrices: W (input to hidden), U (hidden to hidden), and V (hidden to output). An LSTM has five (one per gate/candidate, plus output):

Operation	**Matrix**	**Connects**
Forget gate	W_f	$[h_{t-1}, x_t] \rightarrow$ forget filter
Candidate memory	W_g	$[h_{t-1}, x_t] \rightarrow$ candidate
Input gate	W_i	$[h_{t-1}, x_t] \rightarrow$ input filter
Output gate	W_o	$[h_{t-1}, x_t] \rightarrow$ output filter
Output prediction	V	hidden state $\rightarrow$ output labels

All four gate and candidate matrices start with small random values and are learned through BPTT, exactly like W in the vanilla RNN. The output matrix V is identical to the one in the vanilla RNN – it has not changed. None of these matrices change while the LSTM is reading a single sequence; they are only updated after the loss is computed.

4.11 Shape Compatibility in an LSTM

Using the same concrete numbers as Section 2.14: vocabulary size = 30, hidden size = 5, output classes = 3.

The input vector x_t is $[30 \times 1]$. The previous hidden state h_{t-1} is $[5 \times 1]$. The previous cell state c_{t-1} is $[5 \times 1]$.

Each gate or candidate equation applies $W \cdot [h_{t-1}, x_t]$ and must produce a $[5 \times 1]$ result (same shape as the hidden state). The concatenated vector $[h_{t-1}, x_t]$ has shape $[35 \times 1]$ $(5 + 30 = 35)$. Applying the shape rule from Section 2.14:

- All W matrices (W_f, W_g, W_i, W_o): must be $[5 \times 35]$, because they take a $[35 \times 1]$ concatenated vector and must produce a $[5 \times 1]$ output.

$$[5 \times 35] \cdot [35 \times 1] = [5 \times 1]\checkmark$$

- All gate and candidate vectors (f_t, g_t, i_t, o_t): shape $[5 \times 1]$ – same size as the hidden state.
- Cell state c_t: shape $[5 \times 1]$ – *same shape as the hidden state.* This is required because $o_t \odot \tanh(c_t)$ is an element-wise multiplication, which needs both vectors to be the same size.
- Output matrix V: $[3 \times 5]$ – unchanged from the vanilla RNN.

With hidden size 5: each W matrix has $5 \times 35 = 175$ values. Four operations $\times$ 175 = 700 parameters for the gates and candidate, plus 15 for V. That is roughly 700 total, compared to about 190 for a vanilla RNN with the same dimensions. (This matches the split-form count: $5{\times}5 + 5{\times}30 = 175$ per gate – the two forms have identical parameter counts.)

4.11.1 Why the Gate Matrices Are Not Square

In the vanilla RNN, U was square ($[5 \times 5]$) because it took only h_{t-1} ($[5 \times 1]$) as input. Here, each W takes the concatenated vector $[h_{t-1}, x_t]$ ($[35 \times 1]$) as input and produces a $[5 \times 1]$ output, so the shape is $[5 \times 35]$ – rectangular, not square. The output size is determined by the hidden size (5); the input size is hidden size + vocabulary size $(5 + 30 = 35)$. These are different numbers, so the matrix cannot be square.

4.11.2 Initialization

All four gate and candidate matrices start with small random values (drawn from a Gaussian or uniform distribution), just like W and V in the vanilla RNN (Section 2.16). One practical exception: the bias vector of the forget gate is often initialized to 1 rather than 0. This makes the forget gate start near $\sigma(1) \approx 0.73$ instead of $\sigma(0) = 0.5$, giving the network a slight bias toward keeping information at the start of training rather than erasing it immediately.

4.12 The Cell State and Hidden State: Shape and Values

4.12.1 Shape

Both h and c have shape [hidden size $\times$ 1]. The hidden size is a hyperparameter you choose before training (same as in the vanilla RNN). Because h and c must be the same size for the element-wise multiplications in the gate equations to work, you cannot choose a different size for each.

4.12.2 Values

Neither h nor c are learned parameters. They are recomputed from scratch at every time step during the forward pass:

- $h^{(0)}$ is set to the zero vector. No short-term memory before the sequence starts.
- $c^{(0)}$ is set to the zero vector. No long-term memory before the sequence starts.
- $h^{(t)}$ is computed from $c^{(t)}$ and o_t at every step.
- $c^{(t)}$ is computed from $c^{(t-1)}$, f_t, i_t, and g_t at every step.

After a sequence is fully processed, both h and c are discarded. The next sequence starts again from the zero vector. What gets learned are the eight weight matrices (and V) – not h or c themselves.

4.13 Cost and Tradeoffs: LSTM vs. Vanilla RNN

The tradeoffs here are relative to the vanilla RNN specifically. The comparison between RNNs and feedforward networks is already covered in Section 2.12.

What you gain over vanilla RNN:

- **Explicit, learnable memory management.** The forget and input gates learn, from data, exactly what to keep and what to discard. The vanilla RNN had no such explicit mechanism – it just overwrote the hidden state at every step.
- **A dedicated long-term memory lane.** The cell state can carry a fact (e.g., "the subject is plural") for many steps without it being overwritten by intermediate computations. The vanilla RNN had no separate channel for this.
- **Gradient highway.** Gradients can reach early steps through the cell state's additive update, allowing the network to learn long-range dependencies that the vanilla RNN could not.

What you give up compared to vanilla RNN:

- **Simplicity.** Six equations per step instead of one. Much harder to understand, implement, and debug.
- **Speed per step.** Each step requires four matrix multiplications (one per gate/candidate) instead of one. This makes each step roughly four times more expensive.

- **Parameter count.** Roughly four times more parameters than a vanilla RNN of the same hidden size. More parameters need more training data and take longer to train.
- **Risk of overfitting.** More parameters means the model can memorize training examples more easily on small datasets.

4.14 Forget Gate Output: Vector, Not a Scalar or Matrix

The forget gate produces a vector f_t with the same shape as the cell state c_t. In an architecture with hidden size 5, f_t is a $[5 \times 1]$ vector where every element lies between 0 and 1, produced by the sigmoid function. Each position in f_t corresponds to one dimension of the cell state. When f_t is multiplied element-wise with c_{t-1}, each dimension of the cell state is independently scaled — a value near 1 keeps that dimension, near 0 erases it. The forget gate therefore produces one decision per dimension, not a single global decision for the entire cell state.

4.15 The Forget Gate Does Not Remember x(t): Division of Responsibility

The forget gate is not responsible for remembering the current input $x(t)$. Its sole job is to decide what to erase from old memory c_{t-1}. Writing $x(t)$ into memory is handled by the input gate and candidate gate together. The candidate $g_t = \tanh(W_g \cdot [h_{t-1}, x_t])$ proposes what is worth remembering from $x(t)$, and the input gate $i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$ filters that proposal. Their product $i_t \odot g_t$ is what actually gets written into the cell state. The forget gate plays no role in this write operation.

4.16 Blocking Unimportant Current Input: The Role of the Input Gate

If $x(t)$ is unimportant and should not be remembered, the forget gate does not handle this. The input gate i_t learns to output values near 0 for all dimensions, making $i_t \odot g_t \approx 0$. Nothing from $x(t)$ enters the cell state. The forget gate remains focused on old memory regardless of what $x(t)$ is. The mental model is: the forget gate is an editor of the past; the input gate is a filter on the present.

4.17 The Forget Gate Exclusively Controls the Old Cell State

The forget gate only operates on c_{t-1}. It produces f_t, which is element-wise multiplied with the previous cell state:

$$f_t \odot c_{t-1}$$

Values in f_t near 1 keep the corresponding dimension of old memory; values near 0 erase it. That is the entirety of what the forget gate does — it has no involvement in writing new information into c_t.

4.18 Why the Forget Gate Takes x(t) and h_{t-1} as Input

To decide what to erase from old memory, the forget gate needs to know what is happening right now. Without current context, it would produce the same f_t at every timestep regardless of the sequence, making it useless. $x(t)$ tells the gate what just arrived — for example, a new subject signals that the old subject's information in c_{t-1} is now stale. h_{t-1} provides the accumulated context so far, enabling a more nuanced erasure decision. Importantly, these inputs do not contribute to what gets written into memory; they only inform what old memory is no longer relevant.

4.19 There Are No Labeled Slots in the Cell State

The cell state c_t is simply a vector of floating point numbers. There are no labeled or separated slots for specific concepts like gender or subject. During training, the network learns weight matrices such that certain dimensions of c_t tend to encode certain patterns, but this emerges from the data and is distributed across dimensions in an entangled way. The language of named slots is a conceptual simplification for building intuition. The underlying math is just vectors and learned weights.

4.20 How Weight Matrices Determine Forget Gate Behavior

A high positive value from $W_f \cdot [h_{t-1}, x(t)]$ does not inherently mean the current input is important and old memory should be erased. The weights encode whatever the task requires, not a generic notion of importance. W_f is a dedicated weight matrix trained specifically to answer: "Given what I see right now, should I keep old memory?" It learns to produce high positive values for inputs that signal continuity (keep old memory) and low or negative values for inputs that signal a context shift (erase old memory). The input gate has its own separate weight matrix W_i that learns independently. The same $x(t)$ can cause W_i to produce high values (write new info) while simultaneously causing W_f to produce low values (erase old info). There is no contradiction because each gate's weight matrix learns a different function from the same input.

4.21 The Input Gate and Candidate Gate: Controlling How Much x(t) Gets Written

The candidate gate $g_t = \tanh(W_g \cdot [h_{t-1}, x_t])$ proposes new information to write into the cell state, with values between -1 and 1. The input gate $i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$ produces a scaling vector between 0 and 1. Their element-wise product $i_t \odot g_t$ determines what is actually added to c_t:

$$c_t = (i_t \odot g_t) + (f_t \odot c_{t-1})$$

For each dimension, i_t near 1 allows that dimension of g_t to pass through fully; i_t near 0 suppresses it. The candidate decides *what* to potentially write; the input gate decides *how much* of each dimension actually gets written.

4.22 Why the Candidate Gate Uses tanh Instead of Sigmoid

Sigmoid outputs values in $(0, 1)$ and is suited for gates that make filtering decisions — how much to keep or write. The candidate gate encodes content, not a decision. Using sigmoid

would restrict g_t to only positive values, meaning the cell state could only accumulate upward through the write operation and could never push a dimension's value down. Tanh outputs values in $(-1, 1)$, allowing the candidate to encode both positive and negative contributions. This gives the network the expressive power to increase or decrease any dimension of the cell state as needed.

4.23 The Input Gate Sigmoid as a Magnitude Controller

The sigmoid in the input gate outputs values between 0 and 1, acting purely as a scaler on the candidate g_t. It contributes no content of its own. Its only role is to control the magnitude of what the candidate proposed, dimension by dimension. It answers: "how much of each dimension of g_t should actually be written into the cell state?"

4.24 The Output Gate Sigmoid: Filtering What Gets Exposed as Hidden State

The output gate's sigmoid decides which dimensions of the current cell state c_t should be exposed as the hidden state h_t:

$$h_t = o_t \odot \tanh(c_t)$$

$\tanh(c_t)$ squashes the cell state into $(-1, 1)$, representing the content available to output. o_t scales each dimension: near 1 exposes that dimension, near 0 suppresses it. The cell state holds everything the LSTM is remembering long-term, but not all of it is relevant for the current prediction. The output gate filters out only what is needed right now and passes it as h_t, which then feeds into the prediction. The pattern is consistent across all three gates — sigmoid always answers a "how much" question, differing only in what it filters:

- Forget gate filters the old cell state c_{t-1}
- Input gate filters the candidate g_t
- Output gate filters the current cell state c_t

5 RNN and LSTM — Concepts and Worked Examples

5.1 Is h_t the Top Hidden Layer?

Yes. In a stacked RNN, h_t refers to the hidden state of the top hidden layer (the one closest to the output layer) at time step t.

5.2 Is c_t a Layer?

No. c_t is not a layer — it is an internal memory vector that lives *inside* the LSTM cell alongside h_t. It never connects to the output layer and is never passed to a higher layer. The distinction is:

- h_t is a layer output: passed **forward** to the next time step and **upward** to the next layer in a stacked architecture.
- c_t is an internal state: passed **forward only** (to the next time step), never upward to a higher layer.

5.3 What Does "Passed Forward" and "Passed Upward" Mean?

- **Forward** means passed to the next time step ($t \to t+1$). When the RNN finishes processing "the", its hidden state is passed forward to the step that processes "hair".
- **Upward** means passed to the next layer above (in a stacked/deep model). In a two-hidden-layer RNN, layer 1's h_t becomes the input to layer 2 at the same time step.

c_t only goes forward (step to step, staying within the same layer). It never goes upward to a higher layer.

5.4 Do Multiple Hidden Layers in a Stacked LSTM Each Have Their Own c_t?

Yes, and this is by design. In a stacked LSTM with L hidden layers, each layer maintains its own cell state:

- Layer 1 has $c_t^{(1)}$, which passes forward within layer 1 only.
- Layer 2 has $c_t^{(2)}$, which passes forward within layer 2 only.

The two cell states capture *different levels of abstraction*, not conflicting memories. Layer 1's c_t tracks low-level patterns (e.g. word categories); layer 2's c_t builds on layer 1's hidden state output and tracks higher-level patterns (e.g. overall sentiment direction). Each hidden layer's h_t is what travels upward between layers — not c_t.

5.5 What Does the Colah LSTM Diagram Show?

Each green box in the Colah blog diagram represents **one time step**, not a stack of layers. So for the sentence "the hair dryer is great", the three visible boxes correspond to three consecutive time steps: processing x_{t-1}, x_t, and x_{t+1} respectively.

The two horizontal arrows passing *through* the boxes are h_t and c_t travelling forward from step to step. If you wanted a stacked LSTM (e.g. two hidden layers), that would be drawn as a second row of boxes stacked above these — not layers packed inside one box.

The architecture shown in the diagram is therefore: 1 input layer, 1 hidden layer, 1 output layer (a single-layer LSTM unrolled across time steps).

5.6 POS Tagging with LSTM: Is Output Produced at Every Step?

Yes. POS tagging is a many-to-many task, so the LSTM produces one output at every time step — one POS tag per word. For "the hair dryer is great", the model produces 5 outputs (one per word), each a probability distribution over all POS tags via softmax.

5.7 What Carries Forward Between Steps?

Only h_{t-1} and c_{t-1} carry information from the previous step to the current step. The output layer's softmax probabilities are used only to compute the loss during training; they do not feed into the next step. The recurrence is:

$$h_t, c_t \quad \longrightarrow \quad \text{used at step } t+1 \tag{6}$$

$$o^{(t)} = \text{softmax}(Vh_t) \quad \longrightarrow \quad \text{used only for loss, discarded} \tag{7}$$

5.8 Correct POS Tags for the Example Sentence

For the sentence "the hair dryer is great":

- the → DET (determiner)
- hair → NOUN (noun modifier)
- dryer → NOUN
- is → VERB
- great → ADJ

5.9 Loss Calculation and Backpropagation When a Prediction Is Wrong

Suppose at step 3 ("dryer") the model predicts VERB instead of the correct label NOUN. The cross-entropy loss at that step is:

$$L^{(3)} = -\log\big(\hat{o}^{(3)}[\text{NOUN}]\big) \tag{8}$$

This loss is large because the model assigned low probability to NOUN. However, the total loss averages across all 5 steps:

$$L_{\text{total}} = \frac{1}{5}\sum_{t=1}^{5} L^{(t)} \tag{9}$$

Backpropagation Through Time (BPTT) then runs *after the full sentence is processed*. Gradients flow backward through all 5 steps. Because the weight matrices W, U, V are *shared* across all steps, their gradient update is the sum of contributions from every step — not just the step that made the wrong prediction.

5.10 Why Are Weights Updated After the Full Sequence, Not After Each Step?

Because the weights are shared across all time steps, and because step $t+1$ depends on step t via the hidden state h_t. Specifically:

- The loss at step 3 is partly caused by weights that acted at steps 1 and 2 (since h_1 flows into h_2, which flows into h_3).
- You cannot correctly assign blame to those shared weights without seeing the total loss across the full sequence.
- Updating after each word would compute gradients based on an incomplete picture of how the weights contributed to the overall error.

This applies equally to vanilla RNNs and LSTMs. The full sequence is treated as one complete forward pass; one weight update follows at the end.

5.11 Comparison with Feedforward Networks and SGD

In a feedforward network trained with SGD on tabular data (e.g. 10 rows, 4 features):

- Weights between each layer are *not shared* across rows.
- Each row is an independent input with no dependency on other rows.
- Therefore: row 1 → forward pass → loss → backprop → weight update. Then row 2, with already-updated weights. And so on.

In an RNN, steps within a sequence *are dependent* (via h_t), so the sequence as a whole is treated as one forward pass, with one weight update at the end.

5.12 Bidirectional RNN: One Loss or Two?

One combined loss. The output at each position is produced from both the forward and backward hidden states fed into a single shared output layer:

$$o^{(t)} = \text{softmax}\big(V_f h_f^{(t)} + V_b h_b^{(t)}\big) \tag{10}$$

This gives one prediction per position → one loss per position → one total loss. Where the two RNNs *separate* is in backpropagation: gradients from the shared loss flow backward independently through the forward RNN (updating W_f, U_f) and the backward RNN (updating W_b, U_b).

6 Worked Example 1 — Vanilla RNN POS Tagging

6.1 Setup

Sentence: "the hair dryer is great" **Task:** POS tagging (many-to-many)

Architecture: Input (5 neurons) → Hidden (5 neurons) → Output (5 neurons)

Vocabulary and one-hot encodings:

$$\begin{aligned}
\text{the} &\to x^{(1)} = [1,0,0,0,0]^\top \\
\text{hair} &\to x^{(2)} = [0,1,0,0,0]^\top \\
\text{dryer} &\to x^{(3)} = [0,0,1,0,0]^\top \\
\text{is} &\to x^{(4)} = [0,0,0,1,0]^\top \\
\text{great} &\to x^{(5)} = [0,0,0,0,1]^\top
\end{aligned}$$

POS tag indices: 0 = DET, 1 = NOUN, 2 = VERB, 3 = ADV, 4 = ADJ

True labels: the $\to$ 0, hair $\to$ 1, dryer $\to$ 1, is $\to$ 2, great $\to$ 4

6.2 One-Hot True Label Vectors for the Vanilla RNN Example

The five POS tags are indexed as: 0 = DET, 1 = NOUN, 2 = VERB, 3 = ADV, 4 = ADJ. The one-hot true label vectors are:

$$\begin{aligned}
y^{(1)} &= [1,0,0,0,0]^\top \quad \text{“the” } \to \text{ DET (index 0)} \\
y^{(2)} &= [0,1,0,0,0]^\top \quad \text{“hair” } \to \text{ NOUN (index 1)} \\
y^{(3)} &= [0,1,0,0,0]^\top \quad \text{“dryer” } \to \text{ NOUN (index 1)} \\
y^{(4)} &= [0,0,1,0,0]^\top \quad \text{“is” } \to \text{ VERB (index 2)} \\
y^{(5)} &= [0,0,0,0,1]^\top \quad \text{“great” } \to \text{ ADJ (index 4)}
\end{aligned}$$

Note that ADJ is at index 4, not index 3, because ADV occupies index 3 even though no word in this sentence is tagged ADV. The output layer still has a neuron for every POS tag — the softmax always produces 5 probabilities regardless of which tags appear in a given sentence.

6.3 Assumed Weight Matrices

$$W = \mathbf{I}_5, \qquad U = 0.5\,\mathbf{I}_5, \qquad V = 0.5\,\mathbf{I}_5, \qquad h^{(0)} = \mathbf{0} \tag{11}$$

Since $W = \mathbf{I}_5$, multiplying W by a one-hot vector simply returns that one-hot vector. Since all matrices are diagonal, $Uh^{(t-1)} = 0.5 \cdot h^{(t-1)}$ element-wise, and similarly for V.

6.4 Equations

$$h^{(t)} = \tanh\big(Uh^{(t-1)} + Wx^{(t)}\big) \tag{12}$$

$$o^{(t)} = \text{softmax}\big(Vh^{(t)}\big) \tag{13}$$

$$L^{(t)} = -\log\big(o^{(t)}[\text{true label}]\big) \tag{14}$$

6.5 Forward Pass

Step 1 — "the":

$$\begin{aligned} Uh^{(0)} + Wx^{(1)} &= [0,0,0,0,0] + [1,0,0,0,0] = [1,0,0,0,0] \\ h^{(1)} &= \tanh([1,0,0,0,0]) = [0.7616,0,0,0,0] \\ Vh^{(1)} &= [0.3808,0,0,0,0] \\ o^{(1)} &= \text{softmax}([0.3808,0,0,0,0]) = [0.2679,0.1830,0.1830,0.1830,0.1830] \\ L^{(1)} &= -\log(0.2679) = 1.3173 \quad \checkmark \text{ DET predicted correctly} \end{aligned}$$

Step 2 — "hair":

$$\begin{aligned} Uh^{(1)} + Wx^{(2)} &= [0.3808,0,0,0,0] + [0,1,0,0,0] = [0.3808,1,0,0,0] \\ h^{(2)} &= \tanh([0.3808,1,0,0,0]) = [0.3634,0.7616,0,0,0] \\ Vh^{(2)} &= [0.1817,0.3808,0,0,0] \\ o^{(2)} &= \text{softmax}(\cdots) = [0.2118,0.2584,0.1766,0.1766,0.1766] \\ L^{(2)} &= -\log(0.2584) = 1.3531 \quad \checkmark \text{ NOUN predicted correctly} \end{aligned}$$

Step 3 — "dryer":

$$\begin{aligned} Uh^{(2)} + Wx^{(3)} &= [0.1817,0.3808,0,0,0] + [0,0,1,0,0] = [0.1817,0.3808,1,0,0] \\ h^{(3)} &= [0.1797,0.3634,0.7616,0,0] \\ o^{(3)} &= [0.1900,0.2083,0.2542,0.1737,0.1737] \\ L^{(3)} &= -\log(0.2083) = 1.5687 \quad \times \text{ VERB predicted, true is NOUN} \end{aligned}$$

Step 4 — "is":

$$\begin{aligned} Uh^{(3)} + Wx^{(4)} &= [0.0899,0.1817,0.3808,0,0] + [0,0,0,1,0] = [0.0899,0.1817,0.3808,1,0] \\ h^{(4)} &= [0.0896,0.1797,0.3634,0.7616,0] \\ o^{(4)} &= [0.1802,0.1885,0.2067,0.2522,0.1723] \\ L^{(4)} &= -\log(0.2067) = 1.5766 \quad \times \text{ ADV predicted, true is VERB} \end{aligned}$$

Step 5 — "great":

$$\begin{aligned} Uh^{(4)} + Wx^{(5)} &= [0.0448,0.0899,0.1817,0.3808,0] + [0,0,0,0,1] \\ h^{(5)} &= [0.0448,0.0896,0.1797,0.3634,0.7616] \\ o^{(5)} &= [0.1756,0.1795,0.1878,0.2059,0.2512] \\ L^{(5)} &= -\log(0.2512) = 1.3814 \quad \checkmark \text{ ADJ predicted correctly} \end{aligned}$$

Total Loss:

$$L_{\text{total}} = \frac{1.3173 + 1.3531 + 1.5687 + 1.5766 + 1.3814}{5} = 1.4394 \tag{15}$$

6.6 Backward Pass (BPTT)

For cross-entropy loss combined with softmax, the gradient of the loss with respect to the pre-softmax scores at each step is:

$$\delta_{\text{out}}^{(t)} = o^{(t)} - y^{(t)} \tag{16}$$

where $y^{(t)}$ is the one-hot true label vector.

Output layer gradient:

$$\begin{aligned}
\delta_{\text{out}}^{(1)} &= [-0.7321,\ 0.1830,\ 0.1830,\ 0.1830,\ 0.1830] \\
\delta_{\text{out}}^{(2)} &= [\ 0.2118,\ -0.7416,\ 0.1766,\ 0.1766,\ 0.1766] \\
\delta_{\text{out}}^{(3)} &= [\ 0.1900,\ -0.7917,\ 0.2542,\ 0.1737,\ 0.1737] \\
\delta_{\text{out}}^{(4)} &= [\ 0.1802,\ 0.1885,\ -0.7933,\ 0.2522,\ 0.1723] \\
\delta_{\text{out}}^{(5)} &= [\ 0.1756,\ 0.1795,\ 0.1878,\ 0.2059,\ -0.7488]
\end{aligned}$$

$$\frac{\partial L}{\partial V} = \frac{1}{5}\sum_{t=1}^{5} \delta_{\text{out}}^{(t)} \otimes h^{(t)} \tag{17}$$

BPTT for W and U (backward from $t = 5$ to $t = 1$):

At each step, the gradient of the hidden state receives two contributions: one from the output at that step, and one flowing from the next step. The tanh derivative then gates it before accumulating into ∇W and ∇U:

$$\begin{aligned}
\delta_h^{(t)} &= V^\top \delta_{\text{out}}^{(t)}/5 + \delta_{h,\text{next}} \\
\delta_{\text{pre}}^{(t)} &= \delta_h^{(t)} \odot \left(1 - [h^{(t)}]^2\right) \quad \leftarrow \text{tanh derivative} \\
\frac{\partial L}{\partial W} &\mathrel{+}= \delta_{\text{pre}}^{(t)} \otimes x^{(t)} \\
\frac{\partial L}{\partial U} &\mathrel{+}= \delta_{\text{pre}}^{(t)} \otimes h^{(t-1)} \\
\delta_{h,\text{next}} &\leftarrow U^\top \delta_{\text{pre}}^{(t)}
\end{aligned}$$

Total gradients:

$$\nabla V = \begin{bmatrix}
-0.0845 & 0.0557 & 0.0484 & 0.0402 & 0.0267 \\
-0.0495 & -0.1605 & -0.1004 & 0.0418 & 0.0273 \\
0.0373 & 0.0202 & -0.0122 & -0.1072 & 0.0286 \\
0.0533 & 0.0523 & 0.0522 & 0.0534 & 0.0314 \\
0.0433 & 0.0323 & 0.0121 & -0.0282 & -0.1141
\end{bmatrix} \tag{18}$$

$$\nabla W = \begin{bmatrix}
-0.0240 & 0.0319 & 0.0312 & 0.0266 & 0.0175 \\
-0.0033 & -0.0431 & -0.0571 & 0.0269 & 0.0178 \\
0.0266 & 0.0166 & -0.0021 & -0.0610 & 0.0182 \\
0.0333 & 0.0299 & 0.0245 & 0.0143 & 0.0179 \\
0.0317 & 0.0267 & 0.0181 & 0.0015 & -0.0314
\end{bmatrix} \tag{19}$$

$$\nabla U = \begin{bmatrix} 0.0420 & 0.0366 & 0.0266 & 0.0133 & 0 \\ -0.0472 & -0.0305 & 0.0269 & 0.0136 & 0 \\ 0.0025 & -0.0205 & -0.0398 & 0.0138 & 0 \\ 0.0359 & 0.0271 & 0.0174 & 0.0136 & 0 \\ 0.0244 & 0.0087 & -0.0103 & -0.0239 & 0 \end{bmatrix} \tag{20}$$

The last column of ∇U is all zeros because $h^{(0)} = \mathbf{0}$, so no gradient flows back through the recurrent connection from step 1 to the initial state.

6.7 Weight Update ($\eta = 0.1$)

$$W_{\text{new}} = \begin{bmatrix} 1.0024 & -0.0032 & -0.0031 & -0.0027 & -0.0018 \\ 0.0003 & 1.0043 & 0.0057 & -0.0027 & -0.0018 \\ -0.0027 & -0.0017 & 1.0002 & 0.0061 & -0.0018 \\ -0.0033 & -0.0030 & -0.0025 & 0.9986 & -0.0018 \\ -0.0032 & -0.0027 & -0.0018 & -0.0002 & 1.0031 \end{bmatrix} \tag{21}$$

$$U_{\text{new}} = \begin{bmatrix} 0.4958 & -0.0037 & -0.0027 & -0.0013 & 0 \\ 0.0047 & 0.5030 & -0.0027 & -0.0014 & 0 \\ -0.0003 & 0.0021 & 0.5040 & -0.0014 & 0 \\ -0.0036 & -0.0027 & -0.0017 & 0.4986 & 0 \\ -0.0024 & -0.0009 & 0.0010 & 0.0024 & 0.5 \end{bmatrix} \tag{22}$$

$$V_{\text{new}} = \begin{bmatrix} 0.5084 & -0.0056 & -0.0048 & -0.0040 & -0.0027 \\ 0.0049 & 0.5160 & 0.0100 & -0.0042 & -0.0027 \\ -0.0037 & -0.0020 & 0.5012 & 0.0107 & -0.0029 \\ -0.0053 & -0.0052 & -0.0052 & 0.4947 & -0.0031 \\ -0.0043 & -0.0032 & -0.0012 & 0.0028 & 0.5114 \end{bmatrix} \tag{23}$$

7 Worked Example 2 — Bidirectional RNN POS Tagging

7.1 Setup

Same sentence, same vocabulary, same true labels as Example 1.

Architecture: Two RNNs run in parallel on the same sentence. The forward RNN reads left to right; the backward RNN reads right to left. At each position t, the output combines both hidden states:

$$o^{(t)} = \text{softmax}\big(V_f h_f^{(t)} + V_b h_b^{(t)}\big) \tag{24}$$

Assumed weight matrices:

$$W_f = \mathbf{I}_5, \quad U_f = 0.5\,\mathbf{I}_5, \quad V_f = 0.5\,\mathbf{I}_5 \tag{25}$$

$$W_b = \mathbf{I}_5, \quad U_b = 0.5\,\mathbf{I}_5, \quad V_b = 0.5\,\mathbf{I}_5 \tag{26}$$

$$h_f^{(0)} = \mathbf{0}, \qquad h_b^{(6)} = \mathbf{0} \quad \text{(backward initial state)} \tag{27}$$

7.2 Forward RNN (left to right)

The forward RNN hidden states are identical to those in Example 1:

$$h_f^{(1)} = [0.7616,\ 0,\ 0,\ 0,\ 0] \quad \text{(after "the")}$$
$$h_f^{(2)} = [0.3634,\ 0.7616,\ 0,\ 0,\ 0] \quad \text{(after "hair")}$$
$$h_f^{(3)} = [0.1797,\ 0.3634,\ 0.7616,\ 0,\ 0] \quad \text{(after "dryer")}$$
$$h_f^{(4)} = [0.0896,\ 0.1797,\ 0.3634,\ 0.7616,\ 0] \quad \text{(after "is")}$$
$$h_f^{(5)} = [0.0448,\ 0.0896,\ 0.1797,\ 0.3634,\ 0.7616] \quad \text{(after "great")}$$

7.3 Backward RNN (right to left)

The backward RNN starts at the end of the sentence with $h_b^{(6)} = \mathbf{0}$ and processes words from right to left. The equation at each step is:

$$h_b^{(t)} = \tanh\big(U_b h_b^{(t+1)} + W_b x^{(t)}\big) \tag{28}$$

Step: $t = 5$ — "great" (first backward step):

$$U_b h_b^{(6)} + W_b x^{(5)} = [0,0,0,0,0] + [0,0,0,0,1] = [0,0,0,0,1]$$
$$h_b^{(5)} = \tanh([0,0,0,0,1]) = [0,\ 0,\ 0,\ 0,\ 0.7616]$$

Step: $t = 4$ — "is" (second backward step):

$$U_b h_b^{(5)} + W_b x^{(4)} = [0,0,0,0,0.3808] + [0,0,0,1,0] = [0,0,0,1,0.3808]$$
$$h_b^{(4)} = [0,\ 0,\ 0,\ 0.7616,\ 0.3634]$$

Step: $t = 3$ — "dryer" (third backward step):

$$U_b h_b^{(4)} + W_b x^{(3)} = [0,0,0,0.3808,0.1817] + [0,0,1,0,0] = [0,0,1,0.3808,0.1817]$$
$$h_b^{(3)} = [0,\ 0,\ 0.7616,\ 0.3634,\ 0.1797]$$

Step: $t = 2$ — "hair" (fourth backward step):

$$U_b h_b^{(3)} + W_b x^{(2)} = [0,0,0.3808,0.1817,0.0899] + [0,1,0,0,0]$$
$$= [0,1,0.3808,0.1817,0.0899]$$
$$h_b^{(2)} = [0,\ 0.7616,\ 0.3634,\ 0.1797,\ 0.0896]$$

Step: $t = 1$ — "the" (fifth backward step):

$$U_b h_b^{(2)} + W_b x^{(1)} = [0,0.3808,0.1817,0.0899,0.0448] + [1,0,0,0,0]$$
$$= [1,0.3808,0.1817,0.0899,0.0448]$$
$$h_b^{(1)} = [0.7616,\ 0.3634,\ 0.1797,\ 0.0896,\ 0.0448]$$

Note: $h_b^{(1)}$ ("the") encodes information about the entire sentence read from right to left. $h_b^{(5)}$ ("great") only saw the word "great" with no future context (since it was the first backward step).

7.4 Combined Output and Loss

At each position, $V_f h_f^{(t)} + V_b h_b^{(t)}$ combines left-to-right and right-to-left context.

Position 1 ("the"):

$$\begin{aligned} V_f h_f^{(1)} &= [0.3808,\ 0,\ 0,\ 0,\ 0] \\ V_b h_b^{(1)} &= [0.3808,\ 0.1817,\ 0.0899,\ 0.0448,\ 0.0224] \\ \text{sum} &= [0.7616,\ 0.1817,\ 0.0899,\ 0.0448,\ 0.0224] \\ o^{(1)} &= [0.3293,\ 0.1844,\ 0.1682,\ 0.1608,\ 0.1572] \\ L^{(1)} &= -\log(0.3293) = 1.1107 \quad \checkmark \text{ DET} \end{aligned}$$

Position 2 ("hair"):

$$\begin{aligned} V_f h_f^{(2)} + V_b h_b^{(2)} &= [0.1817,\ 0.7616,\ 0.1817,\ 0.0899,\ 0.0448] \\ o^{(2)} &= [0.1795,\ 0.3206,\ 0.1795,\ 0.1638,\ 0.1566] \\ L^{(2)} &= -\log(0.3206) = 1.1375 \quad \checkmark \text{ NOUN} \end{aligned}$$

Position 3 ("dryer"):

$$\begin{aligned} V_f h_f^{(3)} + V_b h_b^{(3)} &= [0.0899,\ 0.1817,\ 0.7616,\ 0.1817,\ 0.0899] \\ o^{(3)} &= [0.1626,\ 0.1782,\ 0.3183,\ 0.1782,\ 0.1626] \\ L^{(3)} &= -\log(0.1782) = 1.7246 \quad \times \text{ VERB predicted, true is NOUN} \end{aligned}$$

Position 4 ("is"):

$$\begin{aligned} V_f h_f^{(4)} + V_b h_b^{(4)} &= [0.0448,\ 0.0899,\ 0.1817,\ 0.7616,\ 0.1817] \\ o^{(4)} &= [0.1566,\ 0.1638,\ 0.1795,\ 0.3206,\ 0.1795] \\ L^{(4)} &= -\log(0.1795) = 1.7174 \quad \times \text{ ADV predicted, true is VERB} \end{aligned}$$

Position 5 ("great"):

$$\begin{aligned} V_f h_f^{(5)} + V_b h_b^{(5)} &= [0.0224,\ 0.0448,\ 0.0899,\ 0.1817,\ 0.7616] \\ o^{(5)} &= [0.1572,\ 0.1608,\ 0.1682,\ 0.1844,\ 0.3293] \\ L^{(5)} &= -\log(0.3293) = 1.1107 \quad \checkmark \text{ ADJ} \end{aligned}$$

$$L_{\text{total}} = \frac{1.1107 + 1.1375 + 1.7246 + 1.7174 + 1.1107}{5} = 1.3602 \tag{29}$$

The BiRNN achieves lower total loss than the vanilla RNN (1.3602 vs 1.4394) on this example because each word now has access to both past and future context.

7.5 Backward Pass (BPTT)

There is one combined loss. Gradients from this loss flow backward independently through two separate RNNs.

Gradient of V_f and V_b:

$$\frac{\partial L}{\partial V_f} = \frac{1}{5}\sum_{t=1}^{5} \delta_{\text{out}}^{(t)} \otimes h_f^{(t)}, \qquad \frac{\partial L}{\partial V_b} = \frac{1}{5}\sum_{t=1}^{5} \delta_{\text{out}}^{(t)} \otimes h_b^{(t)} \tag{30}$$

$$\nabla V_f = \begin{bmatrix} -0.0791 & 0.0476 & 0.0418 & 0.0353 & 0.0240 \\ -0.0465 & -0.1544 & -0.1075 & 0.0366 & 0.0245 \\ 0.0369 & 0.0240 & -0.0051 & -0.1127 & 0.0256 \\ 0.0502 & 0.0527 & 0.0571 & 0.0622 & 0.0281 \\ 0.0384 & 0.0301 & 0.0137 & -0.0214 & -0.1022 \end{bmatrix} \tag{31}$$

$$\nabla V_b = \begin{bmatrix} -0.1022 & -0.0214 & 0.0137 & 0.0301 & 0.0384 \\ 0.0281 & -0.0901 & -0.1679 & -0.0559 & -0.0037 \\ 0.0256 & 0.0396 & 0.0676 & -0.0924 & -0.0178 \\ 0.0245 & 0.0366 & 0.0448 & 0.0706 & 0.0622 \\ 0.0240 & 0.0353 & 0.0418 & 0.0476 & -0.0791 \end{bmatrix} \tag{32}$$

BPTT for the forward RNN (W_f, U_f): gradients flow from $t = 5$ back to $t = 1$, exactly as in Example 1. The $\delta_{\text{out}}^{(t)}$ values are different because the outputs here are from the combined BiRNN model.

BPTT for the backward RNN (W_b, U_b): gradients flow in the *opposite direction* of processing. Since the backward RNN processed $t = 5$ first and $t = 1$ last, its BPTT flows from $t = 1$ *forward* to $t = 5$. At each step t, the gradient of $h_b^{(t)}$ includes:

- Direct contribution from the output at position t: $V_b^\top \delta_{\text{out}}^{(t)}/5$.
- Contribution from $h_b^{(t-1)}$ via U_b (since $h_b^{(t)}$ was used to compute $h_b^{(t-1)}$ during the forward pass of the backward RNN).

Total gradients for forward RNN weights:

$$\nabla W_f = \begin{bmatrix} -0.0224 & 0.0273 & 0.0270 & 0.0233 & 0.0157 \\ -0.0022 & -0.0414 & -0.0611 & 0.0236 & 0.0160 \\ 0.0258 & 0.0179 & -0.0001 & -0.0641 & 0.0163 \\ 0.0308 & 0.0295 & 0.0262 & 0.0168 & 0.0160 \\ 0.0281 & 0.0248 & 0.0182 & 0.0039 & -0.0282 \end{bmatrix} \tag{33}$$

$$\nabla U_f = \begin{bmatrix} 0.0362 & 0.0319 & 0.0235 & 0.0120 & 0 \\ -0.0480 & -0.0351 & 0.0237 & 0.0121 & 0 \\ 0.0035 & -0.0205 & -0.0429 & 0.0124 & 0 \\ 0.0365 & 0.0290 & 0.0186 & 0.0122 & 0 \\ 0.0236 & 0.0102 & -0.0073 & -0.0215 & 0 \end{bmatrix} \tag{34}$$

Total gradients for backward RNN weights:

$$\nabla W_b = \begin{bmatrix} -0.0282 & 0.0039 & 0.0182 & 0.0248 & 0.0281 \\ 0.0160 & -0.0252 & -0.0948 & -0.0310 & 0.0006 \\ 0.0163 & 0.0226 & 0.0181 & -0.0730 & -0.0197 \\ 0.0160 & 0.0236 & 0.0257 & 0.0189 & 0.0279 \\ 0.0157 & 0.0233 & 0.0270 & 0.0273 & -0.0224 \end{bmatrix} \tag{35}$$

$$\nabla U_b = \begin{bmatrix} 0 & -0.0215 & -0.0073 & 0.0102 & 0.0236 \\ 0 & 0.0122 & -0.0134 & -0.0784 & -0.0611 \\ 0 & 0.0124 & 0.0232 & 0.0250 & -0.0435 \\ 0 & 0.0121 & 0.0237 & 0.0310 & 0.0294 \\ 0 & 0.0120 & 0.0235 & 0.0319 & 0.0362 \end{bmatrix} \tag{36}$$

The first column of ∇U_b is all zeros because $h_b^{(6)} = \mathbf{0}$ (the backward RNN's initial state), mirroring how ∇U's last column was zero in Example 1 for the same reason.

7.6 Weight Update ($\eta = 0.1$)

$$W_{f,\text{new}} = \begin{bmatrix} 1.0022 & -0.0027 & -0.0027 & -0.0023 & -0.0016 \\ 0.0002 & 1.0041 & 0.0061 & -0.0024 & -0.0016 \\ -0.0026 & -0.0018 & 1.0000 & 0.0064 & -0.0016 \\ -0.0031 & -0.0029 & -0.0026 & 0.9983 & -0.0016 \\ -0.0028 & -0.0025 & -0.0018 & -0.0004 & 1.0028 \end{bmatrix} \tag{37}$$

$$U_{f,\text{new}} = \begin{bmatrix} 0.4964 & -0.0032 & -0.0023 & -0.0012 & 0 \\ 0.0048 & 0.5035 & -0.0024 & -0.0012 & 0 \\ -0.0004 & 0.0020 & 0.5043 & -0.0012 & 0 \\ -0.0036 & -0.0029 & -0.0019 & 0.4988 & 0 \\ -0.0024 & -0.0010 & 0.0007 & 0.0021 & 0.5 \end{bmatrix} \tag{38}$$

$$W_{b,\text{new}} = \begin{bmatrix} 1.0028 & -0.0004 & -0.0018 & -0.0025 & -0.0028 \\ -0.0016 & 1.0025 & 0.0095 & 0.0031 & -0.0001 \\ -0.0016 & -0.0023 & 0.9982 & 0.0073 & 0.0020 \\ -0.0016 & -0.0024 & -0.0026 & 0.9981 & -0.0028 \\ -0.0016 & -0.0023 & -0.0027 & -0.0027 & 1.0022 \end{bmatrix} \tag{39}$$

$$U_{b,\text{new}} = \begin{bmatrix} 0.5 & 0.0021 & 0.0007 & -0.0010 & -0.0024 \\ 0 & 0.4988 & 0.0013 & 0.0078 & 0.0061 \\ 0 & -0.0012 & 0.4977 & -0.0025 & 0.0043 \\ 0 & -0.0012 & -0.0024 & 0.4969 & -0.0029 \\ 0 & -0.0012 & -0.0023 & -0.0032 & 0.4964 \end{bmatrix} \tag{40}$$

$$V_{f,\text{new}} = \begin{bmatrix} 0.5079 & -0.0048 & -0.0042 & -0.0035 & -0.0024 \\ 0.0046 & 0.5154 & 0.0107 & -0.0037 & -0.0024 \\ -0.0037 & -0.0024 & 0.5005 & 0.0113 & -0.0026 \\ -0.0050 & -0.0053 & -0.0057 & 0.4938 & -0.0028 \\ -0.0038 & -0.0030 & -0.0014 & 0.0021 & 0.5102 \end{bmatrix} \tag{41}$$

$$V_{b,\text{new}} = \begin{bmatrix} 0.5102 & 0.0021 & -0.0014 & -0.0030 & -0.0038 \\ -0.0028 & 0.5090 & 0.0168 & 0.0056 & 0.0004 \\ -0.0026 & -0.0040 & 0.4932 & 0.0092 & 0.0018 \\ -0.0024 & -0.0037 & -0.0045 & 0.4929 & -0.0062 \\ -0.0024 & -0.0035 & -0.0042 & -0.0048 & 0.5079 \end{bmatrix} \tag{42}$$

8 Worked Example 3 — LSTM POS Tagging

8.1 Setup

Same sentence, vocabulary, and true labels as Examples 1 and 2.

Architecture: Input (5) → LSTM hidden (5) → Output (5)

LSTM Equations (split-form notation, no bias):

$$f_t = \sigma(W_{f,h}\, h_{t-1} + W_{f,x}\, x_t) \quad \text{(forget gate)} \tag{43}$$
$$g_t = \tanh(W_{g,h}\, h_{t-1} + W_{g,x}\, x_t) \quad \text{(candidate)} \tag{44}$$
$$i_t = \sigma(W_{i,h}\, h_{t-1} + W_{i,x}\, x_t) \quad \text{(input gate)} \tag{45}$$
$$o_t = \sigma(W_{o,h}\, h_{t-1} + W_{o,x}\, x_t) \quad \text{(output gate)} \tag{46}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \text{(cell state)} \tag{47}$$
$$h_t = o_t \odot \tanh(c_t) \quad \text{(hidden state)} \tag{48}$$

Note on notation: These are equivalent to Colah's concatenation form $W \cdot [h_{t-1}, x_t]$ where $W = [W_h \mid W_x]$ is the concatenated matrix. Both forms produce identical results.

Assumed weight matrices (all diagonal, no biases):

$$W_{f,h} = 0.4\,\mathbf{I}_5, \quad W_{f,x} = 0.3\,\mathbf{I}_5 \quad \text{(forget gate)} \tag{49}$$

$$W_{g,h} = 0.2\,\mathbf{I}_5, \quad W_{g,x} = 0.7\,\mathbf{I}_5 \quad \text{(candidate)} \tag{50}$$
$$W_{i,h} = 0.5\,\mathbf{I}_5, \quad W_{i,x} = 0.4\,\mathbf{I}_5 \quad \text{(input gate)} \tag{51}$$
$$W_{o,h} = 0.3\,\mathbf{I}_5, \quad W_{o,x} = 0.4\,\mathbf{I}_5 \quad \text{(output gate)} \tag{52}$$
$$V = 0.5\,\mathbf{I}_5, \qquad h_0 = \mathbf{0}, \qquad c_0 = \mathbf{0} \tag{53}$$

Since all matrices are diagonal, multiplying by a vector is equivalent to element-wise scaling. For example, $0.4\,\mathbf{I}_5 \cdot h_{t-1} = 0.4 \cdot h_{t-1}$ element-wise.

8.2 Forward Pass

Step 1 — "the":

$$h_0 = [0,0,0,0,0], \quad c_0 = [0,0,0,0,0], \quad x^{(1)} = [1,0,0,0,0]$$

$$\begin{aligned} f_1 &= \sigma(0.4 \cdot [0,0,0,0,0] + 0.3 \cdot [1,0,0,0,0]) \\ &= \sigma([0.3,0,0,0,0]) = [0.5744,\ 0.5000,\ 0.5000,\ 0.5000,\ 0.5000] \end{aligned}$$

$$\begin{aligned} g_1 &= \tanh(0.2 \cdot [0,0,0,0,0] + 0.7 \cdot [1,0,0,0,0]) \\ &= \tanh([0.7,0,0,0,0]) = [0.6044,\ 0,\ 0,\ 0,\ 0] \end{aligned}$$

$$\begin{aligned} i_1 &= \sigma(0.5 \cdot [0,0,0,0,0] + 0.4 \cdot [1,0,0,0,0]) \\ &= \sigma([0.4,0,0,0,0]) = [0.5987,\ 0.5000,\ 0.5000,\ 0.5000,\ 0.5000] \end{aligned}$$

$$\begin{aligned} o_1 &= \sigma(0.3 \cdot [0,0,0,0,0] + 0.4 \cdot [1,0,0,0,0]) \\ &= \sigma([0.4,0,0,0,0]) = [0.5987,\ 0.5000,\ 0.5000,\ 0.5000,\ 0.5000] \end{aligned}$$

$$\begin{aligned} c_1 &= f_1 \odot c_0 + i_1 \odot g_1 = [0,0,0,0,0] + [0.5987 \times 0.6044,\ 0.5 \times 0,\ \ldots] \\ &= [0.3618,\ 0,\ 0,\ 0,\ 0] \end{aligned}$$

$$\begin{aligned} h_1 &= o_1 \odot \tanh(c_1) = [0.5987,\ 0.5,\ 0.5,\ 0.5,\ 0.5] \odot [0.3468,\ 0,\ 0,\ 0,\ 0] \\ &= [0.2076,\ 0,\ 0,\ 0,\ 0] \end{aligned}$$

$$\begin{aligned} o^{(1)} &= \text{softmax}(Vh_1) = \text{softmax}([0.1038,0,0,0,0]) \\ &= [0.2171,\ 0.1957,\ 0.1957,\ 0.1957,\ 0.1957] \end{aligned}$$

$$L^{(1)} = -\log(0.2171) = 1.5273 \quad \checkmark \text{ DET predicted correctly}$$

Step 2 — "hair":

$$h_1 = [0.2076, 0, 0, 0, 0], \quad c_1 = [0.3618, 0, 0, 0, 0]$$
$$x^{(2)} = [0, 1, 0, 0, 0]$$

$$f_2 = \sigma(0.4 \cdot h_1 + 0.3 \cdot x^{(2)})$$
$$= \sigma([0.0831,\ 0.3,\ 0,\ 0,\ 0]) = [0.5208,\ 0.5744,\ 0.5000,\ 0.5000,\ 0.5000]$$

$$g_2 = \tanh(0.2 \cdot h_1 + 0.7 \cdot x^{(2)})$$
$$= \tanh([0.0415,\ 0.7,\ 0,\ 0,\ 0]) = [0.0415,\ 0.6044,\ 0,\ 0,\ 0]$$

$$i_2 = \sigma(0.5 \cdot h_1 + 0.4 \cdot x^{(2)})$$
$$= \sigma([0.1038,\ 0.4,\ 0,\ 0,\ 0]) = [0.5259,\ 0.5987,\ 0.5000,\ 0.5000,\ 0.5000]$$

$$o_2 = \sigma(0.3 \cdot h_1 + 0.4 \cdot x^{(2)})$$
$$= \sigma([0.0623,\ 0.4,\ 0,\ 0,\ 0]) = [0.5156,\ 0.5987,\ 0.5000,\ 0.5000,\ 0.5000]$$

$$c_2 = f_2 \odot c_1 + i_2 \odot g_2$$
$$= [0.5208 \times 0.3618,\ 0.5744 \times 0,\ \ldots] + [0.5259 \times 0.0415,\ 0.5987 \times 0.6044,\ \ldots]$$
$$= [0.1884,\ 0,\ 0,\ 0,\ 0] + [0.0218,\ 0.3618,\ 0,\ 0,\ 0]$$
$$= [0.2103,\ 0.3618,\ 0,\ 0,\ 0]$$

$$h_2 = o_2 \odot \tanh(c_2) = [0.5156,\ 0.5987,\ \ldots] \odot [0.2072,\ 0.3468,\ \ldots]$$
$$= [0.1068,\ 0.2076,\ 0,\ 0,\ 0]$$

$$o^{(2)} = \text{softmax}([0.0534,\ 0.1038,\ 0,\ 0,\ 0])$$
$$= [0.2043,\ 0.2148,\ 0.1936,\ 0.1936,\ 0.1936]$$
$$L^{(2)} = -\log(0.2148) = 1.5379$$
✓ NOUN predicted correctly

Step 3 — "dryer":

$$h_2 = [0.1068, 0.2076, 0, 0, 0], \quad c_2 = [0.2103, 0.3618, 0, 0, 0]$$

$$f_3 = \sigma([0.0427,\ 0.0831,\ 0.3,\ 0,\ 0]) = [0.5107,\ 0.5208,\ 0.5744,\ 0.5,\ 0.5]$$
$$g_3 = \tanh([0.0214,\ 0.0415,\ 0.7,\ 0,\ 0]) = [0.0214,\ 0.0415,\ 0.6044,\ 0,\ 0]$$
$$i_3 = \sigma([0.0534,\ 0.1038,\ 0.4,\ 0,\ 0]) = [0.5134,\ 0.5259,\ 0.5987,\ 0.5,\ 0.5]$$
$$o_3 = \sigma([0.0320,\ 0.0623,\ 0.4,\ 0,\ 0]) = [0.5080,\ 0.5156,\ 0.5987,\ 0.5,\ 0.5]$$

$$c_3 = f_3 \odot c_2 + i_3 \odot g_3$$
$$= [0.1074,\ 0.1884,\ 0,\ 0,\ 0] + [0.0110,\ 0.0218,\ 0.3618,\ 0,\ 0]$$
$$= [0.1183,\ 0.2103,\ 0.3618,\ 0,\ 0]$$

$$h_3 = o_3 \odot \tanh(c_3) = [0.0598,\ 0.1068,\ 0.2076,\ 0,\ 0]$$

$$o^{(3)} = \text{softmax}([0.0299,\ 0.0534,\ 0.1038,\ 0,\ 0])$$
$$= [0.1984,\ 0.2031,\ 0.2136,\ 0.1925,\ 0.1925]$$
$$L^{(3)} = -\log(0.2031) = 1.5942 \quad \times \text{ VERB predicted, true is NOUN}$$

Step 4 — "is":

$$h_3 = [0.0598, 0.1068, 0.2076, 0, 0], \quad c_3 = [0.1183, 0.2103, 0.3618, 0, 0]$$

$$f_4 = \sigma([0.0239, 0.0427, 0.0831, 0.3, 0]) = [0.5060, 0.5107, 0.5208, 0.5744, 0.5]$$
$$g_4 = \tanh([0.0120, 0.0214, 0.0415, 0.7, 0]) = [0.0120, 0.0214, 0.0415, 0.6044, 0]$$
$$i_4 = \sigma([0.0299, 0.0534, 0.1038, 0.4, 0]) = [0.5075, 0.5134, 0.5259, 0.5987, 0.5]$$
$$o_4 = \sigma([0.0180, 0.0320, 0.0623, 0.4, 0]) = [0.5045, 0.5080, 0.5156, 0.5987, 0.5]$$

$$\begin{aligned} c_4 &= [0.0599, 0.1074, 0.1884, 0, 0] + [0.0061, 0.0110, 0.0218, 0.3618, 0] \\ &= [0.0659, 0.1183, 0.2103, 0.3618, 0] \end{aligned}$$

$$h_4 = [0.0332, 0.0598, 0.1068, 0.2076, 0]$$

$$\begin{aligned} o^{(4)} &= \text{softmax}([0.0166, 0.0299, 0.0534, 0.1038, 0]) \\ &= [0.1951, 0.1977, 0.2024, 0.2129, 0.1919] \end{aligned}$$

$$L^{(4)} = -\log(0.2024) = 1.5974 \quad \times \text{ ADV predicted, true is VERB}$$

Step 5 — "great":

$$h_4 = [0.0332, 0.0598, 0.1068, 0.2076, 0]$$
$$c_4 = [0.0659, 0.1183, 0.2103, 0.3618, 0]$$

$$f_5 = \sigma([0.0133, 0.0239, 0.0427, 0.0831, 0.3]) = [0.5033, 0.5060, 0.5107, 0.5208, 0.5744]$$
$$g_5 = \tanh([0.0066, 0.0120, 0.0214, 0.0415, 0.7]) = [0.0066, 0.0120, 0.0214, 0.0415, 0.6044]$$
$$i_5 = \sigma([0.0166, 0.0299, 0.0534, 0.1038, 0.4]) = [0.5042, 0.5075, 0.5134, 0.5259, 0.5987]$$
$$o_5 = \sigma([0.0100, 0.0180, 0.0320, 0.0623, 0.4]) = [0.5025, 0.5045, 0.5080, 0.5156, 0.5987]$$

$$\begin{aligned} c_5 &= [0.0332, 0.0599, 0.1074, 0.1884, 0] + [0.0033, 0.0061, 0.0110, 0.0218, 0.3618] \\ &= [0.0365, 0.0659, 0.1183, 0.2103, 0.3618] \end{aligned}$$

$$h_5 = [0.0184, 0.0332, 0.0598, 0.1068, 0.2076]$$

$$\begin{aligned} o^{(5)} &= \text{softmax}([0.0092, 0.0166, 0.0299, 0.0534, 0.1038]) \\ &= [0.1933, 0.1948, 0.1974, 0.2021, 0.2125] \end{aligned}$$

$$L^{(5)} = -\log(0.2125) = 1.5488$$
✓ ADJ predicted correctly

$$L_{\text{total}} = \frac{1.5273 + 1.5379 + 1.5942 + 1.5974 + 1.5488}{5} = 1.5611 \tag{54}$$

8.3 Backward Pass (BPTT)

Output layer gradient:

$$\delta_{\text{out}}^{(1)} = [-0.7829,\ 0.1957,\ 0.1957,\ 0.1957,\ 0.1957]$$
$$\delta_{\text{out}}^{(2)} = [\ 0.2043,\ -0.7852,\ 0.1936,\ 0.1936,\ 0.1936]$$
$$\delta_{\text{out}}^{(3)} = [\ 0.1984,\ -0.7969,\ 0.2136,\ 0.1925,\ 0.1925]$$
$$\delta_{\text{out}}^{(4)} = [\ 0.1951,\ 0.1977,\ -0.7976,\ 0.2129,\ 0.1919]$$
$$\delta_{\text{out}}^{(5)} = [\ 0.1933,\ 0.1948,\ 0.1974,\ 0.2021,\ -0.7875]$$

LSTM BPTT chain (from $t = 5$ back to $t = 1$):

At each step t, the gradient flows through the output gate and the cell state. The cell state gradient is the LSTM's gradient highway:

$$\delta_h^{(t)} = V^\top \delta_{\text{out}}^{(t)}/T + \delta_{h,\text{next}} \tag{55}$$
$$\delta_o^{(t)} = \delta_h^{(t)} \odot \tanh(c_t) \tag{56}$$
$$\delta_{o,\text{raw}}^{(t)} = \delta_o^{(t)} \odot o_t \odot (1 - o_t) \quad \leftarrow \sigma' \tag{57}$$
$$\delta_c^{(t)} = \delta_h^{(t)} \odot o_t \odot (1 - \tanh^2(c_t)) + \delta_{c,\text{next}} \tag{58}$$
$$\delta_{f,\text{raw}}^{(t)} = (\delta_c^{(t)} \odot c_{t-1}) \odot f_t \odot (1 - f_t) \quad \leftarrow \sigma' \tag{59}$$
$$\delta_{g,\text{raw}}^{(t)} = (\delta_c^{(t)} \odot i_t) \odot (1 - g_t^2) \quad \leftarrow \tanh' \tag{60}$$
$$\delta_{i,\text{raw}}^{(t)} = (\delta_c^{(t)} \odot g_t) \odot i_t \odot (1 - i_t) \quad \leftarrow \sigma' \tag{61}$$

Note: the gradient of c_t with respect to c_{t-1} passes only through the forget gate f_t (no matrix multiplication, no tanh squashing) — this is the gradient highway that prevents vanishing gradients.

Key values during BPTT ($t = 5$ down to $t = 1$):

t	$\delta_{o,\text{raw}}^{(t)}$	$\delta_c^{(t)}$	$\delta_{f,\text{raw}}^{(t)}$	$\delta_{g,\text{raw}}^{(t)}$
5	$[0.0002, 0.0003, 0.0006, 0.0010, -0.0066]$	$[0.0097, 0.0098, 0.0099, 0.0100, -0.0415]$	$[\approx 0]$	$[0.0049, 0.0050, 0.0051, 0.0052, -0.0158]$
4	$[0.0003, 0.0006, -0.0041, 0.0019, 0.0000]$	$[0.0152, 0.0155, -0.0336, 0.0173, -0.0183]$	$[0.0005, 0.0008, -0.0030, \approx 0]$	$[0.0077, 0.0079, -0.0176, 0.0066, -0.0091]$
3	$[0.0006, -0.0040, 0.0013, \approx 0]$	$[0.0186, -0.0304, -0.0095, 0.0212, -0.0004]$	$[0.0010, -0.0027, \approx 0]$	$[0.0095, -0.0159, -0.0036, 0.0106, -0.0002]$
2	$[0.0012, -0.0070, \approx 0]$	$[0.0208, -0.0601, 0.0037, 0.0213, 0.0094]$	$[0.0019, \approx 0]$	$[0.0109, -0.0229, 0.0019, 0.0107, 0.0047]$
1	$[-0.0062, \approx 0]$	$[-0.0286, -0.0303, 0.0118, 0.0215, 0.0150]$	$[\approx 0]$	$[-0.0109, -0.0151, 0.0059, 0.0108, 0.0075]$

8.4 Total Gradients

$$\nabla V = \begin{bmatrix} -0.0238 & 0.0163 & 0.0147 & 0.0122 & 0.0080 \\ -0.0162 & -0.0460 & -0.0265 & 0.0124 & 0.0081 \\ 0.0102 & 0.0044 & -0.0058 & -0.0289 & 0.0082 \\ 0.0167 & 0.0160 & 0.0150 & 0.0132 & 0.0084 \\ 0.0130 & 0.0092 & 0.0027 & -0.0089 & -0.0327 \end{bmatrix} \tag{62}$$

$$\nabla W_{g,x} = \begin{bmatrix} -0.0109 & 0.0109 & 0.0095 & 0.0077 & 0.0049 \\ -0.0151 & -0.0229 & -0.0159 & 0.0079 & 0.0050 \\ 0.0059 & 0.0019 & -0.0036 & -0.0176 & 0.0051 \\ 0.0108 & 0.0107 & 0.0106 & 0.0066 & 0.0052 \\ 0.0075 & 0.0047 & -0.0002 & -0.0091 & -0.0158 \end{bmatrix} \tag{63}$$

The gradients for the gate weight matrices involving h ($W_{f,h}$, $W_{g,h}$, $W_{i,h}$, $W_{o,h}$) are much smaller in magnitude than the x-side gradients because the hidden states are small in early steps (the model is building up memory). The x-side gradients ($W_{f,x}$, $W_{g,x}$, $W_{i,x}$, $W_{o,x}$) are larger because the input signal is binary (one-hot) and directly controls what the LSTM writes into memory at each step.

8.5 Weight Update ($\eta = 0.1$)

$$V_{\text{new}} = \begin{bmatrix} 0.5024 & -0.0016 & -0.0015 & -0.0012 & -0.0008 \\ 0.0016 & 0.5046 & 0.0027 & -0.0012 & -0.0008 \\ -0.0010 & -0.0004 & 0.5006 & 0.0029 & -0.0008 \\ -0.0017 & -0.0016 & -0.0015 & 0.4987 & -0.0008 \\ -0.0013 & -0.0009 & -0.0003 & 0.0009 & 0.5033 \end{bmatrix} \tag{64}$$

$$W_{g,x,\text{new}} = \begin{bmatrix} 0.7011 & -0.0011 & -0.0010 & -0.0008 & -0.0005 \\ 0.0015 & 0.7023 & 0.0016 & -0.0008 & -0.0005 \\ -0.0006 & -0.0002 & 0.7004 & 0.0018 & -0.0005 \\ -0.0011 & -0.0011 & -0.0011 & 0.6993 & -0.0005 \\ -0.0007 & -0.0005 & 0.0000 & 0.0009 & 0.7016 \end{bmatrix} \tag{65}$$

All other updated gate weight matrices are close to their initial values (changes in the third or fourth decimal place), which reflects that the LSTM requires many more training iterations to show significant weight movement. The complete updated matrices for all 8 gate weight pairs are as follows:

$$\begin{aligned} W_{f,h,\text{new}} &\approx 0.4\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0006) \\ W_{f,x,\text{new}} &\approx 0.3\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0003) \\ W_{g,h,\text{new}} &\approx 0.2\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0006) \\ W_{i,h,\text{new}} &\approx 0.5\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0018) \\ W_{i,x,\text{new}} &\approx 0.4\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0060) \\ W_{o,h,\text{new}} &\approx 0.3\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0018) \\ W_{o,x,\text{new}} &\approx 0.4\,\mathbf{I}_5 \quad (\text{changes } \leq 0.0070) \end{aligned}$$

8.6 Key Observations from the LSTM Walkthrough

- **Cell state accumulates gradually.** At step 5, $c_5 = [0.0365, 0.0659, 0.1183, 0.2103, 0.3618]$. Each new word's information is added through $i_t \odot g_t$, while old information is preserved through $f_t \odot c_{t-1}$. The diagonal weight structure means each position in the cell state tracks the corresponding word independently.
- **Forget gate near 0.5 throughout.** Because h_{t-1} is small and the initial weights are small, the forget gate values hover near $\sigma(0) = 0.5$ for most entries. This means roughly half of the old cell state survives at each step. In a trained model, the forget gate would learn to output values closer to 0 or 1 depending on context.
- **Hidden state is much smaller than cell state.** The output gate multiplies $\tanh(c_t)$ by approximately 0.5 (since $W_{o,h}$ and $W_{o,x}$ are small), so $h_t \approx 0.5 \cdot \tanh(c_t)$.

This demonstrates how the output gate filters what the cell state exposes to the output layer.

- **Gradient highway in action.** During BPTT, $\delta_c^{(t)}$ at step 5 is $[-0.0286, -0.0303, 0.0118, 0.0215, 0.0150]$ at step 1 — a non-vanishing gradient that reaches the earliest time step. In the vanilla RNN, the corresponding gradient at step 1 was much closer to zero due to repeated tanh squashing.

9 Encoder-Decoder and Attention

9.1 Why Encoder-Decoder? The Limitation of Sequence Labeling

In sequence labeling tasks like POS tagging, each input token maps to exactly one output token of the same position. This one-to-one alignment breaks down for tasks like machine translation, where the source and target sentences can have different lengths and completely different word orders. A model that pairs each input word rigidly with one output word cannot handle this. The encoder-decoder architecture was introduced to solve this problem.

9.2 The Three Components of an Encoder-Decoder Model

An encoder-decoder network consists of three parts:

- **Encoder:** An RNN that reads the source sequence x_1^n word by word, updating its hidden state at each step. Its final hidden state h_n^e is a compressed representation of the entire input sequence.
- **Context Vector** c: The final encoder hidden state passed to the decoder. It is the only bridge between the encoder and decoder.

$$c = h_n^e$$

- **Decoder:** An RNN initialized with c as its first hidden state $h_0^d = c$. It autoregressively generates the target sequence one token at a time, conditioning each step on the previous hidden state, the previous output token, and the context vector.

$$h_t^d = g(\hat{y}_{t-1},\, h_{t-1}^d,\, c)$$

$$z_t = f(h_t^d), \qquad y_t = \text{softmax}(z_t)$$

Generation continues until an end-of-sequence marker is produced.

9.3 The Sentence Separator Token

To train this model, source and target sentences are concatenated with a separator token `<s>` placed between them. The encoder processes the source side; output is ignored during this phase. After `<s>`, the decoder begins generating the target sequence. The model is trained to minimize cross-entropy loss averaged over all target tokens:

$$L = \frac{1}{T}\sum_{i=1}^{T} L_i, \qquad L_i = -\log P(y_i)$$

9.4 Teacher Forcing

During inference, the decoder feeds its own predicted output $\hat{y}_t$ as input to the next step. If an early prediction is wrong, the error compounds over subsequent steps. During training, this is avoided using **teacher forcing**: the correct gold target token is always used as the next input, regardless of what the decoder predicted. This stabilizes and accelerates training.

9.5 The Bottleneck Problem

The context vector $c = h_n^e$ must encode the entire meaning of the source sentence into a single fixed-size vector. For short sentences this works reasonably well, but for long sentences information from early tokens tends to be poorly represented by the time the encoder reaches the end. This is a **representational bottleneck** — not a vanishing gradient problem, but a capacity problem.

9.6 Attention: Motivation

Instead of relying on a single static context vector, the **attention mechanism** lets the decoder dynamically look at all encoder hidden states at every decoding step. Each encoder hidden state h_j^e corresponds roughly to one input token. Rather than compressing everything into one vector upfront, the decoder computes a fresh, custom context vector c_i at each step i, weighted toward whichever encoder states are most relevant to the token currently being generated.

9.7 Attention: Step-by-Step Computation

Step 1 — Score each encoder state. At decoding step i, a relevance score is computed between the prior decoder hidden state h_{i-1}^d and each encoder hidden state h_j^e. The simplest form is **dot-product attention**:

$$\text{score}(h_{i-1}^d,\, h_j^e) = h_{i-1}^d \cdot h_j^e$$

A high score means those two vectors are similar, indicating that encoder state j is relevant to the current decoding step.

Step 2 — Normalize with softmax. Scores are converted to a probability distribution over all encoder states, giving attention weights α_{ij} that sum to 1:

$$\alpha_{ij} = \frac{\exp(\text{score}(h_{i-1}^d,\, h_j^e))}{\sum_k \exp(\text{score}(h_{i-1}^d,\, h_k^e))}$$

Step 3 — Compute a weighted context vector. The context vector for step i is a weighted average of all encoder hidden states:

$$c_i = \sum_j \alpha_{ij}\, h_j^e$$

Step 4 — Condition the decoder on it. The decoder hidden state is computed using this dynamic context:

$$h_i^d = g(\hat{y}_{i-1},\, h_{i-1}^d,\, c_i)$$

9.8 Bilinear (Parameterized) Attention

A more powerful scoring function introduces a learned weight matrix W_s:

$$\text{score}(h_{i-1}^d,\, h_j^e) = h_{i-1}^d \, W_s \, h_j^e$$

W_s is trained end-to-end alongside the rest of the model. This has two advantages over dot-product attention: it learns *which aspects* of similarity matter for the task, and it allows the encoder and decoder to have different hidden state dimensionalities (dot-product attention requires them to be the same size).

9.9 Architecture of the Encoder RNN

The encoder is not a vanilla RNN. The standard encoder design is a **stacked bidirectional LSTM (biLSTM)**, combining three ideas:

- **LSTM instead of vanilla RNN:** LSTMs use gated cell states to avoid vanishing gradients, allowing reliable retention of information across long sequences. A vanilla RNN cannot encode a long sentence reliably.
- **Bidirectional:** The encoder reads the source in both directions simultaneously. This gives every token's hidden state access to context from both sides of the sentence — essential for a rich representation. The forward and backward hidden states are concatenated at each time step.
- **Stacked (multi-layer):** Multiple biLSTM layers are stacked. Lower layers capture surface patterns; higher layers capture abstract meaning. The context vector is taken from the top layer's final hidden state.

9.10 Architecture of the Decoder RNN

The decoder is typically a **stacked unidirectional LSTM**. It shares two of the encoder's three properties — LSTM gating and multi-layer stacking — but *cannot* be bidirectional. Bidirectionality requires reading the sequence in both directions, but the decoder generates output one token at a time during inference; future tokens do not yet exist. The decoding direction is strictly left-to-right.

Additionally, the decoder receives inputs that a plain encoder never handles: the context vector c and the previously generated token $\hat{y}_{t-1}$ at every step.

9.11 Why Use Two RNNs at All if the Bottleneck Problem Persists?

The encoder-decoder bottleneck problem (compressing everything into one vector) and the vanilla RNN gradient problem (vanishing gradients over time) are two distinct problems. LSTMs largely solve the gradient problem. The bottleneck is a separate issue of representational capacity, not gradient flow.

More importantly, the encoder-decoder architecture solves a problem that a single RNN simply cannot address: generating output sequences of a different length and with non-aligned word mappings relative to the input. A single RNN produces one output per input token — it has no clean mechanism for the encoder-to-decoder handoff needed for

translation. The two-RNN design made this possible. Attention was then added on top to fix the bottleneck that remained.

The historical progression was:

1. Single RNN → works for aligned same-length tasks, fails for translation.
2. Encoder-Decoder → enables translation, struggles on long sequences.
3. Encoder-Decoder + Attention → fixes the bottleneck, handles long sequences.
4. Transformer → removes the RNN entirely, built purely on attention.

9.12 Why Bidirectionality Requires Separate Networks

A bidirectional RNN is by definition two separate RNNs: one running forward and one running backward, each with its own weight matrices. There is no single RNN that can operate in both directions simultaneously — the "bi" in biRNN means two networks.

Therefore, the moment you want bidirectional encoding, you have already committed to multiple components. You cannot have a single RNN switch to bidirectional mode during encoding and then back to unidirectional mode during decoding — those are structurally incompatible requirements that necessitate separate architectures.

The encoder-decoder split is not just an engineering choice. It is the necessary architectural consequence of wanting bidirectional encoding alongside unidirectional decoding. The two phases have fundamentally different requirements, so they require separate components. **Let's say we have an encoder LSTM with 1 input, hidden, output layer and a decoder LSTM of same architechture. We are doing translation from english to spanish. The sentence is "The car is very cool". Our encoder and decoder is using attention mechanism. For each word in our given sentense we will have one hidden state. So, there's gonna be 5 hidden states created by the encoder. When decoder is translating the sentence to spanish, let's say right now decoder is at t = 2, at this step h_2 is going to be one of the inputs. Is my understanding correct?**

Almost, but there's a subtle but important distinction to make.

At decoder step **i=2**, the inputs are:

- $\hat{y}_1$ — the embedding of the word generated at the previous step
- h_1^d — the decoder's previous hidden state (from step 1)
- c_2 — the attention context vector computed at this step

The decoder then produces h_2^d as the **output** of this step, not an input.

So if you meant **encoder** h_2^e ("car"'s hidden state), it's not a direct input either. What happens is that **all five encoder hidden states** $h_1^e \dots h_5^e$ contribute to c_2 via attention weights α_{2j}. h_2^e gets one attention weight, but so do h_1^e, h_3^e, h_4^e, and h_5^e. The final c_2 is:

$$c_2 = \sum_j \alpha_{2j} h_j^e \quad \text{(weighted sum of all 5 encoder states)}$$

And if you meant **decoder** h_2^d — that's what this step *produces*, not what it takes in.

So the correct picture at i=2 is: $\{\hat{y}_1,\ h_1^d,\ c_2\} \rightarrow h_2^d \rightarrow y_2$

The context 'c' that you are talking about, is this calculated at each time step during decoding?

Yes — that's precisely what attention adds over the basic encoder-decoder.

- **Without attention**: $c = h_n^e$ (encoder's final hidden state), computed once, stays fixed across all decoding steps.
- **With attention**: c_i is recomputed at **every** decoder step i, using the current decoder's previous hidden state h_{i-1}^d to score all encoder states and produce a fresh weighted sum.

So for your 5-word sentence, the decoder computes $c_1, c_2, c_3, \ldots$ separately at each step, each one potentially attending to different encoder states depending on what's being translated at that moment.

Give me a concrete example of how c is calculated. I want to know about the details of the calculations.

Let's compute c_2 at decoder step i=2. I'll use 2D hidden state vectors to keep the arithmetic clean.

Given vectors:

Decoder's previous hidden state: $h_1^d = [0.5,\ 0.3]$

All 5 encoder hidden states:

Word	Vector
h_1^e "The"	$[0.2, 0.1]$
h_2^e "car"	$[0.9, 0.8]$
h_3^e "is"	$[0.3, 0.4]$
h_4^e "very"	$[0.1, 0.2]$
h_5^e "cool"	$[0.5, 0.6]$

Step 1 — Score each encoder state against h_1^d (dot product):

```
score(h1d, h1e) = 0.5x0.2 + 0.3x0.1 = 0.13
score(h1d, h2e) = 0.5x0.9 + 0.3x0.8 = 0.69
score(h1d, h3e) = 0.5x0.3 + 0.3x0.4 = 0.27
score(h1d, h4e) = 0.5x0.1 + 0.3x0.2 = 0.11
score(h1d, h5e) = 0.5x0.5 + 0.3x0.6 = 0.43
```

Step 2 — Softmax over scores to get attention weights α_{2j}:

```
exp(0.13)=1.139,  exp(0.69)=1.994,  exp(0.27)=1.310
exp(0.11)=1.116,  exp(0.43)=1.537

sum = 7.096
```

```
a21 = 1.139/7.096 = 0.16  ("The")
a22 = 1.994/7.096 = 0.28  ("car")
a23 = 1.310/7.096 = 0.18  ("is")
a24 = 1.116/7.096 = 0.16  ("very")
a25 = 1.537/7.096 = 0.22  ("cool")
```

These sum to 1. Notice "car" gets the highest weight — h_1^d was most similar to its encoder state.

Step 3 — Weighted sum to get c_2:

```
c2 = 0.16x[0.2,0.1] + 0.28x[0.9,0.8] + 0.18x[0.3,0.4]
   + 0.16x[0.1,0.2] + 0.22x[0.5,0.6]

c2 = [0.47, 0.48]
```

This c_2 now goes into the decoder RNN alongside h_1^d and $\hat{y}_1$ to produce h_2^d. The whole process repeats from Step 1 at every subsequent decoder step, using the newly produced h^d each time.

Does each value in the 'c_2' attention vector represent a meaning?

Not in any human-interpretable way.

Each dimension in c_2 is a learned numerical feature in a high-dimensional space. The network learns to encode useful information into these dimensions during training, but there's no guarantee that dimension 1 means "subject" and dimension 2 means "tense" or anything like that. The representation is distributed and entangled.

What **does** have a clear meaning is the **attention weights** α_{2j} — those are interpretable. In the example, the weight 0.28 for "car" tells you the decoder is leaning most on that word at step i=2. People often visualize these weights as a heatmap to understand what the model is "looking at" while translating each output word.

But c_2 itself — the weighted sum — is just a compressed signal that carries a blend of encoder information, shaped by those weights. Its individual dimensions are opaque.

At t = 2, alpha value for car is highest. So, can I say that while translating to Spanish, the decoder is paying most attention to the word "car"?

Yes, exactly. That's precisely the intuition behind attention weights. At decoder step i=2, the model is focusing most on "car" while generating the corresponding Spanish output word.

This is also why attention weights are considered interpretable — if you visualize them as a heatmap across all decoding steps, you can see which source words the decoder attended to when producing each target word. For translation between languages with different word orders (like English and Spanish), you'd often see the attention pattern shift in a way that reflects the grammatical reordering.

I have seen examples where lenght of source language sentence is not equal to target language sentence. In those case the decoder looks at or pays attention

to multiple words in the source language. But from the above example we see that only the highest value alpha is payed most attention. So how does paying attention to multiple source word work?

The decoder is **always** attending to all source words simultaneously — it's never picking just one. The attention weights are a **distribution**, not a selection.

In the example, the weights were:

```
"The"  = 0.16
"car"  = 0.28  <- highest
"is"   = 0.18
"very" = 0.16
"cool" = 0.22
```

All five contributed to c_2. "car" contributed the most, but "cool" (0.22) and "is" (0.18) also had meaningful influence. The final c_2 was a blend of all of them.

Now for a case like translating "very cool" $\rightarrow$ "muy genial", at that decoder step the attention weights might look like:

```
"The"  = 0.05
"car"  = 0.08
"is"   = 0.07
"very" = 0.38
"cool" = 0.42
```

Here the weight is **spread across two words**, and c at that step becomes a blend heavily influenced by both "very" and "cool" together. The resulting context vector captures information from both, and the decoder uses that combined signal to produce the right output.

So the difference isn't between attending to one word vs multiple words — it's about **where the weight distribution is concentrated**: sharply peaked on one word, or spread across several.

In a regular LSTM, if we are doing POS tagging or sentiment analysis, after processing a sequence, LSTM will calculate loss, perform backpropagation, gradient descent and weight update. This happens during training of course so that we can get optimal weights for testing. But in an encoder (where LSTM is used), the encoder doesn't need to generate any output for translation task. Output is generated by decoder. In that case does the encoder calculates loss? or perform the other tasks that a regular LSTM performs?

The encoder computes **no loss of its own**. Loss is computed only at the decoder side — cross-entropy between the decoder's predicted output words and the gold target words.

But the encoder's weights **do** get updated during training. Here's how:

1. Decoder produces output words
2. Loss is computed at the decoder output

3. Backpropagation flows **backward through the decoder**, then through the attention mechanism, and **continues back into the encoder**
4. Gradients reach the encoder's weights, and gradient descent updates them

So the encoder is trained **indirectly** — it gets no loss signal of its own, but it receives gradients from the decoder's loss. The entire system — encoder, attention, decoder — is trained **end-to-end as one unified model** with a single loss function.

This is actually why attention helps encoder training too. Without attention, the only gradient path into the encoder is through the single context vector c. With attention, gradients flow back through **all encoder hidden states** via the attention weights, giving the encoder much richer gradient signal to learn from.

If I understand correctly, after decoder has completed translation, it computes loss and then during gradient descent the gradient goes back to encoder as well.

Mostly correct, with one small precision: loss isn't computed once after the full translation is done. It's computed **at each decoder timestep** (per-word cross-entropy), then **averaged** into a single sentence-level loss:

$$L = \frac{1}{T} \sum_{i} L_i$$

You can see this in Figure 9.20 from your textbook — each target word has its own loss value, and they get averaged together.

Then backpropagation runs once on that aggregated loss, and the gradients flow backward through all decoder steps (BPTT), through the attention mechanism, and into the encoder. So it's not that each timestep triggers its own backprop — losses are accumulated first, then a single backward pass updates everything.

But your core understanding is right: the gradient from the decoder's loss reaches back into the encoder, which is how the encoder gets trained despite having no loss of its own.

In encoder decoder architechture, loss is calculated in decoding steps. based on those gradients encoder's weights are getting updated. Since we are updating encoder's weight based on decoder's mistake, doesn't that cause wrong weight update? For example, let's say encoder created good context in step but it was the decoder who did not produce good translation. In this scenario encoder is also getting effected due to decoder's bad performence.

This is a sharp observation, and the concern is valid — it's related to the **credit assignment problem** in deep learning. But here's why it still works correctly:

The gradient naturally distributes blame via the chain rule. When backprop flows from the decoder's loss back to the encoder, the magnitude and direction of the gradient reaching the encoder depends on how much the encoder's output actually contributed to the error. If the encoder produced a genuinely useful context vector and the

decoder misused it, the gradient signal reaching the encoder will be relatively small — most of the blame lands on the decoder's weights.

More importantly — there is no independent definition of "good encoder output". The encoder's job is not to produce some universally correct representation. Its job is to produce representations that are *useful to this specific decoder*. The two components co-adapt during training. The encoder learns to produce what helps the decoder, and the decoder learns to use what the encoder provides. They are jointly optimized as one system.

So the "unfairness" you're describing dissolves when you stop thinking of them as independent components and treat them as **one unified model** with shared parameters being optimized toward a single goal.

That said, your concern does point to a real practical challenge — in very deep or complex architectures, gradient signals can be noisy and credit assignment is imperfect. That's why techniques like careful initialization, gradient clipping, and residual connections exist.

You said gradient naturally distributes blame via the chain rule. Do you mean that this issue is automatically handled by chain rule and we don't need to implement specific fix for this problem?

Yes, exactly. The chain rule during backpropagation **automatically** computes how much each parameter — in both encoder and decoder — contributed to the final loss. No special fix is needed for this specific issue.

The gradient reaching any encoder weight is:

$$\frac{\partial L}{\partial W_e} = \frac{\partial L}{\partial \text{decoder output}} \times \frac{\partial \text{decoder output}}{\partial c} \times \frac{\partial c}{\partial W_e}$$

Each term in this chain naturally scales the gradient. If the encoder's contribution to the error was small, the middle term is small, and the gradient reaching W_e is proportionally small. The math handles the blame distribution automatically.

The techniques I mentioned — gradient clipping, residual connections etc. — exist to solve **different** problems like vanishing/exploding gradients in deep networks, not the credit assignment between encoder and decoder specifically. I shouldn't have lumped them in as if they were fixes for what you described.

I still don't understand how encoder and decoder is producing translations where source and target sentence length is different. Let's say in English the sentence is "I am a student", in Spanish the sentence is "Soy un estudiante". I know with attention mechanism, decoder knows which word to focus on in the source sentence. But how does it know it has to generate 3 words? during training?

The decoder doesn't know in advance it needs to generate 3 words. It has no predetermined length. Instead, it learns to generate a special **end-of-sequence token** `</s>` as part of the output.

During training, every target sentence is terminated with </s>. So the model doesn't learn to produce "Soy un estudiante" — it learns to produce **"Soy un estudiante </s>"**. The </s> is treated as just another word in the vocabulary that the softmax predicts over.

So at each decoding step, the model is always answering the same question: *what is the next word?* — and one of the possible answers is always </s>.

For this example the decoder steps look like:

Step	Output
t=1	Soy
t=2	un
t=3	estudiante
t=4	</s>

At t=4 the model predicts </s> with the highest probability, and generation stops.

The model learns **when** to emit </s> entirely from the training data. Across thousands of sentence pairs, it learns that after producing a complete translation, </s> should follow. It doesn't count words — it learns the pattern of when meaning has been fully expressed.

So length difference between source and target is not a problem the model explicitly solves. It's a natural consequence of autoregressive generation with a learned stop signal.

If source sentence has 5 words and target sentence has 3, will the decoder produce 2 ¡/s¿ tokens?

No, just one </s>. Generation **stops immediately** when the first </s> is predicted. The decoder doesn't run for a fixed number of steps equal to the source length — it runs until </s> appears.

The source sentence length has no bearing on how many steps the decoder runs. The decoder is entirely driven by its own autoregressive output. It could produce 2 words, 10 words, or 50 words regardless of whether the source had 5 words. The source length and target length are completely decoupled.

But as far as I know if decoder is an LSTM, then a LSTM must know how many times it will unroll.

You're right, and this is a good distinction to make between training and inference.

During training — the target sentence is known in advance. So the decoder LSTM unrolls exactly as many steps as the target sentence length (including </s>). For "Soy un estudiante </s>" that's 4 steps. The unroll length is fixed per training example, determined by the gold target sentence.

During inference — there is no gold target sentence, so the length is unknown upfront. Here the decoder doesn't unroll all at once. It runs **one step at a time**, checks if the output is </s>, and if not, feeds that output back in as the next input and runs another step. It's a sequential loop, not a fixed unroll.

So:

- Training = static unroll, length known from target sentence

- Inference = dynamic step-by-step loop, stops when `</s>` is predicted

This is actually the same training vs inference distinction you already know — teacher forcing during training gives you the full target sequence upfront, which is precisely what lets you do the fixed unroll. At inference you have no such luxury, so you go step by step.

10 Transformer

In transformer architechture, the encoders and decoders are also neural networks. is that correct?

Yes, but with precision: the encoder and decoder are not separate standalone neural networks — they are **stacked layers that together form the transformer**, which is itself a neural network.

Each encoder block contains:

- A **multi-head self-attention** layer
- A **feed-forward neural network** (a small 2-layer fully connected network)

Each decoder block contains those two, plus:

- A **cross-attention** layer (attends to the encoder's output)

So the whole transformer — encoder stack + decoder stack — is one neural network trained end-to-end. The "encoder" and "decoder" are just architectural roles within it, not independently trained nets.

Is transformer a seq2seq architechture as well?

Yes. The original transformer (Vaswani et al., 2017) was explicitly designed as a seq2seq model — it takes a sequence as input and produces a sequence as output (e.g., translating a sentence from English to German).

The key difference from earlier seq2seq models (like RNN/LSTM-based ones) is **how** it processes sequences — in parallel via attention, rather than step-by-step recurrently.

That said, not all transformers today are seq2seq:

- **Encoder-only** (e.g., BERT) — takes a sequence, outputs contextual representations. Used for classification, NER, etc.
- **Decoder-only** (e.g., GPT) — takes a sequence, generates the next token autoregressively. Used for language generation.
- **Encoder-decoder** (e.g., original Transformer, T5, BART) — true seq2seq. Used for translation, summarization, etc.

So "transformer" is a broader architectural family; seq2seq is one configuration of it.

In the "Attention is all you need" paper, the encoding component was a stack of 6 encoders. Here what exactly is meant by 6 encoders? Is it 6 neural networks stacked?

"6 encoders" means **6 identical-in-structure blocks stacked sequentially**, where the output of one block becomes the input of the next.

Each block has its own **independent weights** — they don't share parameters. So they're not copies of each other at runtime, even though they have the same architecture.

But they're not 6 separate neural networks in any meaningful sense. It's **one neural network** with 6 repeated architectural units. The whole stack is trained end-to-end together via backpropagation — gradients flow through all 6 blocks in one pass.

Why stack 6? Each layer learns increasingly abstract representations. Early layers might capture syntax, later layers semantics — similar to how deep CNNs learn edges → textures → objects. The number 6 was an empirical choice; more layers generally means more capacity, but with diminishing returns and higher compute cost.

by 6 neural networks I don't mean 6 different neural networks. It can be same or different. What I want to know is were there 6 neural networks or not?

Yes. There are 6 encoder blocks, each with its own independent weights, and each qualifies as a neural network. So yes — 6 neural networks stacked sequentially.

Previously you said 1 neural network that is repeated 6 times as architechtural units. Was any special kind of neural network used like RNN or LSTM?

On the contradiction — both statements are valid depending on the lens. The whole transformer is one end-to-end trained network; each block is also individually a neural network. It's a matter of framing, not a factual conflict.

On your actual question: No RNNs or LSTMs. That's precisely the point of the paper — it replaced them entirely.

Each encoder block uses:

- **Multi-head self-attention** — not a recurrent network. It processes all tokens in the sequence simultaneously using Query, Key, Value matrix operations.
- **Feed-forward network** — a simple 2-layer fully connected (dense) network, i.e., a basic MLP. Applied independently to each position.

RNNs/LSTMs process sequences step-by-step, which is slow and struggles with long-range dependencies. The transformer eliminates that by using attention to relate any two positions in the sequence directly, regardless of distance — all in parallel.

I see encoder has following components: Multi Head Self Attention, Add & Norm, Feed Forward Neural Network and another Add & Norm. This makes one encoder. So, all these components were repeated 6 times in the original paper. Right?

Yes, exactly right. One encoder block = Multi-Head Self-Attention → Add & Norm → Feed-Forward Network → Add & Norm. That entire block was repeated 6 times in the original paper.

Does an Encoder itself does the job of word embedding?

No. Word embedding happens **before** the encoder stack. The input tokens are first converted to vectors via an **embedding layer**, then **positional encoding** is added to inject sequence order information. Only after these two steps does the result get fed into the first encoder block.

The encoder's job is to **transform and contextualize** those embeddings — not to create them.

In Jay Alammar blogs he says "The embedding only happens in the bottom-most encoder. The abstraction that is common to all the encoders is that they receive a list of vectors each of the size 512 – In the bottom encoder that would be the word embeddings, but in other encoders, it would be the output of the encoder that's directly below. The size of this list is hyperparameter we can set – basically it would be the length of the longest sentence in our training dataset." Why does he say that the embedding only happens in the bottom most encoder?

It's a **pedagogical framing choice** by Jay Alammar, not a claim that the embedding layer is literally inside the encoder block.

What he means is:

- Raw tokens enter the system only once — at the **bottom of the stack**
- That's where embedding + positional encoding happens, right before the first encoder block processes them
- Encoders 2 through 6 never see raw tokens — they receive the **output vector from the encoder below them**

So he's associating the embedding step with the bottom encoder because that's the entry point of the whole stack. It's describing **data flow**, not saying the encoder block itself contains the embedding layer.

The factual picture remains the same as before — embedding is a separate step that feeds into the first encoder. Jay Alammar is just grouping it conceptually with the bottom encoder to make the layered flow easier to visualize.

Is the multihead attension in encoder same as self attention?

Yes. In the encoder, multi-head attention **is** self-attention — the "multi-head" part just describes **how** it's implemented.

- **Self-attention** means the Queries, Keys, and Values all come from the **same sequence** (the input itself). Each token attends to every other token in the same sequence.
- **Multi-head** means this self-attention operation is run **multiple times in parallel** with different learned weight matrices, then the results are concatenated and projected. This allows the model to attend to different aspects of the sequence simultaneously.

So multi-head attention in the encoder = self-attention, done multiple times in parallel.

The term "multi-head attention" becomes important to distinguish in the **decoder**, where one of its attention layers is **cross-attention** — Queries come from the decoder, but Keys and Values come from the encoder output. That one is not self-attention. **Before attention mechanism starts in transformers, positional encoding is calculated first. Right?**

Yes, that's correct. The sequence is:

Token → Embedding → + Positional Encoding → Attention

Specifically: input tokens are first converted to dense vectors via the embedding layer, then positional encodings are **added** to those vectors element-wise, and *that combined representation* is what gets fed into the attention mechanism.

This happens only **once at the input** (before the first layer), not before every attention layer. The attention mechanism itself has no built-in sense of order, so positional encoding must be baked in beforehand.

Why use sine for even and cos for odd positions in the word embedding?

The core reason is **relative position representability**.

Using the trig identity:

- $\sin(\text{pos} + k) = \sin(\text{pos}) \cdot \cos(k) + \cos(\text{pos}) \cdot \sin(k)$
- $\cos(\text{pos} + k) = \cos(\text{pos}) \cdot \cos(k) - \sin(\text{pos}) \cdot \sin(k)$

This means **PE(pos + k) is a linear function of PE(pos)** for any fixed offset k. The model can learn to express "k positions ahead" as a linear transformation — which is something attention weights can do easily.

If you only used sine (or only cosine), you'd lose this property because sin alone is symmetric: $\sin(\theta) = \sin(\pi - \theta)$, so two different positions could produce the same value. **Pairing sin + cos at each frequency makes each position uniquely identifiable**, just like (x, y) on a unit circle uniquely identifies an angle.

Intuitively: each $(\sin, \cos)$ pair at frequency i is like a 2D clock hand rotating at that frequency. Position 0 vs position 5 vs position 100 each land at different points on that circle. Stack many such clocks at different frequencies → a unique fingerprint per position.

So the even/odd split isn't the deep insight — the **pairing itself** is what matters. Even/odd is just the implementation convention chosen in the original paper.

This means PE(pos + k) is a linear function of PE(pos) for any fixed offset k. The model can learn to express "k positions ahead" as a linear transformation. What do you mean by k positions ahead?

Say you have a sentence: **"The cat sat on the mat"**

- "cat" is at position 1
- "sat" is at position 2
- "on" is at position 3

If $k = 2$, then from "cat" (pos=1), **"k positions ahead"** means "on" (pos=3).

The point is: when the model is processing position `pos` and needs to **attend to** something 2 positions later, it doesn't have to learn a completely new representation for every pair of positions. Instead, because PE(pos + k) is a linear transformation of PE(pos), the model can learn **one general operation** — "shift by 2" — and apply it anywhere in the sequence.

Without this property, the relationship between position 1 and position 3 would look completely unrelated to the relationship between position 50 and position 52, even though the **relative gap is the same**. The model would have to learn each pair independently.

With it, relative distances are structurally consistent — "2 steps ahead" always looks like the same kind of transformation, regardless of where you are in the sentence.

Explain with an example. I am still not clear on this.

Let's use: **"The cat sat on the mat"** (positions 0–5)

Using a simplified PE with just one frequency (just two numbers per position):

Word	Position	PE (sin, cos)
cat	1	(0.84, 0.54)
sat	2	(0.91, -0.42)
on	3	(0.14, -0.99)
the	4	(-0.76, -0.65)

Now let's say $k = 2$ (two positions ahead).

There exists a **fixed matrix** M_2 (depends only on $k = 2$, not on position) such that:

$M_2 \times \text{PE}(1) = \text{PE}(3) \rightarrow$ Apply M_2 to "cat" $\rightarrow$ you get "on" ✓

$M_2 \times \text{PE}(2) = \text{PE}(4) \rightarrow$ Apply M_2 to "sat" $\rightarrow$ you get "the" ✓

The **same matrix** works from any starting position. "2 steps ahead" is always the same transformation.

Why does this matter for the model?

Without this property:

- The gap between pos 1 and pos 3 looks numerically unrelated to the gap between pos 2 and pos 4.
- The model has to separately learn what "2 apart" means for every pair — which is expensive and doesn't generalize.

With this property:

- The model learns **one operation** for "2 apart" and it works everywhere in the sequence.
- So when attention weights are computed, relative distances are structurally consistent — the model can generalize "these two words are 2 apart" without memorizing every case.

That's the practical payoff of PE(pos + k) being a linear function of PE(pos).

You talked about a matrix M. Where does this come from?

It comes directly from **trig identities** — it's not learned, it's mathematically derivable.

At a single frequency ω, PE(pos) is just:

$$\text{PE}(\text{pos}) = \begin{pmatrix} \sin(\text{pos} \cdot \omega) \\ \cos(\text{pos} \cdot \omega) \end{pmatrix}$$

Now expand PE(pos + k) using the angle addition identities:

- $\sin((\text{pos} + k) \cdot \omega) = \sin(\text{pos} \cdot \omega) \cdot \cos(k \cdot \omega) + \cos(\text{pos} \cdot \omega) \cdot \sin(k \cdot \omega)$
- $\cos((\text{pos} + k) \cdot \omega) = \cos(\text{pos} \cdot \omega) \cdot \cos(k \cdot \omega) - \sin(\text{pos} \cdot \omega) \cdot \sin(k \cdot \omega)$

Write this as a matrix multiplication:

$$\text{PE}(\text{pos} + k) = M_k \times \text{PE}(\text{pos}), \quad \text{where } M_k = \begin{pmatrix} \cos(k\omega) & \sin(k\omega) \\ -\sin(k\omega) & \cos(k\omega) \end{pmatrix}$$

Key observation: every entry in M_k depends only on k **and** ω — not on `pos` at all. So the same matrix works from any starting position.

This M_k is actually a **rotation matrix** — it rotates the $(\sin, \cos)$ vector by angle $k \cdot \omega$. Which also explains the "clock hand" intuition: shifting k positions forward is literally rotating the clock hand by a fixed angle.

So to directly answer your question: **M comes from algebra**, not from the model. It's a consequence of choosing sin/cos — those functions happen to have this rotation property baked in. That's precisely *why* sin/cos were chosen in the first place.

PE(pos + k) = M_k × PE(pos). Explain this. Is this a trigonometry thing?

Yes, purely trig. Let me unpack it mechanically.

PE(pos) is a **2D column vector**:

$$\text{PE}(\text{pos}) = \begin{pmatrix} \sin(\text{pos} \cdot \omega) \\ \cos(\text{pos} \cdot \omega) \end{pmatrix}$$

M_k is a **2×2 matrix**:

$$M_k = \begin{pmatrix} \cos(k\omega) & \sin(k\omega) \\ -\sin(k\omega) & \cos(k\omega) \end{pmatrix}$$

Now do the multiplication $M_k \times \text{PE}(\text{pos})$ row by row:

Row 1:

$$\cos(k\omega) \cdot \sin(\text{pos} \cdot \omega) + \sin(k\omega) \cdot \cos(\text{pos} \cdot \omega) = \sin((\text{pos} + k)\omega)$$

Row 2:

$$-\sin(k\omega)\cdot\sin(\text{pos}\cdot\omega)+\cos(k\omega)\cdot\cos(\text{pos}\cdot\omega)=\cos((\text{pos}+k)\omega)$$

So the result is:

$$\begin{pmatrix}\sin((\text{pos}+k)\omega)\\ \cos((\text{pos}+k)\omega)\end{pmatrix}=\text{PE}(\text{pos}+k)$$

Those two expansions in Row 1 and Row 2 **are exactly the angle addition formulas** from trigonometry:

- $\sin(A+B)=\sin A\cdot\cos B+\cos A\cdot\sin B$
- $\cos(A+B)=\cos A\cdot\cos B-\sin A\cdot\sin B$

So $\text{PE}(\text{pos}+k)=M_k\times\text{PE}(\text{pos})$ is just those trig identities **written in matrix form**. The matrix notation doesn't add new math — it just packages the identities compactly to show that shifting position by k is a single linear operation on the PE vector.

What do you mean by PE(pos) and PE(pos + k)?

PE(pos) means **the positional encoding vector of the word at position** `pos`.

Concrete example with our sentence: **"The cat sat on the mat"**

- PE(1) = the positional encoding vector of **"cat"** (position 1)
- PE(3) = the positional encoding vector of **"on"** (position 3)

So when we write **PE(pos + k)** with pos=1 and k=2:

PE(1 + 2) = PE(3) = the positional encoding vector of **"on"**

In other words:

- `pos` is just a **position index** (0, 1, 2, 3...)
- `PE(pos)` is the **vector assigned to that position** using the sin/cos formula
- `PE(pos + k)` is the **vector assigned to the position k steps later**

So the whole statement $\text{PE}(\text{pos}+k)=M_k\times\text{PE}(\text{pos})$ simply means:

"The encoding of the word at position 3 can be obtained by applying matrix M to the encoding of the word at position 1"

It's a relationship **between two encoding vectors** at two different positions.

When we are processing (It can be anything) a word at position pos and we need another word at position pos + k, We don't need to recalculate the entire positional encoding for pos + k. We just need to apply matrix M. This is the advantage of using sin and cos. Is my understanding correct?

Your intuition is in the right direction, but the framing needs a small correction.

In practice, **positional encodings are precomputed for all positions upfront** before any processing happens. So there's no runtime recalculation to avoid — PE(pos+k) is already sitting there, computed directly from the sin/cos formula.

The **actual advantage** is different, and it's about the **attention mechanism**, not computation.

When attention computes how much word at `pos` should attend to word at `pos+k`, it works with their vectors. Because $\text{PE}(\text{pos} + k) = M_k \times \text{PE}(\text{pos})$, the relationship between any two positions that are k apart is **always the same linear transformation**.

So the model learns **one general concept** — "these two words are k apart" — and it applies everywhere. Position 1 attending to position 3 looks structurally identical to position 50 attending to position 52, because the same M_k connects them.

In short:

- Your version: sin/cos saves recomputation → **not quite**, PEs are precomputed anyway
- Correct version: sin/cos makes relative distances **structurally consistent**, so the attention mechanism can generalize relative positions instead of memorizing every pair independently

Explain attention in transformers in plain english, assume you are explaining to an 8th grade student.

Attention in Transformers — Plain English

Imagine you're reading this sentence:

> *"The trophy didn't fit in the suitcase because **it** was too big."*

What does **"it"** refer to — the trophy or the suitcase? You figured it out instantly because your brain paid more *attention* to "trophy" than to "suitcase" when reading the word "it."

That's exactly what attention does in a Transformer — it lets every word look at every other word in a sentence and ask: **"Who here is most relevant to me?"**

The Library Analogy

Think of it like searching in a library:

- You walk in with a **question** (this is called the **Query**) — *"I'm looking for books about space."*
- Every book has a **label on its spine** (this is the **Key**) — *"Physics," "Cooking," "Astronomy"...*
- Inside each book is the actual **content** (this is the **Value**).

You compare your question against every book's label. The more a label matches your question, the more you read from that book's content. That weighted mix of content is what you walk out with.

Every word in a sentence does this same process — simultaneously, for every other word.

What Actually Happens, Step by Step

1. Each word gets turned into three things: a **Query**, a **Key**, and a **Value** — just lists of numbers.
2. The word compares its Query against every other word's Key (using a dot product — basically a similarity score).
3. Those scores get converted into percentages that add up to 100% (this is called **softmax**). High percentage = pay a lot of attention. Low percentage = mostly ignore.
4. Finally, the word collects a **weighted mix** of everyone's Values, based on those percentages.

The result? Every word now carries information not just about itself, but about the whole sentence — specifically the parts most relevant to it.

Why It's Powerful

Before Transformers, models read sentences word-by-word, left to right, like reading with one eye closed. Attention lets every word look at the entire sentence at once, so the model can understand context much better — no matter how far apart the related words are.

Multi-Head Attention (Bonus)

The Transformer actually runs this attention process **8 times in parallel**, each looking for *different kinds of relationships* — one head might track grammar, another tracks meaning, another tracks references like "it → trophy." Then all 8 results get combined into one final answer. Think of it as 8 people reading the same sentence, each underlining different things, then sharing notes.

Give me a concrete math walkthrough of attention calculations. For a single word.

Concrete Math Walkthrough: Attention for Word "cat"

Sentence: "The cat sat" — we'll compute attention for **"cat"** (word 2).

We use tiny 2D vectors so the arithmetic is followable by hand. In real transformers these are 64D, but the math is identical.

Setup

```
x1 = [1, 0]   -> "The"
x2 = [0, 1]   -> "cat"   <- our focus word
x3 = [1, 1]   -> "sat"
```

The model has learned weight matrices W_Q, W_K, W_V. Applying them to every embedding gives us Q, K, V vectors for each word. For this example, assume the results are already computed:

```
q2 = [1, 1]   <- "cat"'s Query

k1 = [1, 0]   <- "The"'s Key
k2 = [0, 1]   <- "cat"'s Key
k3 = [1, 1]   <- "sat"'s Key

v1 = [1, 0]   <- "The"'s Value
v2 = [0, 1]   <- "cat"'s Value
v3 = [1, 1]   <- "sat"'s Value
```

Step 1 — Raw Scores (dot product: $q_2 \cdot$ each key)

"cat" asks: *how relevant is each word to me?*

Dot product = multiply element-by-element, then add.

```
score("The") = q2 . k1 = (1x1) + (1x0) = 1
score("cat") = q2 . k2 = (1x0) + (1x1) = 1
score("sat") = q2 . k3 = (1x1) + (1x1) = 2
```

Raw scores: **[1, 1, 2]**

"sat" scored highest — "cat" finds "sat" most relevant. Makes sense.

Step 2 — Scale by $\sqrt{d_k}$

Raw scores tend to get very large in practice (vectors are 64D, not 2D), which breaks softmax. We divide by $\sqrt{d_k} = \sqrt{2} \approx 1.414$ to stabilize them.

```
[1/1.414, 1/1.414, 2/1.414]
= [0.707, 0.707, 1.414]
```

Step 3 — Softmax

Convert scores into percentages that add up to 1.

Softmax formula: for each score $s_i \rightarrow e^{s_i} / \sum(\text{all } e^s)$

```
e^0.707 ~ 2.028
e^0.707 ~ 2.028
e^1.414 ~ 4.113

Sum = 8.169

a1 = 2.028 / 8.169 ~ 0.248   -> pay 24.8% attention to "The"
a2 = 2.028 / 8.169 ~ 0.248   -> pay 24.8% attention to "cat"
a3 = 4.113 / 8.169 ~ 0.504   -> pay 50.4% attention to "sat"
```

Total: $0.248 + 0.248 + 0.504 = 1.0$ ✓

Step 4 — Weighted Sum of Values

```
output = 0.248 x v1  +  0.248 x v2  +  0.504 x v3
       = 0.248x[1,0] + 0.248x[0,1] + 0.504x[1,1]
       = [0.248, 0.000]
       + [0.000, 0.248]
       + [0.504, 0.504]
       -----------------
       = [0.752, 0.752]
```

Result

"cat"'s new representation = [0.752, 0.752]

This is no longer just "cat" in isolation. It's a blend — mostly pulled toward "sat" (50.4%), with smaller contributions from itself and "The". The word "cat" now carries context from the whole sentence, weighted by relevance.

The Full Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Each step maps directly:

Formula piece	What we did
$QK^\top$	dot products → [1, 1, 2]
$\div\sqrt{d_k}$	divide by 1.414 → [0.707, 0.707, 1.414]
softmax$(\cdot)$	→ [0.248, 0.248, 0.504]
$\times V$	weighted sum → [0.752, 0.752]

The final representation of the word is called z in the lecture slides. Right?

Yes. Both sources use $\mathbf{Z}$ for this.

In the lecture notes, the final output of the attention step for a single word is written as lowercase $\mathbf{z}$, as in:

$$z = \alpha_1 v_1 + \alpha_2 v_2$$

And when written in matrix form (all words at once), it's uppercase $\mathbf{Z}$:

$$Z = AV$$

So in the walkthrough we did, the [0.752, 0.752] we computed for "cat" — that's z_2.

In multihead attention, let's say we are using 8 heads. For all 3 words (The, cat, sat) we have calculated 'z' once. Before this z values goes into feed forward network, different 'z' will be calculated 7 times. Right?

Almost right — the count is correct, but one nuance: it's not "calculate z once, then 7 more times." All 8 heads run **in parallel**, each with their own separate W_Q, W_K, W_V weight matrices. So you get 8 different Z matrices simultaneously — $Z_1, Z_2, \ldots, Z_8$.

Each head sees the same input words but learns to attend differently, so each Z captures different relationships.

But 8 separate Z matrices can't go into the FFN directly — the FFN expects a single Z. So before passing forward:

1. All 8 Z matrices are **concatenated** side by side into one big matrix.
2. That big matrix is multiplied by a learned weight matrix W_O to compress it back to the original size.

Only then does the single combined Z go into the feed-forward network.

So the flow is:

```
Input -> [Head1, Head2, ..., Head8] -> [Z1, Z2, ..., Z8]
      -> Concatenate -> x W_O -> single Z -> FFN
```

Give me a concrete math example to clear the concept of matrix dimensions during z calculations and then passing to feed forward. How the dimensions are handled?

I'll use small numbers to keep the math trackable. The principles are identical to a real transformer.

Setup

```
Sentence:  "The  cat  sat"   -> 3 words
d_model = 4    (real transformer: 512)
Heads   = 2    (real transformer: 8)
d_k     = d_model / heads = 4/2 = 2    (real: 512/8 = 64)
```

Input Matrix X — shape (3×4)**:**

```
X = [1, 0, 1, 0]   <- "The"
    [0, 1, 0, 1]   <- "cat"
    [1, 1, 0, 0]   <- "sat"
```

Step 1 — Each Head Computes Its Own Q, K, V

Each head has its own weight matrices: W_Q, W_K, W_V — each shaped (4×2).

Multiplying X (3×4) by W_Q (4×2) gives Q of shape (3×2). Same for K and V.

```
Q = X . W_Q  ->  (3x4).(4x2) = (3x2)
K = X . W_K  ->  (3x4).(4x2) = (3x2)
V = X . W_V  ->  (3x4).(4x2) = (3x2)
```

This happens **independently for each head**, with different weight matrices.

Step 2 — Compute Z for Each Head

```
Scores = Q . K^T  ->  (3x2).(2x3) = (3x3)   <- one score per word-pair
Scores / sqrt(d_k)  ->  (3x3)                <- scale
Softmax             ->  (3x3)                <- attention weights A
Z = A . V           ->  (3x3).(3x2) = (3x2)  <- output
```

Each head produces a Z of shape (3×2).

```
Z1 = (3x2)   <- from head 1
Z2 = (3x2)   <- from head 2
```

Step 3 — Concatenate All Z's

Stack Z_1 and Z_2 side by side along the column axis:

```
Z1           Z2           Concatenated
[a, b]       [e, f]   ->  [a, b, e, f]
[c, d]  +    [g, h]   ->  [c, d, g, h]
[i, j]       [k, l]   ->  [i, j, k, l]
```

Shape: (3×4) — back to original width.

With 8 heads in a real transformer: $8 \times 64 = 512$ columns $\rightarrow (n \times 512)$ ✓

Step 4 — Multiply by W_O

W_O has shape (4×4) — it mixes and compresses the concatenated output.

```
Z_final = Concat . W_O  ->  (3x4).(4x4) = (3x4)
```

Final Dimension Flow

```
X         W per head    Q/K/V      Scores    Z per head  Concat   W_O     Z_final
(3x4)  .  (4x2)     =  (3x2)  ->  (3x3)  -> (3x2)   -> (3x4) . (4x4) = (3x4)
```

Z_{final} is (3×4) — exactly the same shape as the original input X.

Key Insight on Why Dimensions Work Out

Thing	Shape	Why
X	$3 \times d_{\text{model}}$	3 words, each d_{model} wide
$W_Q/W_K/W_V$ per head	$d_{\text{model}} \times d_k$	projects down to smaller slice
Q, K, V per head	$3 \times d_k$	each word gets a smaller representation
Z per head	$3 \times d_k$	same size as V
Concatenated	$3 \times (\text{heads} \times d_k)$	$= 3 \times d_{\text{model}}$
After W_O	$3 \times d_{\text{model}}$	back to original width

heads $\times\, d_k = d_{\text{model}}$ is the key relationship that guarantees the dimensions stay consistent end-to-end.

In the above example, Q, K, V matrixes have dimension 4 * 2. Is it because each word is represented with 4 numbers and there are 2 heads?

Close, but not exactly. Let me be precise.

W_Q, W_K, W_V are shaped (4×2) where:

- **4 rows** → because the input vector for each word has 4 numbers ($d_{\text{model}} = 4$). The matrix must accept 4 numbers in.
- **2 columns** → because $d_k = 2$, which is the size each head works with. Not directly "because there are 2 heads", but because $d_k = d_{\text{model}}/\text{heads} = 4/2 = 2$.

So the number of heads influences the 2 **indirectly** — it determines how small each head's slice is. If you had 4 heads instead, d_k would be $4/4 = 1$, and W_Q would be (4×1).

Also a small correction on terminology — to be precise:

- W_Q, W_K, W_V → the weight matrices, shape (4×2)
- Q, K, V → the result after multiplying X by those weights, shape (3×2)

The 3 in Q, K, V comes from the number of words, not from the weight matrix itself.

```
X       .    W_Q    =    Q
(3x4)      (4x2)       (3x2)
 ^                       ^
words                words, but now
                     in d_k space
```

Now show me a concrete math example of multihead attention. Assume each word is represented with 2 numbers and head = 2.

Setup

```
Sentence : "The  cat  sat"   -> 3 words
d_model  = 2
heads    = 2
d_k      = d_model / heads = 2/2 = 1
```

Input X — shape (3×2)**:**

```
X = [1, 0]   <- "The"
    [0, 1]   <- "cat"
    [1, 1]   <- "sat"
```

Head 1

Weight matrices — shape (2×1):

```
W_Q1 = [1]     W_K1 = [1]     W_V1 = [1]
       [0]            [0]            [0]
```

Head 1 focuses on the **first number** of each word's embedding.

Q1, K1, V1 = $\mathbf{X \cdot W}$ **— shape** (3×1)**:**

```
Q1 = K1 = V1 = [1]   <- "The"
               [0]   <- "cat"
               [1]   <- "sat"
```

Scores = $\mathbf{Q1 \cdot K1^\top}$ **— shape** $(3 \times 1) \cdot (1 \times 3) = (3 \times 3)$**:**

```
        The  cat  sat
"The" [  1,   0,   1 ]
"cat" [  0,   0,   0 ]
"sat" [  1,   0,   1 ]
```

Scale by $\sqrt{d_k} = \sqrt{1} = 1 \rightarrow$ no change.

Softmax row by row:

```
"The": [1, 0, 1] -> [0.42, 0.15, 0.42]
"cat": [0, 0, 0] -> [0.33, 0.33, 0.33]
"sat": [1, 0, 1] -> [0.42, 0.15, 0.42]
```

Z1 = A1 · V1 — shape $(3 \times 3) \cdot (3 \times 1) = (3 \times 1)$**:**

```
"The": 0.42x1 + 0.15x0 + 0.42x1 = 0.84
"cat": 0.33x1 + 0.33x0 + 0.33x1 = 0.66
"sat": 0.42x1 + 0.15x0 + 0.42x1 = 0.84

Z1 = [0.84]   <- "The"
     [0.66]   <- "cat"
     [0.84]   <- "sat"
```

Head 2

Weight matrices — shape (2×1):

```
W_Q2 = [0]    W_K2 = [0]    W_V2 = [0]
       [1]           [1]           [1]
```

Head 2 focuses on the **second number** of each word's embedding.

Q2, K2, V2 = X · W — shape (3×1)**:**

```
Q2 = K2 = V2 = [0]   <- "The"
               [1]   <- "cat"
               [1]   <- "sat"
```

Scores = Q2 · $\mathbf{K2}^\top$ **— shape** (3×3)**:**

```
        The  cat  sat
"The" [  0,   0,   0 ]
"cat" [  0,   1,   1 ]
"sat" [  0,   1,   1 ]
```

Softmax:

```
"The": [0, 0, 0] -> [0.33, 0.33, 0.33]
"cat": [0, 1, 1] -> [0.15, 0.42, 0.42]
"sat": [0, 1, 1] -> [0.15, 0.42, 0.42]
```

Z2 = A2 · V2 — shape (3×1)**:**

```
"The": 0.33x0 + 0.33x1 + 0.33x1 = 0.66
"cat": 0.15x0 + 0.42x1 + 0.42x1 = 0.84
"sat": 0.15x0 + 0.42x1 + 0.42x1 = 0.84
```

```
Z2 = [0.66]   <- "The"
     [0.84]   <- "cat"
     [0.84]   <- "sat"
```

Concatenate Z1 and Z2 — shape (3×2)

```
Z1      Z2      Concatenated
[0.84]  [0.66]  ->  [0.84, 0.66]   <- "The"
[0.66]  [0.84]  ->  [0.66, 0.84]   <- "cat"
[0.84]  [0.84]  ->  [0.84, 0.84]   <- "sat"
```

Shape: (3×2) ✓

Multiply by W_O **— shape** (2×2)

```
W_O = [1, 0]
      [0, 1]
```

(identity for simplicity — in reality this is learned)

Z_{final} **= Concat** $\cdot\, W_O$ **— shape** $(3 \times 2) \cdot (2 \times 2) = (3 \times 2)$**:**

```
Z_final = [0.84, 0.66]   <- "The"
          [0.66, 0.84]   <- "cat"
          [0.84, 0.84]   <- "sat"
```

Dimension Flow Summary

```
X       W per head  Q/K/V    Scores   Z per head  Concat  W_O    Z_final
(3x2) . (2x1)    = (3x1) -> (3x3) -> (3x1)    -> (3x2) . (2x2) = (3x2)
```

Z_{final} is (3×2) — same shape as input X. The FFN receives exactly this.

What the Two Heads Learned (intuitively)

Head 1 only saw the first number in each embedding — "The" and "sat" both had 1 there, so they attended to each other strongly. Head 2 only saw the second number — "cat" and "sat" both had 1, so they attended to each other. Two heads, two different relationships found simultaneously.

After calculating scores in MHA, why divide by d_k**?**

The dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ is a sum of d_k terms. If each component has mean 0 and variance 1, the dot product has **mean 0 but variance** d_k — it grows linearly with d_k.

At large d_k (e.g., 512), these raw scores become very large. When large values go into softmax, one score dominates and gets weight ≈ 1 while all others collapse to ≈ 0. This is **softmax saturation**.

Saturated softmax has near-zero gradients — the output barely changes even when inputs change, so the model stops learning effectively.

The fix: divide by $\sqrt{d_k}$, which scales the variance back down:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) \propto \frac{d_k}{(\sqrt{d_k})^2} = \frac{d_k}{d_k} = 1$$

It is the unique scaling that keeps variance constant regardless of d_k — not too compressed, not too large. That is why the full formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Consider this example: our sentence is "There is a cat". In MHA, each word will have its own Q, K, V matrix. So, in total there will be 12 Matrices for one head. Right?

Not quite — the count of 12 is right but the framing is off.

Each word does not get its own **matrix**. Each word gets its own **vectors** — one q, one k, one v. That is 4 words $\times$ 3 = 12 vectors total per head.

But in practice, those vectors are stacked into **3 matrices** per head:

$$Q = \begin{bmatrix} q_{\text{There}} \\ q_{\text{is}} \\ q_{\text{a}} \\ q_{\text{cat}} \end{bmatrix}, \quad K = \begin{bmatrix} k_{\text{There}} \\ k_{\text{is}} \\ k_{\text{a}} \\ k_{\text{cat}} \end{bmatrix}, \quad V = \begin{bmatrix} v_{\text{There}} \\ v_{\text{is}} \\ v_{\text{a}} \\ v_{\text{cat}} \end{bmatrix}$$

Each is a $4 \times d_k$ matrix. So per head: **3 matrices**, each holding all 4 words' worth of that role.

These are computed using the shared weight matrices for that head:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

where X is the full input ($4 \times d_{\text{model}}$) and the weight matrices are what is learned. The Q, K, V matrices are just the result of that projection — one per role per head, not one per word.

Assume $d_{\text{model}} = 4$, head $= 2$. Show me how Q, K, V is calculated for above sentence example.

Setup first:

- $d_k = d_{\text{model}}/\text{heads} = 4/2 = 2$
- Each weight matrix: $d_{\text{model}} \times d_k = 4 \times 2$
- Resulting Q, K, V per head: 4×2 (4 words, each projected to $d_k = 2$)

Input matrix X (4×4) — one embedding row per word:

$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{matrix} \text{There} \\ \text{is} \\ \text{a} \\ \text{cat} \end{matrix}$$

(Using one-hot embeddings here to keep the arithmetic clean — in practice these would be dense learned vectors.)

Head 1

$$W_1^Q = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad W_1^K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad W_1^V = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 3 & 1 \end{bmatrix}$$

$$Q_1 = XW_1^Q = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad K_1 = XW_1^K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad V_1 = XW_1^V = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 3 & 1 \end{bmatrix}$$

Since X is one-hot, each row of Q simply picks the corresponding row of W_1^Q. For example, "There" $= [1, 0, 0, 0]$, so its query vector is just row 1 of $W_1^Q = [1, 2]$.

Head 2

$$W_2^Q = \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 0 \\ 4 & 2 \end{bmatrix}, \quad W_2^K = \begin{bmatrix} 0 & 1 \\ 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix}, \quad W_2^V = \begin{bmatrix} 1 & 3 \\ 2 & 0 \\ 0 & 1 \\ 1 & 2 \end{bmatrix}$$

$$Q_2 = \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 0 \\ 4 & 2 \end{bmatrix}, \quad K_2 = \begin{bmatrix} 0 & 1 \\ 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix}, \quad V_2 = \begin{bmatrix} 1 & 3 \\ 2 & 0 \\ 0 & 1 \\ 1 & 2 \end{bmatrix}$$

Summary of what you get:

	Head 1	Head 2
Q	4×2	4×2
K	4×2	4×2
V	4×2	4×2

Each head gets its **own independent** Q, K, V — computed from the same X but through different learned weight matrices. That is what allows each head to specialize in finding different relationships.

Each row of Q, K, V represents a word of the sentence. Correct?

Yes, correct.

Each row corresponds to one word's projected vector for that role. So for Q_1:

$$Q_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix} \quad \leftarrow \quad \begin{matrix} \text{There} \\ \text{is} \\ \text{a} \\ \text{cat} \end{matrix}$$

Row 1 = "There"'s query vector, row 2 = "is"'s query vector, and so on. Same structure applies to K and V.

In multihead self attention we get z. In the encoder I see the z goes into Add and Norm component. What happens here?

Two things happen in sequence: a **residual connection (Add)**, then **layer normalization (Norm)**.

Add — Residual Connection

Let's say the input to the Multi-Head Self-Attention was **X**, and the output was **z** (i.e., MHA(X)). The Add step computes:

$$X' = X + z$$

You're not replacing the original representation with the attention output — you're adding the attention output *on top of* the original. This preserves what the word already "knew" before attention, and layers in what attention learned about context.

The practical benefit: during training, gradients can flow directly back through the + shortcut, bypassing the attention sublayer entirely. This prevents the **vanishing gradient problem**, which is a serious issue in deep networks where learning signals shrink to near-zero as they travel backward through many layers.

Norm — Layer Normalization

After adding, layer normalization is applied to X'. It rescales the numbers in each word's representation so they sit in a stable, consistent range.

Without this, after many encoder blocks, values can drift very large or very small, making training unstable. Layer norm acts like a pressure valve — keeping the numbers well-behaved after every sublayer.

Where this sits in the full block

After this first Add & Norm, the output X' goes into the **Feed-Forward Network**. Then *another* Add & Norm is applied after the FFN for the same reasons. So every sublayer in the encoder block is wrapped in this same residual + normalize pattern.

List the main reasons of residual connection and layer normalization.

Residual Connection (Add)

- **Prevents vanishing gradients** — gradients can flow directly back through the + shortcut during training, without having to pass through the attention sublayer. Deep networks without this tend to have learning signals shrink to near-zero by the time they reach early layers.
- **Preserves original information** — the word's representation before attention isn't discarded. The attention output is added *on top*, so both the original and the new context-aware information are retained.
- **Makes deeper networks trainable** — stacking 6 (or more) encoder blocks would be very hard to train without residual connections. They're what make deep transformer stacks practically usable.

Layer Normalization (Norm)

- **Prevents numerical instability** — after repeated additions and transformations across many layers, values can drift very large or very small. Layer norm rescales them back to a controlled range after every sublayer.
- **Stabilizes training** — consistent value ranges mean the model learns more smoothly and reliably, without the loss spiking or collapsing.
- **Applied per token, independently** — unlike batch normalization (which normalizes across the batch dimension), layer norm normalizes across the feature dimension of each token individually. This works well for variable-length sequences where batch statistics are unreliable.

In the residual connection we add attention output on top of original representation. But original representation did not have attention in it. So doesn't it affect the output of attention badly?

Good challenge. The intuition behind the concern makes sense, but here's why it doesn't hurt:

Attention output is trained as an additive correction, not a replacement.

The model is trained end-to-end, knowing that the residual addition will happen. So MHA(X) doesn't try to be a *complete* new representation — it's optimized to produce *what needs to be added* to X to make it better. Think of it as a delta, not a full rewrite.

$$X' = X + \text{MHA}(X)$$

X carries: token identity + positional information.
$\text{MHA}(X)$ carries: contextual updates — what each word learned from attending to others.
X' carries both. Neither cancels the other.

The mixing is the point.

The original representation X already knows *what word this is* and *where it sits*. What it's missing is *how it relates to other words in context*. Attention fills exactly that gap. Adding them together gives you a representation that is both identity-aware and context-aware simultaneously.

If you discarded X and only kept $\text{MHA}(X)$, you'd actually risk losing precise token-level information — attention is a weighted average over all positions, which can blur individual token identity.

The network compensates automatically.

Since training is end-to-end, the attention layers learn to produce outputs that are *complementary* to X, not redundant or conflicting with it. If the original X already captures something well, the model learns to produce small corrections there. Where context matters more, attention contributes more. The balance is learned, not fixed.

In layer normalization we are keeping the numbers to a certain range. Let's say a vector is [2,4,5,1] and we turn it into [0.3, 0.45, 0.7, 0.1]. Isn't the vector loosing information when we are changing the numbers?

This is a sharp question, and the answer has two parts.

What layer norm actually preserves

The *relative structure* of the vector — which dimensions are large, which are small, and by how much — is fully preserved. The pattern of the vector doesn't change, only its scale and center shift.

So $[2, 4, 5, 1]$ normalized doesn't lose the fact that the 3rd element was the dominant one and the 4th was the weakest. That relationship survives intact.

What it does lose — and why that's okay

It does discard the absolute magnitude. After normalization, you can't tell if the original values were in the range $[1, 5]$ or $[100, 500]$.

But absolute magnitude of activations across layers carries no inherent semantic meaning. What the model encodes semantically lives in the *relative pattern* of the vector — which dimensions are more activated than others. That is preserved.

Layer norm has learnable parameters that compensate

After normalizing, layer norm applies two learnable parameters per dimension:

- γ **(scale)** — rescales each dimension
- β **(shift)** — shifts each dimension

These are trained alongside the rest of the model. So if a certain scale was actually important, the model learns to restore it through γ. The normalization step just puts everything into a clean, stable baseline — then γ and β reshape it into whatever the network actually needs.

In short: the structurally meaningful part (relative pattern) is preserved. The part that's lost (absolute scale) is either unimportant or recoverable through learned parameters.

Relative pattern of the vectors are important than absolute magnitude because when we are representing words with vectors we care about angular distance between two vectors to find out similarity or dissimilarity, not magnitude of the vectors. Is this the correct?

Yes, that reasoning is essentially correct.

In vector representations of words, similarity is measured by **cosine similarity** — which only depends on the angle between two vectors, not their magnitudes. So two vectors pointing in the same direction are considered identical in meaning regardless of whether their magnitudes are 1 or 1000.

This is exactly why absolute magnitude being lost in layer norm doesn't hurt — the semantically meaningful information lives in the *direction* of the vector (which dimensions dominate, and by how much relative to each other), and that directional structure is what layer norm preserves.

One small nuance to be precise:

Layer norm doesn't just rescale — it also **mean-centers** (subtracts the mean of the vector). So it's not purely angle-preserving in the strict mathematical sense the way L2 normalization would be. It shifts the vector too.

But the practical effect is the same: the relative relationships between dimensions — the pattern — survive, and the model can recover any needed scale through the learned γ and β parameters.

So your intuition is correct: the reason absolute magnitude is safe to discard is precisely because the representation system is built around angular distance, not Euclidean magnitude.

The picture of the decoder in transoformer architechture shows "Outputs (shifted right)". What does it mean?

It's about how the decoder receives its input during **training**.

The core idea: The decoder is trained to predict the next word at every position simultaneously. To do this without cheating, the target sequence is shifted one position to the right before being fed in.

Example — suppose the target (correct translation) is: `I am happy`

Decoder Input (shifted right)	Decoder Output (what it should predict)
`[START]`	`I`
`[START] I`	`am`
`[START] I am`	`happy`
`[START] I am happy`	`[END]`

So at each position t, the decoder sees all tokens *before* t and predicts token t. The shift guarantees that at no point does the model see the token it's trying to predict.

Why shift instead of just feeding the full target?

Without the shift, position t would have the ground truth token right there in the input — it could just copy it instead of learning to predict it. The shift enforces the rule: *predict the next token from everything before it.*

How "shifted right" is enforced mechanically:

The decoder's first sublayer is **Masked Multi-Head Attention**. The mask zeroes out attention weights to future positions, so even if somehow tokens were misaligned, the model still cannot attend to any position $\geq t$ when predicting at position t.

During inference (actual use), there is no target to shift — the decoder generates tokens one by one, feeding each newly generated token back in as the next input. The "shifted right" is a training-time concept; the autoregressive loop replaces it at test time.

During inference there are no given target. Are the inputs of the decoder still shifted?

During inference, "shifted right" doesn't apply as an explicit operation — but the **same structure holds naturally**.

Here's what happens step by step:

Step	Decoder Input	Predicts
1	`[START]`	`I`
2	`[START] I`	`am`
3	`[START] I am`	`happy`
4	`[START] I am happy`	`[END]`

At every step, the decoder receives only what it has already generated (starting with `[START]`), and predicts the next token. That's the same pattern as training — it's just happening one step at a time instead of all at once.

The key difference between training and inference:

- **Training:** The full target is available, so the model processes all positions *in parallel* using the shifted input + mask to simulate sequential generation.
- **Inference:** No full target exists, so the model genuinely runs *step by step*, appending each new prediction before the next step.

The "shifted right" label is a training-time description of how the input is prepared. During inference, that structure emerges automatically from the autoregressive loop — you don't need to explicitly shift anything because you're building the sequence one token at a time anyway.

So, during training the decoder generates all token parallelly? That means the sentence "I am happy" will be generated all at once?

Yes, exactly. During training, all three tokens are predicted in one forward pass.

The decoder receives `[START] I am happy` (the shifted target) and outputs predictions at all positions simultaneously:

Position	Input token seen	Predicts
1	`[START]`	`I`
2	`[START] I`	`am`
3	`[START] I am`	`happy`

All three predictions happen in parallel. The **mask** is what makes this safe — at position 2, even though `am` and `happy` are physically present in the input matrix, the mask blocks attention to them. So position 2 effectively only "sees" `[START] I`.

Why do it this way?

Inference is inherently sequential — you can't avoid it because you don't have the target. But training *doesn't have to be* sequential. You already know the full correct target, so you can simulate all steps at once using the mask. This makes training **much faster** — one forward pass computes the loss for every position rather than running the decoder T times for a sequence of length T.

This technique is called **teacher forcing** — feeding the ground truth tokens as input instead of the model's own previous predictions.

On page 14 of `4_Seq2seq_transformers.pptx.pdf` file, there are 6 encoders and 6 decoders. Let's name them e1 to e6 and d1 to d6. I see the output from e6 goes to all decoder from d1 to d6. And, d2 to d6 is also receiving input from their previous decoders. Why is that?

These are two separate things happening for two different reasons.

1. Why e_6's output goes to ALL decoders (d_1 through d_6)

Each decoder block has a cross-attention sublayer. Cross-attention needs K and V from the encoder, and Q from the decoder. The encoder's job is fully done after e_6 — it has produced the richest, most complete understanding of the source sentence. So e_6's output is converted into K and V matrices once, and those **same K and V are shared with every decoder block**.

Every decoder layer needs to ask "which part of the source sentence matters right now?" — so every layer needs access to the encoder output. d_1 uses it, d_2 uses it, ..., d_6 uses it. All of them, independently.

2. Why d_2–d_6 also receive input from their previous decoder

This is simply the **stacked architecture** — exactly like the encoder stack. The output of d_1 flows into d_2 as input, d_2's output flows into d_3, and so on. Each decoder block

refines the representation a bit more, just like $e_1 \to e_2 \to \cdots \to e_6$ did on the encoder side.

So d_n's input is the output of d_{n-1}, which it first runs through masked self-attention, then cross-attention (using e_6's K and V), then FFN — and passes the result up to d_{n+1}.

Summary in one line each:

- $e_6 \to$ all decoders: because **every decoder layer needs to cross-attend to the source**
- $d_{n-1} \to d_n$: because **decoders are stacked, each refining the representation further**

e1 receives unprocessed input. I understand that each layer above e1 refines what e1 has learned. But once the output of e6 goes into d1 and d1 receiving shifted target, shouldn't d1 already generate the output? What is d1 generating? And what does it pass to d2?

Great question, and the answer directly mirrors why the encoder needs 6 layers too.

d_1 **does not generate the final tokens.** No individual decoder block does. Token prediction (via linear + softmax) only happens after d_6's output. Every decoder block — d_1 through d_6 — outputs a **matrix of hidden representations**, not words.

Specifically, d_1 receives:

- The shifted target embeddings (e.g., `[START] I am happy` as vectors)
- e_6's K and V (for cross-attention)

And it outputs: a matrix of the **same shape** — one vector per target position — but now each vector has been enriched by:

1. Masked self-attention (target tokens attending to each other)
2. Cross-attention (target tokens attending to the source via e_6)
3. FFN (deeper non-linear transformation)

This enriched matrix is what gets passed to d_2.

So what is d_1 actually doing if not predicting words?

It's building a **preliminary contextual representation** of the target sequence in relation to the source. Think of it in layers of abstraction:

- d_1 might learn rough source-target alignments and basic syntactic dependencies
- d_3/d_4 refine those into more nuanced contextual relationships
- d_5/d_6 produce highly abstract representations ready for final prediction

This is identical to the encoder: e_1 sees raw embeddings and could theoretically output something, but it takes 6 layers of refinement to produce the rich H that the decoder actually uses.

One-line summary: d_1 outputs a refined hidden representation matrix (not words), passes it to d_2 as input, and this refinement continues until d_6 hands off to the linear + softmax for actual token prediction.

For what type of tasks transformer architecture was used in the original paper?

The original paper introduced the Transformer specifically for **sequence transduction** — tasks where you map an input sequence to an output sequence. The primary task was **machine translation** (English→German and English→French on the WMT 2014 benchmarks). They also demonstrated it generalizes to **English constituency parsing**, but that was secondary, just to show the architecture isn't translation-specific.

Let's say we are training a transformer to translate from English to Spanish. English sentence is "The dog sat" and Spanish is "El perro se sentó." Assume sixth encoder (e6) has given it's output to d1 (first decoder layer) and d1 has received positional embedding of target. Now show me concrete example of Masked multi head attention inside the decoder.

Setup

Tokens: Decoder receives `<SOS>`, "El", "perro" (3 tokens, simplified from the full target). During training, the **full target is fed at once** (teacher forcing), but masking enforces left-to-right causality — token at position i cannot attend to any position $j > i$.

Dimensions: $d_{\text{model}} = 4$, 2 heads $\rightarrow d_k = d_v = 2$

Input embeddings + positional encoding:

$$x_0\ (\texttt{<SOS>}) = [1,\ 0,\ 1,\ 0]$$
$$x_1\ (\texttt{El}) = [0,\ 1,\ 0,\ 1]$$
$$x_2\ (\texttt{perro}) = [1,\ 1,\ 0,\ 0]$$

Head 1 — attends to dims 0–1

Weight matrices project to first 2 dimensions, giving:

$$Q_1 = K_1 = V_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

Raw scores $= Q_1 K_1^\top$:

	k_0	k_1	k_2
q_0	1	0	1
q_1	0	1	1
q_2	1	1	2

Scale by $\frac{1}{\sqrt{2}} \approx 0.707$:

	k_0	k_1	k_2
q_0	0.71	0.00	0.71
q_1	0.00	0.71	0.71
q_2	0.71	0.71	1.41

Apply causal mask — future positions $\to -\infty$:

	k_0	k_1	k_2
q_0	0.71	$-\infty$	$-\infty$
q_1	0.00	0.71	$-\infty$
q_2	0.71	0.71	1.41

$-\infty$ becomes 0 after softmax — those positions are completely blocked.

Softmax row by row:

- `<SOS>` (row 0): only one finite value $\to [1.000,\ 0.000,\ 0.000]$
- `El` (row 1): $e^0 = 1,\ e^{0.71} \approx 2.034 \to [0.330,\ 0.670,\ 0.000]$
- `perro` (row 2): $e^{0.71} \approx 2.034,\ e^{0.71} \approx 2.034,\ e^{1.41} \approx 4.095 \to [0.249,\ 0.249,\ 0.502]$

Attention weights A_1:

	`<SOS>`	`El`	`perro`
`<SOS>`	1.000	0.000	0.000
`El`	0.330	0.670	0.000
`perro`	0.249	0.249	0.502

Output $Z_1 = A_1 \cdot V_1$:

$$\texttt{<SOS>}: \quad 1.000 \cdot [1,0] = [1.000,\ 0.000]$$

$$\texttt{El}: \quad 0.330 \cdot [1,0] + 0.670 \cdot [0,1] = [0.330,\ 0.670]$$

$$\texttt{perro}: \quad 0.249[1,0] + 0.249[0,1] + 0.502[1,1] = [0.751,\ 0.751]$$

Head 2 — attends to dims 2–3

Projecting to last 2 dimensions gives:

$$Q_2 = K_2 = V_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

"perro" maps to $[0,0]$ because its original dims 2–3 are both 0.

After scaling, masking, and softmax:

- `<SOS>`: $[1.000,\ 0.000,\ 0.000]$
- `El`: $[0.330,\ 0.670,\ 0.000]$
- `perro`: $[0.333,\ 0.333,\ 0.333]$ ← all three scores are 0 → uniform over visible tokens

Output Z_2:

$$\texttt{<SOS>} : [1.000,\ 0.000]$$
$$\texttt{El} : [0.330,\ 0.670]$$
$$\texttt{perro} : 0.333[1,0] + 0.333[0,1] + 0.333[0,0] = [0.333,\ 0.333]$$

Concatenate → Final Output

Concat $Z_1 \| Z_2$ (each $3 \times 2 \to 3 \times 4$), then multiply by W^O (using identity here):

$$\texttt{<SOS>} : [1.000,\ 0.000,\ 1.000,\ 0.000]$$
$$\texttt{El} : [0.330,\ 0.670,\ 0.330,\ 0.670]$$
$$\texttt{perro} : [0.751,\ 0.751,\ 0.333,\ 0.333]$$

What the mask actually enforced

Token	Could attend to	Could NOT attend to
`<SOS>`	only itself	El, perro
`El`	`<SOS>`, El	perro
`perro`	`<SOS>`, El, perro	—

This output now goes into **Add & Norm**, then flows into the **cross-attention** sub-layer where it will query the encoder output from e6.

Where exactly is the masking happening?

After scaling, before softmax.

You have the scaled score matrix:

$$\begin{bmatrix} 0.71 & 0.00 & 0.71 \\ 0.00 & 0.71 & 0.71 \\ 0.71 & 0.71 & 1.41 \end{bmatrix}$$

You add a mask matrix where future positions are $-\infty$ and current/past positions are 0:

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Element-wise addition gives:

$$\begin{bmatrix} 0.71 & -\infty & -\infty \\ 0.00 & 0.71 & -\infty \\ 0.71 & 0.71 & 1.41 \end{bmatrix}$$

Then softmax is applied. Because $e^{-\infty} = 0$, those positions contribute exactly zero to the weighted sum. They don't just get low attention — they get **none**.

So the full operation is:

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

For this same sentence let's say d6 (sixth decoder layer) has completed it's computations. Show me with a concrete example of what happens at Linear and Softmax blocks.

Setup

d6 output: one 4-dim vector per token position (3 tokens $\rightarrow 3 \times 4$ matrix)

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad \begin{matrix} \leftarrow \texttt{<SOS>} \\ \leftarrow \texttt{El} \\ \leftarrow \texttt{perro} \end{matrix}$$

Vocabulary (size 6): {`<SOS>`, `<EOS>`, `El`, `perro`, `se`, `sentó`}

Linear Layer

A learned weight matrix $W \in \mathbb{R}^{4\times 6}$ projects each d_{model} vector $\rightarrow$ one score per vocab word (logits):

$$W = \begin{bmatrix} 0.1 & 0.1 & \mathbf{2.0} & 0.1 & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.1 & \mathbf{2.0} & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.1 & \mathbf{2.0} & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.1 & 0.1 & 0.1 \end{bmatrix}$$

Logits $= H \cdot W$ (3×6):

	`<SOS>`	`<EOS>`	`El`	`perro`	`se`	`sentó`
`<SOS>`	0.1	0.1	**2.0**	0.1	0.1	0.1
`El`	0.1	0.1	0.1	**2.0**	0.1	0.1
`perro`	0.1	0.1	0.1	0.1	**2.0**	0.1

Softmax

Each row is converted independently to a probability distribution. For the `<SOS>` row:

$$e^{0.1} \approx 1.105, \quad e^{2.0} \approx 7.389$$

$$\text{sum} = 5 \times 1.105 + 7.389 = 12.914$$

$$P(\texttt{El} \mid \texttt{<SOS>}) = \frac{7.389}{12.914} \approx 0.572, \quad \text{all others} \approx 0.086$$

Same math applies to all rows:

	<SOS>	<EOS>	El	perro	se	sentó
<SOS>	0.086	0.086	**0.572**	0.086	0.086	0.086
El	0.086	0.086	0.086	**0.572**	0.086	0.086
perro	0.086	0.086	0.086	0.086	**0.572**	0.086

What Each Row Means

Each position is predicting the **next** token — this is the critical point:

Position	Predicts	Correct answer
<SOS>	→ El (0.572) ✓	El
El	→ perro (0.572) ✓	perro
perro	→ se (0.572) ✓	se

All three predictions happen **simultaneously** during training (that's why teacher forcing + masking is needed — you feed the full target at once but prevent each position from peeking ahead).

Loss

Cross-entropy loss is computed at each position against the true next token, then averaged:

$$\mathcal{L} = -\frac{1}{3}\left[\log(0.572) + \log(0.572) + \log(0.572)\right] \approx 0.557$$

This loss propagates gradients back through the entire network to update all weights, including W.

During training a transformer, is the loss calculated after the output has been generated?

No. During training, the decoder produces probability distributions for **all positions simultaneously** (as shown in the example — all 3 rows came out at once). Loss is computed over all those positions at once, not after a complete sentence is generated.

What you're thinking of is **inference**, where the model actually generates tokens one at a time — predict one token, feed it back in, predict the next, and so on until <EOS>. That's a sequential process.

During **training**:

- Full target is fed in at once via teacher forcing
- Masking enforces causality (no peeking ahead)
- All output distributions are computed in one forward pass
- Loss is computed over all positions simultaneously
- One backward pass updates all weights

The whole point of teacher forcing + masking is to **get the efficiency of parallel computation during training**, while still preparing the model for sequential generation at inference time.

Does it mean loss is calculated in all decoder layers?

No. Loss is calculated **only once** — after the final decoder layer (d6) passes through the Linear + Softmax block.

Layers d1 through d5 just transform representations and pass them forward. They don't produce vocabulary distributions, so there's nothing to compute loss against.

However, all layers still **receive gradients** during backpropagation. The loss at the end propagates backward through $d6 \to d5 \to \cdots \to d1$, updating every weight in the network. So the loss is computed once, but its effect reaches everywhere.

Don't the encoder layers receive gradients?

Yes, they do. Gradients flow from the loss all the way back through the decoder, then through the **cross-attention** sublayer (which connects decoder to encoder), and then back through all encoder layers $e6 \to e1$.

The encoder's output is used as K and V in every decoder layer's cross-attention. Since those values participated in computing the final loss, gradients flow back through them into the encoder weights.

So the full backward path is:

$$\mathcal{L} \to \text{Linear} \to d6 \to d5 \to \cdots \to d1 \to \text{cross-attention} \to e6 \to e5 \to \cdots \to e1$$

Everything that participated in the forward pass gets updated. I should have said the whole network, not just the decoder.

What is byte pair encoding?

BPE is a **tokenization algorithm** — it solves the problem of how to split text into units before feeding it into the model.

The core problem: If you tokenize by whole words, your vocabulary is massive and you can't handle unseen words. If you tokenize by characters, sequences become very long and the model has to learn everything from scratch. BPE sits in between.

The algorithm:

1. Start with every word split into individual characters. Add a special end-of-word symbol (e.g. $\cdot$).
2. Count all adjacent character pairs across the entire corpus.
3. Merge the most frequent pair into a single new symbol.
4. Repeat for a fixed number of merge operations (the only hyperparameter).

Concrete example:

Corpus contains: `low` (×5), `lower` (×2), `newest` (×6), `widest` (×3)

Initially: `l o w ·`, `n e w e s t ·`, etc.

Step	Most frequent pair	Merge
1	`e, s`	→ `es`
2	`es, t`	→ `est`
3	`l, o`	→ `lo`
4	`lo, w`	→ `low`

So `newest` becomes `new est ·` → eventually `newest·` as a single token, while `lower` becomes `low er·` — split into known subword units.

Why it matters for transformers:

- Rare words get broken into meaningful subword pieces instead of being `<UNK>`
- The model can handle words it never saw during training by composing them from known subwords
- Vocabulary stays compact and manageable

What is the hyper parameter?

The number of **merge operations**.

It controls how large your final vocabulary gets. Fewer merges → more character-level splits, longer sequences. More merges → larger chunks, shorter sequences, bigger vocabulary. You choose it before training based on your dataset size and language pair.

Give me a more concrete example of byte pair encoding from start to end. Mention the hyperparameter chosen for the example.

Setup

Corpus: 4 words with frequencies:

Word	Frequency
low	5
lower	2
newest	6
widest	3

Hyperparameter: number of merge operations = **10**

Step 0 — Initialize

Split every word into characters + end-of-word symbol `</w>`:

Word	Character sequence
low	`l o w </w>`
lower	`l o w e r </w>`
newest	`n e w e s t </w>`
widest	`w i d e s t </w>`

Step 1 — Count all adjacent pairs

Pair	Count
`e s`	$6+3=9$
`l o`	$5+2=7$
`o w`	$5+2=7$
`e w`	6
`s t`	$6+3=9$
...	...

Most frequent: `e s` and `s t` are tied at 9. Pick `e s`.

Merge `e s` → `es`**:**

Word	Sequence
low	`l o w </w>`
lower	`l o w e r </w>`
newest	`n e w es t </w>`
widest	`w i d es t </w>`

Step 2 — Count again

Most frequent: `es t` (appears $6+3=9$ times)

Merge `es t` → `est`**:**

Word	Sequence
newest	`n e w est </w>`
widest	`w i d est </w>`

Step 3

Most frequent: `l o` ($5+2=7$ times)

Merge `l o` → `lo`**:**

Word	Sequence
low	`lo w </w>`
lower	`lo w e r </w>`

Step 4

Most frequent: `lo w` ($5+2=7$ times)

Merge `lo w` → `low`**:**

Word	Sequence
low	`low </w>`
lower	`low e r </w>`

Step 5

Most frequent: `n e` (6 times)

Merge `n e` → `ne`**:**

Word	Sequence
newest	`ne w est </w>`

Step 6

Most frequent: `ne w` (6 times)

Merge `ne w` → `new`**:**

Word	Sequence
newest	`new est </w>`

Step 7

Most frequent: `new est` (6 times)

Merge `new est` → `newest`**:**

Word	Sequence
newest	`newest </w>`

Steps 8–10

Remaining merges handle lower-frequency pairs like `w i`, `w id`, `e r`, etc. in `lower` and `widest`. After all 10 merges, `lower` ends up as `low er </w>` since `er` never becomes frequent enough to merge with `low`.

Final Vocabulary

The learned vocabulary now includes: `l, o, w, e, r, n, s, t, i, d, </w>, es, est, lo, low, ne, new, newest, wi, wid, ...`

At Inference (unseen word)

Word `lowest` was never in training. BPE applies the **learned merge rules in order** to it:

`l o w e s t </w>` → `lo w e s t </w>` → `low e s t </w>` → `low es t </w>` → `low est </w>`

So `lowest` → `low` + `est` — two known subword units. The model can handle it without ever having seen it.

So using byte pair encoding does lowest becomes two word? low and est?

Yes, exactly. In BPE terminology they are called **subword units**, not words — but yes, `lowest` gets split into two tokens: `low` and `est`.

This is actually the key insight of BPE. The model never sees `lowest` as a whole, but it knows `low` from "low" and "lower", and it knows `est` from "newest" and "widest". So it can reason about `lowest` by composing meaning from those two familiar pieces.

How the number of merge operation is decided?

It's a hyperparameter, so it's decided by you before training, typically through **experimentation**. But there are practical guidelines:

- **Language complexity:** Morphologically rich languages (Turkish, Finnish, Arabic) need more merges because words carry more meaning through suffixes and prefixes. English needs fewer.
- **Vocabulary size target:** More merges → larger vocabulary, shorter sequences. The original BPE paper used **32,000–37,000** merge operations for machine translation, which is the most common starting point.
- **Dataset size:** Larger datasets can support more merges because rarer subword units still appear enough times to be meaningful.
- **Sequence length constraints:** More merges mean tokens are larger chunks, so sequences get shorter. If your model has a context length limit, this matters.

In practice, 32,000 is treated as the default for most NLP tasks, and you adjust up or down based on validation performance.

Which weights are shared among multiple components in a transformer and which are not shared? For example, consider below architechture of a transformer: 2 encoders stacked, 2 decoder stacked, 3 head multi head attention.

Weight Sharing in a Transformer (2 enc, 2 dec, 3-head MHA)

Weights that ARE shared

Only one case of true weight sharing exists in the standard transformer:

Embedding matrix ↔ Output linear projection

The original paper explicitly states that all three uses — encoder input embedding, decoder input embedding, and the final pre-softmax linear layer — share the same single weight matrix $W_E \in \mathbb{R}^{|V| \times d_{\text{model}}}$. This is called **weight tying**.

Weights that are NOT shared

Component	Distinct weight sets
e1 vs e2	Completely separate — self-attention (all heads), FFN, LayerNorm
d1 vs d2	Completely separate — masked self-attn, cross-attn, FFN, LayerNorm
Head 1 vs Head 2 vs Head 3 (within any single MHA)	Each head has its own W_Q^h, W_K^h, W_V^h
e1's self-attn vs d1's self-attn vs d1's cross-attn	All separate attention modules, all separate weights

The paper is explicit: "the linear transformations use different parameters from layer to layer."

Critical distinction: shared weights vs. shared tensor

The encoder output from e2 is passed into cross-attention in **both** d1 and d2 — but this is a shared **activation tensor**, not shared weights. Each decoder layer projects that same tensor using its own W_K^{cross} and W_V^{cross} matrices. The input is the same; what's done with it differs per layer.

Count for your architecture

In any single 3-head MHA module, you have:

- $3 \times (W_Q^h, W_K^h, W_V^h) =$ **9 projection matrices** (one set per head)
- $1 \times W_O =$ **1 output projection** (shared across all heads in that module)

Multiply that across e1, e2, d1 (masked self-attn + cross-attn), d2 (masked self-attn + cross-attn) — every one of those is independent. No weight sharing across layers whatsoever except the embedding/output tie.

So, weights used for encoder and decoder input embedding are same?

Yes — in the original transformer, the encoder input embedding and decoder input embedding use the **same weight matrix**.

This works because the paper uses a **shared vocabulary** (via BPE) between the source and target languages. Since both sides map tokens from the same vocabulary to vectors of the same d_{model}, one matrix handles both.

So the single matrix $W_E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ does three jobs:

1. Embeds encoder input tokens
2. Embeds decoder input tokens
3. Acts as the output linear projection (transposed)

Important caveat: This only works when source and target share a vocabulary. If you're translating between two languages with completely separate vocabularies (different scripts, no shared BPE tokens), you'd need separate embedding matrices for encoder and decoder — and weight tying between decoder input and output would still be possible, but not between encoder and decoder.

What is meant by source and target share a vocabulary?

When you train BPE on the **combined** text of both languages together, the resulting vocabulary contains subword tokens from both languages in a single unified list.

Example: Suppose you're doing English → French translation. You run BPE on a corpus of both English and French sentences mixed together. The resulting vocabulary might look like:

```
["the", "un", "ing", "tion", "re", "##ment", "play", "jouer", ...]
```

Some tokens are English, some French, some are subword pieces that appear in both (like `"tion"` which is common in both languages). This is one single vocabulary — both encoder and decoder index into the same list.

So token ID 47 means the same subword unit whether the encoder is reading it from the English input or the decoder is reading it from the French output.

Contrast with separate vocabularies:

If instead you build BPE separately — one vocab for English, one for French — then token ID 47 in the English vocab and token ID 47 in the French vocab are completely

unrelated. You can't use the same embedding matrix because the same index points to different things on each side.

The original transformer paper (English↔German, English↔French) used shared BPE vocabularies of ∼37K tokens, which is why weight tying across encoder embedding, decoder embedding, and output projection was possible.

The weights used for encoder and decoder input embedding was reused in the linear and softmax layer. Right?

Yes, exactly right.

The matrix $W_E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ is used in three places:

- **Encoder input embedding** — looks up a row of W_E for each token → produces a d_{model} vector
- **Decoder input embedding** — same lookup
- **Output linear layer** — multiplies the decoder's final d_{model} vector by W_E^T, producing $|V|$ logits, which then go through softmax

The embedding operation and the output linear operation are just transposes of each other. Embedding is $W_E \cdot$ one-hot (pick a row), the output linear is $W_E^T \cdot \mathbf{h}$ (project back to vocab size). Same weights, opposite direction.

In the final calculation of MHE, I saw W_O was multiplied with Z for projection. Is W_O and the weight used in linear softmax layer same?

No, they are completely different matrices.

- W_O lives *inside* the multi-head attention module. It takes the concatenated head outputs and projects them back to d_{model}. Shape: $\mathbb{R}^{h \cdot d_v \times d_{\text{model}}}$. Its job is to blend and resize the heads' combined output.
- **Output linear layer** (W_E^T) comes at the very end of the entire decoder stack, after all layers are done. It projects the final d_{model} vector to $|V|$ logits. Shape: $\mathbb{R}^{d_{\text{model}} \times |V|}$.

Different shapes, different positions in the network, completely different purposes. W_O is internal to attention; W_E^T is the final vocabulary projection. They are never shared.

For each attention head there are 3 weight matrices. Right?

Yes. Each head h has its own three projection matrices:

- $W_Q^h \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_K^h \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_V^h \in \mathbb{R}^{d_{\text{model}} \times d_v}$

So for your 3-head MHA, that's **9 projection matrices** per attention module, plus one shared W_O for that module.

W_O is shared among all heads?

Not exactly — "shared among heads" is a slight mischaracterization.

W_O doesn't operate on each head individually. The heads all finish their computations, their outputs get **concatenated** into one long vector, and then W_O is applied **once** to that concatenated result.

So W_O is a single matrix per attention module, but no individual head ever touches it. It only sees the combined output after all heads are done.

The flow is:

$$\text{Concat}(\text{head}_1, \text{head}_2, \text{head}_3) \xrightarrow{W_O} \text{MHA output}$$

Each head has its own W_Q^h, W_K^h, W_V^h. Then there is one W_O per attention module applied after concatenation.

In encoder we have two add and norm layers. Do these layers share same weights?

No, they have separate weights.

The **Add** step has no learnable weights — it's just a residual addition.

The **Norm** step (Layer Normalization) has two learnable parameters: a scale γ and a shift β, both of shape d_{model}.

Within a single encoder layer, the two LayerNorm instances — one after self-attention, one after FFN — each have their **own independent** γ and β. They are not shared.

And naturally, e1 and e2 also each have their own separate LayerNorm parameters. So in your 2-encoder architecture, you have **4 sets** of (γ, β) just for the encoder LayerNorms.

Then in a single decoder we have 3 sets of gamma and beta. Right?

Yes, exactly right.

Each decoder layer has three sublayers — masked self-attention, cross-attention, FFN — each followed by its own Add & Norm. So one decoder layer has **3 independent sets** of (γ, β).

In your 2-decoder architecture, that's **6 sets** of (γ, β) just for the decoder LayerNorms.

The feedforward network inside encoder and decoder, does it have its own weights?

Yes, each FFN has its own independent weights.

Each FFN consists of two linear transformations:

$$\text{FFN}(x) = \max(0,\ xW_1 + b_1)W_2 + b_2$$

So each FFN has 4 learnable components: W_1, b_1, W_2, b_2.

Every encoder and decoder layer has its own independent copy of these. The original paper is explicit: "the linear transformations use different parameters from layer to layer."

In your architecture (2 encoders + 2 decoders), you have **4 completely independent FFNs**, each with their own W_1, b_1, W_2, b_2.

In decoder there are two MHA, masked and unmasked. Do they share weights?

No, they have completely separate weights.

The masked self-attention and the cross-attention are two independent MHA modules, each with their own full set of weights:

- 3 heads $\times$ (W_Q^h, W_K^h, W_V^h) — separate for each module
- W_O — separate for each module

So within a single decoder layer, you have **2 completely independent sets** of all attention weights. And across d1 and d2, none of these are shared either.

Consider following transformer architechture: 1 encoder, 1 decoder, 1 head. Now give me the list of weights that are used in this architecture.

Shared (weight tying)

$W_E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ — encoder input embedding, decoder input embedding, and output linear projection

Encoder

Self-attention:

- $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_O \in \mathbb{R}^{d_v \times d_{\text{model}}}$

LayerNorm 1: γ_1, β_1

FFN: W_1, b_1, W_2, b_2

LayerNorm 2: γ_2, β_2

Decoder

Masked self-attention:

- $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_O \in \mathbb{R}^{d_v \times d_{\text{model}}}$

LayerNorm 1: γ_1, β_1

Cross-attention:

- $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_O \in \mathbb{R}^{d_v \times d_{\text{model}}}$

LayerNorm 2: γ_2, β_2

FFN: W_1, b_1, W_2, b_2

LayerNorm 3: γ_3, β_3

Positional encodings are fixed (sinusoidal) in the original paper — not learnable weights.

11 BERT

The encoder architecture used in BERT, is it the same architecture used for the transformer?

Yes, structurally — BERT reuses the exact Transformer **encoder** block: multi-head self-attention → add & norm → feedforward → add & norm, stacked N times. Same scaled dot-product attention, same residual connections.

A few differences worth noting:

1. **Positional encoding**: original Transformer uses fixed sinusoidal encodings; BERT uses **learned** positional embeddings.
2. **Segment embeddings**: BERT adds a third embedding (sentence A/B) summed with token + position embeddings — the original Transformer has no such thing since it's single-sequence-per-side.
3. **Activation**: original Transformer's FFN uses ReLU; BERT uses **GELU**.
4. **No decoder**: the original Transformer pairs its encoder with a decoder (for seq2seq translation) that also has cross-attention. BERT drops the decoder entirely and only uses stacked encoders, since it's not generating sequences autoregressively.

So the core computational block is identical, but the surrounding embedding scheme and the overall architecture (encoder-only vs encoder-decoder) differ.

Explain the third embedding (sentence A/B) summed with token + position embeddings.

BERT input representation is the elementwise sum of three embeddings, all of the same dimension (768 for BERT-base), so they can be added directly:

Token embedding — standard vocabulary embedding lookup.

Position embedding — learned vector per position index (0, 1, 2, ...).

Segment embedding — indicates which of the two sentences a token belongs to.

For tasks like NSP or QA, BERT's input is packed as two sentences:

```
[CLS] tok1 tok2 [SEP] tok3 tok4 tok5 [SEP]
  A    A    A    A     B    B    B    B
```

There are only **two possible segment vectors**, call them E_A and E_B, each a learned vector of size 768. Every token in sentence A gets E_A added to it; every token in sentence B (plus its `[SEP]`) gets E_B. The `[CLS]` token is also assigned segment A.

So for a token at position i in sentence A:

$$\text{input}_i = E_{\text{token}}(w_i) + E_{\text{pos}}(i) + E_A$$

For single-sentence tasks (e.g. plain classification, no sentence pair), every token just gets E_A — segment embeddings are still present but degenerate to a constant.

Why it's needed: self-attention has no inherent notion of sentence boundaries beyond the `[SEP]` token itself. The segment embedding gives every token an explicit, learned signal of "which side" it's on, which matters directly for NSP (predicting if B follows A) and later for tasks like question-answering (segment A = question, segment B = passage).

A concrete toy example of all three embeddings.

Toy setup: sequence `[CLS] the cat [SEP] sat down [SEP]`, with $d_{model} = 4$.

Assignments:

Token	Position	Segment
[CLS]	0	A
the	1	A
cat	2	A
[SEP]	3	A
sat	4	B
down	5	B
[SEP]	6	B

Made-up (toy) embedding vectors:

Token embeddings:

- $E_{tok}(cat) = [0.2, 0.9, 0.1, 0.0]$
- $E_{tok}(sat) = [0.3, 0.2, 0.4, 0.1]$
- $E_{tok}([SEP]) = [0.0, 0.0, 0.5, 0.5]$ — same vector both times it appears

Position embeddings:

- $E_{pos}(2) = [0.0, 0.2, 0.0, 0.1]$
- $E_{pos}(3) = [0.2, 0.0, 0.0, 0.2]$
- $E_{pos}(4) = [0.0, 0.1, 0.2, 0.0]$
- $E_{pos}(6) = [0.0, 0.2, 0.0, 0.0]$

Segment embeddings (only two exist, shared across all tokens in that segment):

- $E_A = [0.05, 0.05, 0.05, 0.05]$
- $E_B = [0.15, 0.15, 0.15, 0.15]$

Computing final input vectors (sum of the three):

"cat" (pos 2, segment A):

$$[0.2, 0.9, 0.1, 0.0] + [0.0, 0.2, 0.0, 0.1] + [0.05, 0.05, 0.05, 0.05] = [0.25, 1.15, 0.15, 0.15]$$

"sat" (pos 4, segment B):

$$[0.3, 0.2, 0.4, 0.1] + [0.0, 0.1, 0.2, 0.0] + [0.15, 0.15, 0.15, 0.15] = [0.45, 0.45, 0.75, 0.25]$$

The key illustrative point — the two `[SEP]` tokens have identical token embeddings but end up different after summation, because position and segment differ:

`[SEP]` at pos 3 (segment A):

$$[0.0, 0.0, 0.5, 0.5] + [0.2, 0.0, 0.0, 0.2] + [0.05, 0.05, 0.05, 0.05] = [0.25, 0.05, 0.55, 0.75]$$

`[SEP]` at pos 6 (segment B):

$$[0.0, 0.0, 0.5, 0.5] + [0.0, 0.2, 0.0, 0.0] + [0.15, 0.15, 0.15, 0.15] = [0.15, 0.35, 0.65, 0.65]$$

Same token, different final representation — this is exactly what lets the model distinguish "this is the separator ending sentence A" from "this is the separator ending sentence B," purely from the summed embedding, before any attention layer even runs.

Was BERT trained on a dataset where each sample contained two sentences?

Yes — every training example was a **pair** of text spans (A, B), packed together as `[CLS] A [SEP] B [SEP]`, used jointly for two pretraining objectives: **MLM** and **NSP**.

Key details on how pairs were constructed:

- **50% of the time**: B is the actual sentence that follows A in the original text (label = `IsNext`).
- **50% of the time**: B is a random sentence sampled elsewhere from the corpus (label = `NotNext`).

This label is exactly what the NSP objective trains the `[CLS]` output vector to predict.

Two important nuances:

1. **"Sentence" ≠ linguistic sentence.** In the BERT paper, "sentence" just means an arbitrary contiguous span of text — could be shorter or longer than an actual grammatical sentence. What matters is that it's a coherent chunk.
2. **Combined length constraint.** A and B together were capped at 512 tokens (BERT's max sequence length), so the sampling process trimmed one or both sides to fit.

Corpus used: BooksCorpus (˜800M words) and English Wikipedia (˜2,500M words, text passages only — tables, lists, and headers stripped out).

So the two-sentence structure wasn't incidental — it was baked into the pretraining data pipeline specifically to support NSP alongside MLM.

MLM and NSP are trained jointly not one after the other. What does it mean? How both are trained simultaneously?

"Jointly" means: for every single training example, **one forward pass** through the shared BERT encoder produces outputs used for *both* tasks simultaneously, and **one combined**

loss is computed and backpropagated in **one update step**. It's not "train MLM to convergence, then train NSP" — both signals push the same shared weights on every single batch.

Mechanism, step by step:

1. **Input**: one packed sequence `[CLS] A [SEP] B [SEP]`, with ˜15% of tokens masked (per the MLM procedure) and a known `IsNext`/`NotNext` label for the pair.
2. **Single forward pass** through the shared Transformer encoder produces contextual embeddings for every position.
3. **Two separate heads read from the same output**, but different positions:
   - **MLM head**: takes the output vectors at the *masked* token positions → feeds each through a linear layer + softmax over the vocabulary → predicts the original token.
   - **NSP head**: takes the output vector at the `[CLS]` position only → feeds it through a linear layer + softmax over 2 classes → predicts `IsNext`/`NotNext`.
4. **Combined loss** is just the sum:

   $$\mathcal{L} = \mathcal{L}_{MLM} + \mathcal{L}_{NSP}$$

   where $\mathcal{L}_{MLM}$ is averaged cross-entropy over all masked positions, and $\mathcal{L}_{NSP}$ is cross-entropy over the single `[CLS]` prediction.
5. **One backward pass**: gradients from both losses flow back through their respective heads and *converge* into the same shared encoder weights, since both heads sit on top of the same Transformer stack. The encoder gets updated by the combined gradient signal, not sequentially by one task then the other.

So "joint" specifically refers to steps 4–5: a single combined loss, single optimizer step, per batch — both tasks shaping the same shared parameters at the same time, rather than alternating phases or separate training runs.

In the BERT paper, each token was represented as a 768 size vector.

Not universally — the BERT paper introduced **two** model sizes, and the hidden size differs:

- **BERT-base**: hidden size = 768 (12 layers, 12 attention heads, ˜110M params)
- **BERT-large**: hidden size = 1024 (24 layers, 16 attention heads, ˜340M params)

So 768 is correct **only for BERT-base**. If you've mainly seen worked examples using 768, that's likely because BERT-base is the more commonly referenced/used variant in explanations and fine-tuning — but the paper itself specifies both sizes explicitly.

After token, positional, segment embedding each token remains a 768 size vector.

Right — for BERT-base. Token, position, and segment embeddings are each 768-dimensional vectors, and since they're summed elementwise (not concatenated), the result is still a single 768-dim vector per token. This summed vector is then passed

through LayerNorm and dropout before entering the first encoder layer — dimensionality stays 768 throughout the entire stack, layer to layer, all the way to the output.

After multi-head self-attention and FFN, is each encoder producing a hidden state of size 768 again?

Yes — every encoder layer preserves the 768 dimension, both at the attention output and the FFN output. This is a structural requirement, not a coincidence.

Why attention output is 768:

- Q, K, V are each split into 12 heads, each of size $768/12 = 64$.
- Each head produces a 64-dim output (weighted sum of V).
- All 12 heads' outputs are **concatenated**: $12 \times 64 = 768$.
- This concatenated vector passes through a final output projection matrix W^O (768×768) $\rightarrow$ still 768.
- Add residual (input + attention output) $\rightarrow$ LayerNorm $\rightarrow$ still 768.

Why FFN output is 768:

- The FFN is two linear layers: $768 \rightarrow 3072 \rightarrow 768$ (expansion factor of 4, with GELU in between).
- It expands to 3072 internally, but the second linear layer projects back down to 768.
- Add residual $\rightarrow$ LayerNorm $\rightarrow$ still 768.

Why it must stay 768 throughout: the residual connections (`x + Sublayer(x)`) require the sublayer's output to match the input's dimensionality exactly, so addition is even possible. This forces every layer's output — across all 12 (or 24) stacked encoder layers — to stay at 768 (or 1024 for BERT-large). The dimension only changes internally within the FFN's hidden expansion, never at the interface between layers.

One thing is confusing, if each token is 768 size vector and let's say in a sample we have 5 tokens. Then shouldn't the input become 768 * 5 size vector? And shouldn't the same happen for output of the encoder?

The confusion is between **concatenating vectors into one long vector** vs. **stacking them as rows in a matrix**. BERT does the latter, not the former.

The input is a matrix, not one flat vector.

With 5 tokens and $d_{model} = 768$, the input is a matrix of shape:

$$5 \times 768$$

Each **row** is one token's 768-dim vector. They sit side-by-side as separate rows, not concatenated end-to-end into a single $5 \times 768 = 3840$-dim vector. So token count and vector size are two *separate* dimensions of a 2D matrix, not multiplied into one number.

Why this matters for attention:

Self-attention needs each token to attend to *every other token individually* — computing similarity scores between each pair of tokens. That only makes sense if each token has its own separate 768-dim vector (a row) that can be compared against other rows via dot products. If everything were flattened into one 3840-dim vector, there'd be no way to isolate "token 3's vector" to compute attention scores with "token 1's vector" — the individual token boundaries would be lost.

Output of the encoder is also a matrix, same shape.

Every operation inside the encoder — attention, FFN, LayerNorm — is applied per-row (per-token position), and each row stays 768-dim throughout. So the encoder output is again:

$$5 \times 768$$

— five rows, each a 768-dim contextualized vector for that token, incorporating information from the other 4 tokens via attention.

Summary: sequence length (5) and hidden size (768) are independent axes of a matrix, not multiplied together. The number of tokens can vary per input; the hidden size (768) is fixed by the architecture and stays constant per token, at every layer.

In that case, let's say for high accuracy, I concatenate last four hidden layers, what would be the final size?

If you concatenate the last 4 hidden layers **along the feature dimension** (the standard approach from the BERT paper for feature-based use, e.g. NER), each token's vector grows by 4x:

$$4 \times 768 = 3072$$

So for your 5-token sequence, the result is a matrix of shape:

$$5 \times 3072$$

Each row is still one token, but now represented by a 3072-dim vector — the 768-dim hidden state from layer 9, 10, 11, and 12 (the last four) stacked side-by-side for that token. The number of tokens (5) is unaffected; only the per-token vector size changes, from 768 $\rightarrow$ 3072.

This is different from **summing** the last four layers (another option BERT's authors tested), which would keep the size at 768 per token instead of expanding it — concatenation preserves more information but costs 4x the dimensionality.

12 LLM

Page 3 - 7 of lecture `5_LLMs.pdf`

Page 3 — Decoder-only Generative Models

The slide establishes the architectural foundation of modern LLMs. You already know the full transformer has an encoder and a decoder. Decoder-only models **discard the**

encoder entirely — they only keep the decoder stack, removing the cross-attention sublayer (since there is no encoder output to attend to). What remains is: masked self-attention → feed-forward, repeated N times.

The key claim is that this stripped-down architecture is sufficient for **causal language modeling** — predicting the next token given all previous tokens. That task, trained at massive scale, turns out to be sufficient to produce human-like generation and emerge surprising capabilities beyond simple next-word prediction.

GPT is the canonical example. The "paper (4.2.3 and 4.2.4)" reference points to sections of the original "Attention Is All You Need" paper, which described the decoder design that GPT is directly built on.

Page 4 — Pretraining: The Training Objective

This slide lays out exactly how a decoder-only model is trained. The example makes it concrete:

Input: "The capital of Bangladesh is" → **Target:** "Dhaka"

Four concepts are defined:

Language Modeling — the goal is to maximize $P(\text{next token} \mid \text{previous tokens})$. The model outputs a probability distribution over the entire vocabulary at each position.

Cross-entropy loss — the predicted distribution is compared to a one-hot vector (all probability mass on the true next token). The loss penalizes how much probability was assigned elsewhere. The lower the probability assigned to the correct token, the higher the loss.

Teacher forcing — during training, even after the model makes a prediction, the *ground truth* token is fed as the next input, not the model's own prediction. This prevents error accumulation during training and enables parallelism — the entire sequence can be processed in one forward pass. You have seen this before; the slide is confirming it applies directly to GPT-style pretraining.

Autoregressive generation — at inference only, there is no ground truth to feed. So the model's own predicted token becomes the next input. This is sequential and cannot be parallelized.

The asymmetry — parallel during training, sequential during inference — is the defining operational split.

Page 5 — Pretraining: Tokenization

The slide maps which tokenization algorithm each major model uses:

BPE / Byte-level BPE — used by GPT-2, GPT-3, RoBERTa. You covered BPE before midterm. Byte-level BPE operates on raw bytes instead of Unicode characters, so it handles any language and avoids the unknown-token problem entirely — every possible byte sequence is representable.

WordPiece — used by BERT. Conceptually similar to BPE (iterative merging of subword units) but uses a different merge criterion: instead of selecting the most frequent

pair, it selects the pair that maximizes the likelihood of the training data under a language model. In practice they produce similar vocabularies.

SentencePiece — used by T5 and Llama. A language-agnostic framework that can implement either BPE or unigram language model tokenization. Its key difference: it treats the raw text (including whitespace) as-is, without language-specific pre-tokenization rules, making it portable across languages.

The practical takeaway: the tokenizer choice determines vocabulary structure, out-of-vocabulary behavior, and sequence length — all of which affect model performance. Different models made different tradeoffs here.

Page 6 — Pretraining: Training Data Sources

Where does the training text actually come from? Five categories:

Common Crawl — a nonprofit that continuously crawls the web and makes the data publicly available. Petabyte-scale. Extremely noisy — much of it is low-quality, duplicate, or toxic content, so heavy filtering is always applied before use.

Books — Project Gutenberg (public domain works) and less legitimate sources. Books provide long-form, coherent, high-quality prose — useful for learning to reason across long contexts.

Wikipedia — clean, factual, and well-structured. Small in volume relative to web crawls but very high signal.

Academic papers — ArXiv (preprints in math, physics, CS, ML) and PubMed (biomedical literature). Adds technical and scientific reasoning capability.

Code — GitHub repositories. Critical for code generation tasks, but also known to improve general reasoning ability.

The slide then makes two higher-level points:

Data quality > data quantity — this is not obvious but empirically well-established. Training on a smaller, cleaner corpus often outperforms training on a much larger dirty one. Filtering, deduplication, and quality scoring are where a lot of the real engineering effort goes.

Possible issues — diversity, quality, contamination — diversity means the corpus should cover many domains, languages, and writing styles; gaps here create capability gaps. Quality refers to noise and toxicity. *Contamination* is a specific and serious problem: if benchmark test sets (e.g., from standard NLP evaluation tasks) accidentally appear in the training data, the model effectively "memorizes" the answers, making benchmark scores unreliable measures of generalization.

Page 7 — Pretraining: Scaling Laws

This slide covers one of the most consequential empirical findings in LLM development, from **Hoffmann et al. 2022** (the Chinchilla paper — which is in your project files).

The prior belief (pre-Chinchilla): Following Kaplan et al. (2020), the field believed that when given more compute, you should primarily invest it in making the model *larger*. Training tokens were treated as a secondary concern.

The problem the slide identifies: Most models were *compute-inefficient* — they were too large relative to how much data they were trained on. A model with more parameters than its training data can justify is *overparameterized*: it has the capacity to learn more, but there is no signal left to learn from.

Chinchilla's insight: Model size and training tokens should scale *equally*. If you double the model size, you should also double the number of training tokens. The prior recommendation (Kaplan et al.) dramatically underweighted training data.

The practical demonstration: Chinchilla has 70 billion parameters — about 4× smaller than Gopher (280B parameters), which was DeepMind's prior model. By training Chinchilla on significantly more tokens while using the same compute budget, it outperformed Gopher on most benchmarks despite being much smaller.

The implication: **a smaller model trained on more data beats a larger model trained on less data, at the same compute cost.** This reframed how the field thinks about resource allocation during pretraining.

Explain GPT architecture (page 8 of `5_LLMs.pdf`)

The slide shows the full transformer on the left for reference, and the **GPT architecture on the right**. Here is what is happening:

GPT is Decoder-Only

GPT takes only the **decoder stack** from the original transformer and discards the encoder entirely. Since there is no encoder, it also eliminates cross-attention. What remains per block:

Masked Multi-Head Self-Attention → residual add → Layer Norm → Feed Forward → residual add → Layer Norm

This block repeats **12**× (in GPT-1). Each token can attend only to itself and tokens before it — never future tokens — enforced by the causal mask.

Bottom-Up Walkthrough

Text & Position Embed — input tokens are embedded into vectors, then positional information is added (same as the standard transformer).

Masked Multi Self-Attention — the core of each block. Self-attention here means Q, K, V all come from the same sequence, not from a separate encoder output. The mask sets future positions to $-\infty$ before softmax, so they contribute zero weight — the model genuinely cannot see what comes next.

Residual Add + Layer Norm (×2 per block) — applied after attention and again after the feed-forward. This is Post-LN ordering (same as the original paper): sublayer first, then add, then normalize.

Feed Forward — position-wise MLP applied identically to each token's representation. Expands dimensionality, applies nonlinearity, projects back.

Two Output Heads at the Top

The slide shows two heads branching off the final layer:

- **Text Prediction** — the standard language model head. Projects the hidden state through a linear layer + softmax to predict the next token. This is the pretraining objective.
- **Task Classifier** — added during fine-tuning for downstream classification tasks (sentiment, entailment, etc.). Takes the representation of a special delimiter token and classifies it.

During pretraining, only the text prediction head is active. During fine-tuning, the task classifier is added on top and the whole network is updated end-to-end.

Why Discard the Encoder?

GPT's goal is **language modeling** — predict the next token given all previous tokens. That is inherently a left-to-right, single-sequence task. You do not need a separate encoder producing a compressed source representation (that is only needed for seq2seq tasks like translation). The decoder stack alone, with causal masking, is exactly sufficient.

What is position-wise MLP in Feed Forward?

"Position-wise" means the **same MLP is applied independently to each token's vector**, one position at a time, with no interaction between positions.

If your sequence has tokens $[x_1, x_2, \ldots, x_T]$, each $x_t \in \mathbb{R}^{d_{\text{model}}}$, the feed-forward sub-layer does:

$$\text{FFN}(x_t) = \max(0,\ x_t W_1 + b_1)\ W_2 + b_2$$

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$ — expands to a larger hidden dimension (typically $d_{ff} = 4 \times d_{\text{model}}$)
- ReLU (or GELU in GPT) applies the nonlinearity
- $W_2 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$ — projects back down

The key point: W_1 and W_2 are **shared across all positions** — every token goes through the exact same weight matrices — but each token is processed **independently** with no information flow between positions at this stage. Token x_3 has no idea what x_1 or x_2 look like when it is being processed by the FFN.

Why is this useful if tokens do not interact?

Token interaction already happened in the attention sub-layer. By the time a token's vector reaches the FFN, it has already absorbed context from other tokens via attention weights. The FFN's job is to **transform that individual representation** — apply learned nonlinear functions to refine it — not to mix information across positions again. Think of attention as the "communication" step and the FFN as the "computation" step.

What happens inside linear layer in text prediction once hidden state is passed to linear layer?

The linear layer projects the final hidden state $h \in \mathbb{R}^{d_{\text{model}}}$ to a vector of **logits** $\in \mathbb{R}^{|V|}$, one score per vocabulary token:

$$\text{logits} = hW_E^T + b$$

where $W_E^T \in \mathbb{R}^{d_{\text{model}} \times |V|}$ — this is the **transposed embedding matrix** (weight tying, same as the standard transformer). Each logit is a raw, unnormalized score representing how likely that vocabulary token is to come next.

Softmax then converts these logits to a probability distribution:

$$P(\text{next token} = v) = \frac{e^{\text{logit}_v}}{\sum_{v'} e^{\text{logit}_{v'}}}$$

The token with the highest probability is the model's prediction for the next position.

Concretely: if $d_{\text{model}} = 768$ and $|V| = 50{,}257$ (GPT-2), then the linear layer is a $768 \times 50{,}257$ matrix multiplication — every hidden dimension contributes to scoring every word in the vocabulary simultaneously.

A concrete example of above projection in linear layer.

Setup: $d_{\text{model}} = 4$, vocabulary of 5 tokens.

Hidden state from the final decoder layer:

$$h = [0.5,\ -0.2,\ 0.8,\ 0.1]$$

Embedding matrix W_E (5 tokens $\times$ 4 dims) — each row is a token's learned embedding:

Token	dim1	dim2	dim3	dim4
cat	1.0	0.0	0.5	0.2
dog	0.0	1.0	0.3	−0.1
sat	0.5	−0.5	1.0	0.0
mat	0.2	0.1	0.8	0.3
the	−0.3	0.2	0.1	0.5

Projection: $\text{logit}_v = h \cdot W_E[v]$ — dot product of h with each row:

$$\begin{aligned}
\text{logit}_{\text{cat}} &= (0.5)(1.0) + (-0.2)(0.0) + (0.8)(0.5) + (0.1)(0.2) = 0.92 \\
\text{logit}_{\text{dog}} &= (0.5)(0.0) + (-0.2)(1.0) + (0.8)(0.3) + (0.1)(-0.1) = 0.03 \\
\text{logit}_{\text{sat}} &= (0.5)(0.5) + (-0.2)(-0.5) + (0.8)(1.0) + (0.1)(0.0) = 1.15 \\
\text{logit}_{\text{mat}} &= (0.5)(0.2) + (-0.2)(0.1) + (0.8)(0.8) + (0.1)(0.3) = 0.75 \\
\text{logit}_{\text{the}} &= (0.5)(-0.3) + (-0.2)(0.2) + (0.8)(0.1) + (0.1)(0.5) = -0.06
\end{aligned}$$

Logits: $[0.92,\ 0.03,\ 1.15,\ 0.75,\ -0.06]$

Softmax converts to probabilities (sum of $e^{\text{logit}} = 9.756$):

Token	Logit	Probability
cat	0.92	0.257
dog	0.03	0.106
sat	**1.15**	**0.324**
mat	0.75	0.217
the	−0.06	0.097

"sat" wins — the model predicts it as the next token.

Intuition behind the dot product: a high logit for a token means h is geometrically close (in direction) to that token's embedding row. The hidden state h encodes "what should come next" and the dot product measures how well each vocabulary embedding matches that signal.

Comprehensive explanation of page 9–13 of `5_LLMs.pdf`. Go through all project files thoroughly to gather context.

Page 9 — GPT-1: Fine-tuning on Downstream Tasks

GPT-1 is pretrained as a language model (predict next token), but the goal is to make it useful on **labeled downstream tasks** without training a separate model for each. The strategy: format every task's input as a flat token sequence that GPT can consume, add a lightweight task-specific linear head on top, then fine-tune the whole network end-to-end.

Four tasks are shown, each using three special tokens — **[Start]**, **[Delim]** (delimiter), **[Extract]** — to structure the input:

Classification — single input, single label (e.g., sentiment):

[Start] → Text → [Extract] → Transformer → Linear

The [Extract] token's final hidden state is passed to the linear classifier.

Entailment — does a hypothesis follow from a premise?

[Start] → Premise → [Delim] → Hypothesis → [Extract] → Transformer → Linear

The delimiter tells the model where one text ends and the other begins. The model must learn the relationship between them.

Similarity — is Text A similar to Text B? Order should not matter, so both orderings are fed separately through the same transformer, their outputs are **added** (+), then passed to a linear layer. This makes the representation symmetric.

Multiple Choice — one context paired with N answer candidates: [Start] → Context → [Delim] → Answer$_i$ → [Extract] — one sequence per answer, each producing its own linear score. A softmax over all N scores selects the answer.

The key architectural insight: **the transformer weights are shared across all tasks**. Only the linear head on top is task-specific. This is what made GPT-1's approach broadly applicable — you pretrain once on language modeling, then fine-tune cheaply.

Page 10 — GPT-2: Scale Without Fine-tuning

GPT-2 is architecturally identical to GPT-1 but scaled by roughly 10× in both parameters (120M → 1.5B) and training data.

The training data is **WebText**: pages scraped from URLs that appeared in Reddit posts with at least 3 karma upvotes. The 3-karma filter acts as a proxy for human quality judgment — links people found worthwhile. This produced 45 million links, 40 GB of text.

The central claim of GPT-2 is that **fine-tuning is unnecessary** if the model is large and diverse enough. A sufficiently capable language model implicitly learns to do many tasks just from the distribution of text on the internet — news articles implicitly teach summarization, Q&A forums teach question answering, and so on. GPT-2 demonstrated this with zero-shot performance, though not yet at a level that outperformed fine-tuned task-specific models.

GPT-2 was also the **last GPT model released publicly in full** — OpenAI's stance on public release changed from GPT-3 onward.

Page 11 — GPT-3: Scale as the Paradigm Shift

GPT-3 pushes GPT-2's architecture to 175 billion parameters — another 100× scale-up — and trains on 300 billion tokens drawn from CommonCrawl, WebText, English Wikipedia, and two books corpora (570 GB of plaintext total). The scale makes it accessible only via API, not as downloadable weights.

Three key contributions:

In-context learning — GPT-3 can solve tasks from just a few examples placed in the prompt (few-shot), one example (one-shot), or none (zero-shot), without any gradient updates. The examples in the prompt act as implicit task specifications. This works because a large enough language model learns to recognize and continue patterns, including the pattern "input → output" that task examples establish.

GPT-3.5 / ChatGPT — The slide notes that GPT-3.5 made the current LLM boom possible. GPT-3.5 is a fine-tuned version of GPT-3 using **RLHF** (Reinforcement Learning from Human Feedback), formalized in the InstructGPT paper. RLHF trains a reward model on human preference data, then uses RL (specifically PPO) to further tune the language model to maximize that reward. The result is a model that follows instructions rather than just completing text — this is what became ChatGPT.

The cost — 355 GPU-years and an estimated $4.6M to train once. This is directly relevant to the Chinchilla paper (`training_llm_paper.pdf`): the Chinchilla authors argue GPT-3 is *undertrained* given its parameter count — they show that for a given compute budget, you should train a smaller model on more tokens rather than a larger model on fewer tokens. A compute-optimal GPT-3-scale training run would use far fewer parameters and far more tokens.

Page 12 — GPT-4 and Beyond

GPT-4 introduces three major shifts relative to GPT-3:

Multimodality — GPT-4 accepts image inputs in addition to text, meaning the input embedding layer is extended to handle visual tokens (typically through a separate vision

encoder whose outputs are projected into the same space as text embeddings).

Instruction-tuning + RLHF — Both human feedback (RLHF) and AI feedback (RLAIF — using another model to generate preference labels) are used to align the model. This is what "reinforcement learning from human and AI capabilities" means on the slide.

Larger context window — Up to 32k tokens (vs GPT-3's 4k). This requires either extrapolating positional encodings beyond training length or switching to positional encoding schemes like RoPE (Rotary Position Embedding) or ALiBi that generalize better to longer sequences than the original sinusoidal encodings.

Page 13 — LLaMA Model Family

LLaMA is Meta AI's answer to the closed-model problem. It is **decoder-only** like GPT, but released with weights publicly available, making it the backbone of most open-source LLM work.

The table compares LLaMA 1 and LLaMA 2 across several axes:

	LLaMA 1	LLaMA 2
Sizes	7B, 13B, 33B, 65B	7B, 13B, 34B, 70B
Context length	2k	4k
Training tokens	1.0T–1.4T	2.0T
GQA	× all sizes	× for 7B, 13B ✓ for 34B, 70B

GQA — Grouped Query Attention is the most technically interesting entry. Standard multi-head attention has h separate Q, K, V heads — all distinct. GQA sits between that and Multi-Query Attention (MQA): it groups the h query heads into g groups, with each group sharing one K head and one V head. For example, with 32 query heads and 8 groups, you have 32 Q projections but only 8 K and 8 V projections.

Why does this matter? The K and V matrices are what gets cached during autoregressive generation (the **KV cache**). At inference time, every new token needs the keys and values of all previous tokens in context. With a 70B model and a long sequence, this cache becomes enormous. GQA reduces K and V memory by a factor of h/g, dramatically reducing memory bandwidth and enabling longer contexts at scale. LLaMA 2 only enables GQA for the 34B and 70B models where this pressure is most acute.

The training data for LLaMA 2 (2.0T tokens from publicly available sources) also reflects Chinchilla-optimal thinking — more tokens per parameter than GPT-3 used, consistent with the finding that the field had been systematically undertraining large models.